

ETC

Indice

1	Nozione di Base ETC	7
1.1	Insiemi	7
1.2	Logica Booleana	12
1.3	Dimostrazioni	14
1.4	Grafi	15
1.5	Alfabeto	16
2	Automi Finiti Deterministici	21
2.1	Usi DFA	21
2.2	Parti DFA	21
2.3	Definizione Formale DFA	22
2.4	Relazione fra DFA e Linguaggi	22
2.5	Dal DFA al Linguaggio Riconosciuto	26
2.6	Linguaggi Regolari	31
2.7	Calcolo Correttezza Automa	33
2.8	Esempio Tecnica Progettazione Diretta DFA senza Calcolo Correttezza	34
2.9	Chiusura Linguaggi Regolari e Dimostrazione Correttezza DFA	35
2.10	Teorema 1.25 Chiusura Classe REG Unione	36
2.11	Teorema 1.25 Bis Chiusura Classe REG Intersezione	38
2.12	Teorema (4.5 HUM) Chiusura Classe REG Complemento ..	45
2.13	Teorema 1.26 Chiusura Classe REG Concatenazione	45
2.14	Organizzazione Riposta DFA con Teorema Chiusura	45
3	Automi Finiti Non Deterministici	46
3.1	Linguaggio Accettato o Riconosciuto NFA	48
3.2	Semplici NFA	48
3.3	Esempio NFA	48
3.4	Definizione Formale NFA	51
3.5	Epsilon-Chiusura (HUM 2.5.3-2.5.4)	51
3.6	Funzione di Transizione Estesa NFA	53

3.7	Teorema 1.45 Chiusura Classe REG Unione NFA	55
3.8	Teorema 1.47 Chiusura Classe REG Concatenazione NFA .	56
3.9	Teorema 1.49 Chiusura Classe REG Chiusura di Kleene . . .	57
3.10	Chiusura Classe REG Reverse	58
3.11	Chiusura Classe REG Prefissi e Suffissi	60
4	Ogni NFA è un DFA e Viceversa	62
4.1	Ogni DFA è un NFA	63
4.2	Teorema 1.39 NFA e DFA equivalenti	64
5	Espressioni Regolari	66
5.1	Definizione Formale Espressioni Regolari	66
5.2	Precedenza Operatori	66
5.3	Definizione Ricorsiva Linguaggi Rappresentati REG	66
5.4	Proprietà Algebriche RE	67
6	Pumping Lemma	68
6.1	Tecnica Per Provare la Non Regolarità	68
7	Macchine di Turing	69
7.1	Descrizione Informale	69
7.2	Definizione Formale MdT	69
7.3	Funzione di Transizione MdT	69
7.4	Diagramma di Stato	69
7.5	Computazione di una MdT Informale	70
7.6	Configurazione di una MdT	70
7.7	Passo di Computazione	71
7.8	Computazione	71
7.9	Parola Accetta da una MdT	72
7.10	Parola Rifiutata da una MdT	72
7.11	Linguaggio Riconosciuto da una MdT	73
7.12	MdT Decisore	73
7.13	Linguaggio Deciso	73
7.14	Linguaggio Riconoscibile	74
7.15	Linguaggio Decidibile	74
8	Varianti di MdT	75
8.1	Macchine di Turing che Stanno Ferme	75
8.2	Macchine di Turing Multinastro	77

8.3	Equivalenza MdTM e MdT	79
8.4	MdTM e MdT Linguaggi Riconoscibile	80
8.5	Ordine Radix	81
9	Macchina di Turing Non deterministica	82
9.1	Descrizione Formale	82
9.2	Albero delle computazioni	83
9.3	Linguaggio Riconosciuto da una MdT non Deterministica ..	83
10	Equivalenza Modello Deterministico e non Deterministico	84
10.1	Idea della Prova	84
10.2	Rappresentazione delle Computazioni	85
10.3	Rappresentazione dei Cammini	86
10.4	Descrizione dei 3 Nastri	86
10.5	Descrizione di D	86
10.6	Corollario Macchina di Turing non Deterministica	87
10.7	Decisori non Deterministici	88
11	Problemi di Decisione	90
11.1	Richiami di Logica	90
11.2	Istanze Problema di Decisione	91
11.3	Problemi di Ricerca	91
11.4	Livelli di Descrizione di una Macchina di Turing	92
11.5	Codifiche	92
11.6	Codifica di una MdT	92
11.7	Linguaggio Associato a un Problema di Decisione	94
11.8	Problemi di Decisione nella Teoria degli Automi	94
11.9	Problema del Equivalenza di due DFA	95
12	Metodo della Diagonalizzazione	95
12.1	Funzione	95
12.2	Funzione Iniettiva	95
12.3	Funzione Suriettiva	95
12.4	Funzione Biettiva	95
12.5	Cardinalità	96
12.6	Numerabilità	98
12.7	Metodo della Diagonalizzazione Linguaggi non Turing riconoscibili	100

12.8	Esistono dei Linguaggi non Turing Riconoscibili	108
13	Indecidibilità	109
13.1	Linguaggio Indecidibile	109
13.2	Macchina di Turing Universale	109
13.3	Teorema Esistenza MdT Universale	109
13.4	A_{TM} è Turing Riconoscibile	110
13.5	A_{TM} è Indecidibile	111
13.6	Linguaggi co-Turing Riconoscibili	112
13.7	La Classe dei Linguaggi Decidibili è Chiusa Rispetto al Complemento	113
13.8	Linguaggio Decidibile Se e Solo se Turing e co-Turing Riconoscibile	114
13.9	$\overline{A_{TM}}$ non è Turing Riconoscibile	115
13.10	Chiusura Linguaggi Turing Riconoscibili Rispetto al Complemento	115
14	Riducibilità	116
14.1	Esempio Descrizione Informale	116
14.2	Riducibilità Descrizione Informale	117
14.3	Richiamo di Logica	118
14.4	Riduzione Mediante Funzione	119
14.5	Teorema 1 $A \leq_m B$ se e solo se $\overline{A} \leq_m \overline{B}$	119
14.6	Teorema 5.22 $A \leq_m B$ e B è Decidibile, allora A è Decidibile	120
14.7	Teorema 5.28 $A \leq_m B$ e B è Turing Riconoscibile, allora A è Turing Riconoscibile	121
14.8	Corollario Se $A \leq_m B$ e A è Indecidibile, allora B è Indecidibile	121
14.9	Corollario Se $A \leq_m B$ e A non è Turing Riconoscibile, allora B non è Turing Riconoscibile	122
14.10	Funzioni Calcolabili	122
14.11	$\{ab\} \leq_m A_{TM}$	125
14.12	$A_{TM} \leq_m HALT_{TM}$	126
14.13	$HALT_{TM}$	127

14.14	$A_{\text{TM}} \leq_m \overline{E_{\text{TM}}}$	127
14.15	Teorema 5.27 $\overline{E_{\text{TM}}}$	128
14.16	Corollario E_{TM}	128
14.17	$A_{\text{TM}} \leq_m \text{REGULAR}_{\text{TM}}$	128
14.18	Teorema $\text{REGULAR}_{\text{TM}}$	129
14.19	$E_{\text{TM}} \leq_m EQ_{\text{TM}}$	130
14.20	Teorema EQ_{TM}	130
14.21	$A_{\text{TM}} \leq_m EQ_{\text{TM}}$	131
14.22	$A_{\text{TM}} \leq_m \overline{EQ_{\text{TM}}}$	132
14.23	Teorema $A \leq_m B$ se e solo se $\overline{A} \leq_m \overline{B}$	133
14.24	Corollario 4.23 $\overline{A_{\text{TM}}}$ non è Turing Riconoscibile	133
14.25	Teorema EQ_{TM} non è Nè Turing Riconoscibile e Nè co-Turing Riconoscibile	133
14.26	Teorema di Rice	134
15	Complessità di Tempo	135
15.1	Definizione Complessità di Tempo	135
15.2	Astrazioni	135
15.3	Analisi Asintotica	136
15.4	Classi di Complessità	136
15.5	Osservazione sulla Complessità di Tempo	136
15.6	Relazione fra Modelli: MdT Multinastro	137
15.7	Tempo di Esecuzione MdT non Deterministica	138
15.8	Relazione fra Modelli: MdT non Deterministica	138
15.9	La Classe P: Tempo Polinomiale	138
15.10	Teorema Classe P	139
15.11	Tipi di Problemi	142
15.12	Classe EXPTIME	142
15.13	HAMPATH	142
15.14	COMPOSITES	143
15.15	Algoritmo di Verifica	143
15.16	Complessità degli Algoritmi di Verifica	144
15.17	NP	145
15.18	Classe Di Complessità in Tempo non Polinomiale	145

15.19	Teorema 7.20 Linguaggio in NP	145
15.20	Corollario 7.22	145
15.21	CLIQUE	145
15.22	SUBSET-SUM	147
15.23	$P \subseteq NP$	148
15.24	Una Gerarchia di Classi	149
15.25	Chiusura Classe P Rispetto al Complemento	149
15.26	coNP	149
15.27	Funzioni Calcolabili in Tempo Polinomiale	150
15.28	Riduzioni di Tempo Polinomiali	150
15.29	Riducibilità in Tempo Polinomiale	151
15.30	Proprietà Transitiva di $\leq_P$	152
15.31	Richiami di Logica	153
15.32	SAT	155
15.33	3SAT è Riducibile in Tempo Polinomiale a CLIQUE	155
15.34	Definizione Linguaggio NP-Completo	157
15.35	Teorema NP-Completo	157
15.36	Riducibilità Polinomiale e NP-Completezza	158
15.37	Strategia per Mostrare che B è NP-Completo	158
15.38	Teorema di Cook-Levin	159
15.39	Teorema SAT_{CNF} è NP-Completo	159
15.40	Teorema 3SAT è NP-Completo	159
15.41	CLIQUE è NP-Completo	160
15.42	VERTEX-COVER	160

1 Nozione di Base ETC

1.1 Insiemi

Gli insiemi sono collezioni non ordinate di oggetti o elementi distinti

- Ordine e ridondanza non contano
- **Insieme universale** U è l'insieme di tutti gli elementi pertinenti al contesto

1.1.1 Metodi di rappresentazione

- Insieme finito può essere descritto dalla lista dei suoi elementi separati da virgola tra {}
- Insieme infinito può essere descritto mediante la notazione “...”
 - $N = \{0, 1, 2, \dots, n, \dots\}$
- Altro modo per definire un insieme è di specificare la proprietà caratteristica dei suoi elementi
 - $A = \{w \mid w \text{ ha la proprietà } P\}$

1.1.1.1 Esempio di rappresentazione

- L'insieme dei numeri interi non negativi pari è
 - $\{0, 2, 4, \dots, 2n, \dots\} = \{2n \mid n \in N, n \geq 0\}$
- L'insieme dei numeri interi positivi pari è
 - $\{2, 4, \dots, 2n, \dots\} = \{2n \mid n \in N, n \geq 1\}$
- L'insieme dei numeri interi non negativi dispari è
 - $\{1, 3, 5, 7, \dots, 2n + 1, \dots\} = \{2n + 1 \mid n \in N, n \geq 0\}$

1.1.2 Definizione Ricorsiva (o Induttiva)

Una definizione ricorsiva di un insieme è composta da:

- Passo base: Definisce uno o più elementi elementari dell'insieme
- Passo ricorsivo: Definisce regole per formare nuovi elementi dell'insieme a partire da quelli già definiti

1.1.2.1 Esempio Definizione Ricorsiva

Sia A l'insieme definito ricorsivamente nel modo seguente

- Passo base: $i_1 = 1 \in A$.
- Passo ricorsivo: Se $i_k \in A$ allora $i_{k+1} = i_k + 2 \in A$.

1.1.3 Cardinalità

La cardinalità $|S|$ di un insieme finito S è il numero di elementi in S

- Insieme vuoto $\emptyset$ ha cardinalità 0

1.1.3.1 Lemma

Se S e T sono finiti allora $|S \cup T| = |S| + |T| - |S \cap T|$

- Se S e T sono disgiunti allora $|S \cup T| = |S| + |T|$

1.1.4 Sottoinsieme

Siano S e T insiemi, diciamo che $S \subseteq T$ se $w \in S$ implica $w \in T$

- Ogni elemento di S è anche un elemento di T

1.1.4.1 Esempio Sottoinsieme

- $S = \{ab, ba\}$ e $T = \{ab, ba, aaa\}$ allora $S \subseteq T$ ma $T \not\subseteq S$
- $S = \{ba, ab\}$ e $T = \{aa, ba\}$ allora $S \not\subseteq T$ e $T \not\subseteq S$

1.1.5 Insieme Uguali

Due insiemi T e S sono uguali se $S \subseteq T$ e $T \subseteq S$

1.1.5.1 Esempio Insiemi Uguali

- Siano $S = \{ab, ba\}$ e $T = \{ba, ab\}$, allora $S \subseteq T$ e $T \subseteq S$; quindi $S = T$

1.1.6 Unione

Dati due insiemi S e T , la loro unione è l'insieme

- $S \cup T = \{w \mid w \in S \text{ oppure } w \in T\}$

1.1.6.1 Esempio Unione

- $S = \{a, ba\}$ e $T = \emptyset$, allora $S \cup T = S$
- $S = \{a, ba\}$ e $T = \{e\}$ allora $S \cup T = \{e, a, ba\}$

1.1.7 Intersezione

Dati due insiemi S e T , la loro intersezione è l'insieme

- $S \cap T = \{w \mid w \in S \text{ e } w \in T\}$

Due insiemi si dicono **disgiunti** se $S \cap T = \emptyset$

1.1.8 Differenza

Dati due insiemi S e T , la loro differenza è

- $S - T = \{w \mid w \in S \text{ e } w \notin T\}$

1.1.8.1 Esempio Differenza

- Sia $S = \{a, b, bb, bbb\}$ e $T = \{a, bb, bab\}$ allora $S - T = \{b, bbb\}$.
- Sia $S = \{ab, ba\}$ e $T = \{ab, ba\}$ allora $S - T = \emptyset$

1.1.9 Complemento

Dato un insieme universale U , il completo di un insieme $S \subseteq U$ è

- $\bar{S} = \{w \mid w \in U, w \notin S\} = U - S$
- $\bar{S}$ è l'insieme di tutti gli elementi di U che non sono in S

1.1.10 Sequenze

Una sequenza di oggetti è una lista di oggetti in un qualche ordine

- Denotata scrivendo la lista fra $()$
- Ordine e ridondanza sono importanti

Una **k-upla** è una sequenza di k elementi

1.1.11 Prodotto Cartesiano

Dati due insiemi A e B, il prodotto cartesiano $A \times B$ è l'insieme di coppie

- $A \times B = \{(x, y) \mid x \in A, y \in B\}$
- $A \times B \neq B \times A$
- $\emptyset \times B = \emptyset$
- Se A e B sono insiemi finiti, $|A \times B| = |A||B|$

1.1.11.1 Esempio Prodotto Cartesiano

- Siano $A = \{a, ba, bb\}$ e $B = \{ba\}$, allora
- $A \times B = \{(a, ba), (ba, ba), (bb, ba)\}$
- $B \times A = \{(ba, a), (ba, ba), (ba, bb)\}$

1.1.11.2 Prodotto Cartesiano di due o più insiemi

- $A_1 \times \dots \times A_k = \{(x_1, \dots, x_k) \mid x_i \in A_i, 1 \leq i \leq k\}$

1.1.12 Funzioni

Dati due insiemi non vuoti X e Y, una funzione $f: X \rightarrow Y$ è una relazione che associa ad ogni elemento di x in X uno e uno solo elemento $y = f(x)$ in Y

- X Dominio
- Y Codominio
- Occorre definire anche la relazione che associa ogni elemento del dominio a un elemento del codominio
- Un **predicato** o **proprietà** è una funzione il cui range è $\{true, false\}$
- Una **proprietà** il cui dominio è un insieme di k-tuple è detto **relazione**, di due tuple è detta **relazione binaria**
 - Si dice **relazione di equivalenza** se è riflessiva (se per ogni x, xRx), transitiva (se per ogni x e y e z, xRy e yRz implica xRz) e simmetrica (se per ogni x e y, xRy implica yRx)

1.1.12.1 Classificazioni Funzioni

- Una funzione $f : X \rightarrow Y$ è **iniettiva** se

$$\forall x, x' \in X, x \neq x' \Rightarrow f(x) \neq f(x')$$

- Una funzione $f : X \rightarrow Y$ è **suriettiva** se

$$\forall y \in Y, \exists x \in X : y = f(x)$$

- Una funzione $f : X \rightarrow Y$ è **biettiva** di X su Y se f è iniettiva e suriettiva

1.1.13 Insieme Potenza

Per ogni insieme S, l'insieme potenza (o insieme delle parti) $P(S)$ è l'insieme di tutti i possibili sottoinsiemi di S

$$P(S) = \{A \mid A \subseteq S\}$$

1.1.13.1 Lemma

- Se S è un insieme finito allora

$$|P(S)| = 2^{|S|}$$

1.2 Logica Booleana

Un'espressione booleana Φ sulle variabili $x_1, \dots, x_n$ è soddisfatta da un assegnamento che rende vera Φ

- Vero = 1
- Falso = 0

1.2.1 Negazione o Not

Rappresenta il valore opposto e si usa il simbolo $\neg$

1.2.2 Congiunzione o And

Vera se entrambi i valori sono veri e si usa il simbolo $\wedge$

1.2.3 Disgiunzione o Or

Vera se almeno uno dei due valori è vero e si usa il simbolo $\vee$

1.2.4 Or Esclusivo

Vero se esattamente uno dei due operandi è 1 e usa il simbolo $\oplus$

1.2.5 Uguaglianza

Vera se entrambi gli operandi hanno lo stesso valore e si usa il simbolo $\leftrightarrow$

1.2.6 Implicazione

Falso se il suo primo operando ha valore vero e il secondo falso, vero in tutti gli altri casi $\rightarrow$

- $0 \rightarrow 0 = 1$
- $0 \rightarrow 1 = 1$
- $1 \rightarrow 0 = 0$
- $1 \rightarrow 1 = 1$

1.2.7 Relazioni fra Operazioni

- $P \vee Q = \neg(\neg P \wedge \neg Q)$
- $P \rightarrow Q = \neg P \vee Q$
- $P \leftrightarrow Q = (P \rightarrow Q) \wedge (Q \rightarrow P)$
- $P \oplus Q = \neg(P \leftrightarrow Q)$

1.2.8 Legge Distributiva

- $P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$
- $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$

1.3 Dimostrazioni

Ragionamento convincente per dire che un enunciato è vero

- Se un enunciato è falso si usa il controesempio
- Dimostrazione per **assurdo**: Assumiamo che il teorema sia falso per poi dimostrare che questa ipotesi porta ad una contraddizione o assurdo
- Dimostrazione **costruttiva**: Mostrare come costruire l'oggetto
- Dimostrazione per **induzione**: Permette di dimostrare che tutti gli elementi di un insieme infinito godono di una data proprietà
 - Base induttiva: Dimostra che $P(1)$ vero
 - Passo induttivo: Se $P(1)$ vero (**ipotesi induttiva**), allora per ogni $i \geq 1$ anche $P(i + 1)$ vero
- **Lemma**: Enunciato interessante che serve alla dimostrazione di un enunciato più interessante

1.4 Grafi

Un grafo non orientato è un insieme di punti e di linee che connettono alcuni dei punti

- Punti chiamati **nodi**
- Linee chiamate **archi**
- Numero di archi rappresenta il grado del grafo

1.4.1 Sottografo

Diciamo che il grafo G è un sottografo di H se i nodi di G sono un sottoinsieme dei nodi di H e gli archi di G sono anche archi di H sui nodi corrispondenti

1.4.2 Cammino

Un cammino in un grafo è una sequenza di nodi collegati da archi

- **Cammino semplice**: Un cammino che non contiene nodi ripetuti
- **Grafo connesso**: Se esiste un cammino per ogni coppia di nodi
- **Ciclo**: Un cammino che inizia e termina nello stesso nodo
- **Ciclo semplice**: Contiene almeno tre nodi e ripete solo il primo e ultimo nodo

1.4.3 Albero

Un grafo connesso senza cicli semplici

- In un albero con n nodi ci sono sempre esattamente $n - 1$ archi
- Presenta una **radice** e i nodi di grado 1 ad eccezione della radice sono detti **foglie**

1.4.4 Grafo Orientato o Diretto

Presenta delle frecce che indicano la direzione

- Il numero di frecce che escono da un nodo è il **grado uscente**, mentre il numero di frecce che entrano è detto **grado entrante**
- Archi rappresentati da una coppia ordinata, dove l'ordine indica la direzione
- Un grafo orientato G è (V, E) , con V insieme dei nodi ed E è l'insieme degli archi
- Un cammino in cui tutte le frecce puntano nella stessa direzione è detto **cammino orientato** ed è **fortemente connesso**

1.5 Alfabeto

Un insieme finito di elementi.

Sia $\Sigma = \{a_1, \dots, a_k\}$ un alfabeto con k simboli, la cardinalità di $|\Sigma|$ è k

1.5.1 Stringa

Su un alfabeto è una sequenza di simboli dell'alfabeto

- La stringa vuota ε non contiene nessun simbolo
- Lunghezza della stringa s è denotata dal numero di simboli $|s|$

1.5.1.1 Definizione Ricorsiva di Stringa

- Passo base: La stringa vuota ε è una stringa
- Passo ricorsivo: Se w è una stringa e $x \in \Sigma$ è un simbolo dell'alfabeto, allora wx è una stringa

1.5.2 Σ^*

Dato un alfabeto Σ , denotiamo con Σ^* l'insieme di tutte le possibili stringhe su Σ

- Σ e Σ^* sono insiemi diversi

1.5.2.1 Definizione Ricorsiva di Σ^*

- Passo base: $\varepsilon \in \Sigma^*$
- Passo ricorsivo: Se $w \in \Sigma^*$ e $x \in \Sigma$, allora $wx \in \Sigma^*$

1.5.2.2 Definizione Ricorsiva di Lunghezza di una Stringa

- Passo base: $|\varepsilon| = 0$
- Passo ricorsivo: Se w è una stringa e $x \in \Sigma$, allora $|wx| = |w| + 1$

1.5.3 Stringhe Uguali

Due stringhe

$$x = a_1 a_2 \dots a_h$$

e

$$y = b_1 b_2 \dots b_k$$

con a_i e $b_j \in \Sigma$, $1 \leq i \leq h$, $1 \leq j \leq k$

si dicono uguali se $h = k$ e $a_i = b_i$; per $i = 1, \dots, h$

- Caratteri letti da destra e sinistra sono uguali

1.5.4 Concatenazione di Stringhe

Date le stringhe $x = a_1a_2...a_h$, $y = b_1b_2...b_k$, con $a_i, b_j \in \Sigma$, $1 \leq i \leq h$, $1 \leq j \leq k$, la concatenazione è definita come

$$x \cdot y = a_1a_2...a_hb_1b_2...b_k$$

Inoltre

$$x\varepsilon = \varepsilon x = x$$

- Non è commutativa, Siano x e y stringhe $xy \neq yx$
- La concatenazione è associativa, Siano x, y, z stringhe $(xy)z = x(yz)$
- Lunghezza di $|xy| = |x| + |y|$

1.5.4.1 Definizione Ricorsiva di Concatenazione

- Passo base: Se $w \in \Sigma^*$ allora $w\varepsilon = w \in \Sigma^*$
- Passo ricorsivo: Se $w_1 \in \Sigma^*$, $w_2 \in \Sigma^*$, $w_2 \neq \varepsilon$, $\exists w_2 \in \Sigma^*$, $x \in \Sigma$ tali che $w_2 = w'_2x$ allora $w_1 \cdot w_2 = w_1 \cdot (w'_2x) = (w_1 \cdot w'_2)x \in \Sigma^*$

1.5.5 Potenza di una Stringa

Sia $m \geq 0$ un intero non negativo

- Potenza con esponente $m = 0$ di una stringa è la stringa vuota ε
- Per $m > 0$, la potenza di una stringa w è la concatenazione di w con se stessa $m - 1$ volte

1.5.5.1 Definizione Ricorsiva di Potenza

- Passo base: $w^0 = \varepsilon$
- Passo ricorsivo: $w^m = w^{m-1}w$, per $m > 0$

1.5.6 Sottostringa

w è una sottostringa di s se esistono stringhe x e y tali che $s = xwy$

1.5.6.1 Esempio Sottostringa

- 567 è sottostringa di 34567
- 42 non è sottostringa di 472

1.5.7 Inversa di una Stringa

L'inversa w^R di una stringa w è la stringa ottenuta scrivendo i caratteri da destra verso sinistra

$$\varepsilon^R = \varepsilon$$

e se

$$w = a_1 a_2 \dots a_n$$

con a_j simboli

$$w^R = a_n a_{n-1} \dots a_1$$

1.5.7.1 Definizione Ricorsiva dell'Inversa di una Stringa

- Passo base: $\varepsilon^R = \varepsilon$
- Passo ricorsivo: Per ogni $x \in \Sigma^*$ e $\sigma \in \Sigma$, $(x\sigma)^R = \sigma x^R$

1.5.8 Precedenze

- $(ab)^2 = abab \neq abb = ab^2$
- $(ab)^R = ba \neq ab^R = ab$

1.5.9 Linguaggi

Un linguaggio è un insieme di stringhe su un alfabeto $L \subseteq \Sigma^*$

- $\bar{L} = \Sigma^* - L = \{w \in \Sigma^* \mid w \notin L\}$
- Non sono sempre finiti
- Cardinalità è il numero delle sue stringhe $|L| = |\{aba, aab\}| = 2$
- $|\emptyset| = 0$
- $\varepsilon \notin \emptyset$
- $\emptyset \neq \{\varepsilon\}$

1.5.9.1 Esempio Linguaggi 1

- Sia $\Sigma = \{a, b\}$
- $L_1 = \{aa, aaa\}$

1.5.9.2 Esempio Linguaggi 2

- Alfabeto $\{a, b\}$
- Linguaggio $L = \{w \in \{a, b\}^* \mid \text{la prima lettera di } w \text{ è } b\}$
- L : insieme delle stringhe su $\{a, b\}$ che non iniziano con b .
- NON insieme stringhe che iniziano con a (es. stringa vuota $\epsilon \in L$)

1.5.10 Prodotto di Linguaggi

Dati due linguaggi S e T sull'alfabeto Σ , il prodotto è l'insieme di tutte le stringhe che sono concatenazione di una stringa S e una stringa T

$$ST = S \cdot T = \{uv \in \Sigma^* \mid u \in S, v \in T\}$$

- Diversa dal prodotto cartesiano in cui consideriamo le coppie
 - $S \times T = \{(a, \epsilon), (a, ba), (ba, \epsilon), (ba, ba), (bb, \epsilon)\}$, $ST \neq S \times T$
- Nel prodotto di linguaggi il risultato è un insieme di stringhe (concatenate), mentre nel prodotto cartesiano è un insieme di tuple

1.5.10.1 Esempio Prodotto di Linguaggi

- Se $S = \{a, aa\}$ e $T = \{\epsilon, a, ba\}$, allora
- $ST = \{a, aa, aba, aaa, aaba\}$, $TS = \{a, aa, aaa, baa, baaa\}$

1.5.11 Potenza di un Linguaggio

Sia L un linguaggio sull'alfabeto Σ

$$L^0 = \{\epsilon\}$$

$$L^k = L^{k-1}L, k \geq 1$$

1.5.12 Casi Particolari Operazioni sui Linguaggi

- $\emptyset^0 = \{\epsilon\}$
- $L \cdot \emptyset = \emptyset \cdot L = \emptyset$
- $L \cdot \{\epsilon\} = \{\epsilon\} \cdot L = L$

1.5.13 Chiusura di Kleene

La chiusura di Kleene di un linguaggio L è

$$L^* = \bigcup_{n \geq 0} (L^n)$$

- L^* è il linguaggio ottenuto concatenando un numero qualsiasi di stringhe L
- $L^* = \{w_1 w_2 \dots w_k \mid k \geq 0, w_i \in L, 1 \leq i \leq k\}$
- Con $k = 0$, $w_1 w_2 \dots w_k = \varepsilon$

1.5.13.1 Esempio Chiusura di Kleene

- Se $L = \{a, b\}$, allora $L^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, \dots\}$
- Se $L = \emptyset$, allora $L^* = \{\epsilon\}$
- Se $L = \{\epsilon\}$, allora $L^* = \{\epsilon\}$
- $(L^*)^* = L^*$

1.5.14 Chiusura Positiva

La chiusura positiva si distingue dalla chiusura di Kleene perchè nell'unione non compare la potenza L^0

$$L^+ = \bigcup_{n > 0} L^n = \{w_1 w_2 \dots w_k \mid k > 0, w_i \in L, 1 \leq i \leq k\}$$

1.5.14.1 Esempio Chiusura Positiva

- Se $L = \{a\}$, allora
- $L^+ = \{a, aa, aaa, \dots\} = \{a^n \mid n > 0, n \in \mathbb{N}\}$
- $L^+ = \bigcup_{n > 0} (L^n) = \emptyset = L$
- $L^+ = \bigcup_{n > 0} (L^n) = \{\epsilon\} = L$

2 Automi Finiti Deterministici

Meccanismo di computazione con poca memoria

- Descrivono comportamenti
- Ricevono in input delle stringhe e risponderanno accettando o rifiutando l'input
- Numero di stati finito
- Unico Stato iniziale
- **Deterministico**: Se e solo se ha esattamente 1 transizione da ogni stato per ogni simbolo

2.1 Usi DFA

- Prima fase del processo di compilazione
 - Legge il codice sorgente e controlla la “correttezza” di quanto scritto: raggruppa i caratteri, interagendo con una tavola di simboli e automi
 - Espande le macro, prepara messaggi di errore, elimina spazi, linee, commenti...
 - Produce in output un flusso di token (ad esempio: numero, operatore) associato ai lessemi (ad esempio: 17, +, :)
- Automi per il pattern matching sia per la bioinformatica che per la sicurezza

2.2 Parti DFA

- Presenta degli stati
 - Es. $\{q_0, q_1, q_2, q_3, q_4\}$
- Stato iniziale è lo stato da cui inizia l'input, è unico
- Stati accettati che portano ad accettare l'input
- Stati non accettati che portano a rifiutare l'input
- Presenta un alfabeto per l'input
- Stato pozzo o Trappola, non è mai accettante, definisce uno stato non accettante per tutti i simboli dell'alfabeto e dal quale non si può uscire non avendo archi uscenti

2.3 Definizione Formale DFA

Un DFA M è una 5-tupla $(Q, \Sigma, \delta, q_0, F)$

- Q : Insieme degli stati
- Σ : Alfabeto delle stringhe in input
- δ : Funzione di transizione tra gli stati $\delta : Q \times \Sigma \rightarrow Q$
 - Associa ad una coppia (stato, carattere) un solo stato
 - Può essere definito come: diagramma di stato, tabella di transizione o scrittura esplicita della funzione
 - Esattamente una transizione da ogni stato per ogni simbolo dell'alfabeto (**deterministico**)
- q_0 : Stato iniziale
- F : Insieme degli stati accettati, $F \subseteq Q$
 - Può essere vuoto (accetta solo $\emptyset$), coincidere con l'insieme degli stati Q (accetta tutto Σ^*), o essere un suo sottoinsieme

2.4 Relazione fra DFA e Linguaggi

Dato un automa M , con $L(M)$ indichiamo il linguaggio che l'automata riconosce, quindi il linguaggio L è riconosciuto da un automa M se $L(M) = L$

Ogni DFA accetta un insieme di stringhe, quindi un linguaggio L , rifiutando quelle che non sono in L

- Per ogni stringa in Σ^* l'automata M accetta o non accetta
- Il linguaggio che un DFA riconosce è l'insieme delle stringhe di input per le quali il DFA accetta, ed è **unico**

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito deterministico, il linguaggio accettato o riconosciuto da A è

$$L(A) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$$

- **Cammino Accettante**: Cammino che ha come ultimo nodo un nodo accettante
 - Se esiste un cammino accettante, tale cammino è unico (come tutti i cammini)
- Non è possibile costruire gli automi per ogni elemento di Σ^*

- Dato un linguaggio L possiamo avere più automi
- Dato un automa M riconosce un solo linguaggio unico
- Dato un linguaggio L , costruire se possibile un automa M che lo riconosce equivale a dire $L = L(M)$

2.4.1 Descrizione Formale delle Stringhe che l'Automa Accetta

Sia w una stringa $w = a_1 a_2 \dots a_n, a_i \in \Sigma$

L'automa M accetta w se e solo se

$$\exists (r_0, r_1 \dots r_n), r_i \in Q, r_0 = q_0, r_n \in F$$

e

$$\forall i = 0, 1, \dots, n-1, \delta(r_i, a_{i+1}) = r_{i+1}$$

- Quindi la macchina inizia da una condizione iniziale che appartiene alla sequenza di stati
- La macchina passa da uno stato ad un altro in base ad una funzione di transizione
- La macchina accetta il suo input se termina la lettura in uno stato accettante

$$L(M) = \{w \mid M \text{ accetta } w\}$$

2.4.2 Descrizione Ricorsiva Formale delle Stringhe che l'Automa Accetta

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito deterministico

La funzione di transizione estesa $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ è definita ricorsivamente come segue:

- Passo base:

$$\forall q \in Q, \hat{\delta}(q, \varepsilon) = q$$

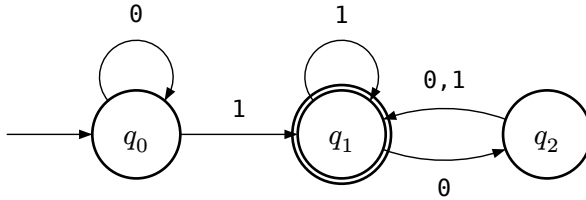
Non è la transizione, ma $\hat{\delta}$ è uno stato

- Passo Ricorsivo:

$$\forall q \in Q, w \in \Sigma^*, a \in \Sigma,$$

$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

2.4.2.1 Esempio Descrizione Ricorsiva Formale delle Stringhe che l'Automa Accetta



Sia $w = 0100$

$$\hat{\delta}(q_0, 0100) = \delta(\hat{\delta}(q_0, 010), 0) =$$

$$\delta(\delta(\hat{\delta}(q_0, 01), 0), 0) =$$

$$\delta(\delta(\delta(\hat{\delta}(q_0, 0), 1), 0), 0) =$$

$$\delta(\delta(\delta(\delta(\hat{\delta}(q_0, \varepsilon), 0), 1), 0), 0) =$$

Ma $\hat{\delta}(q_0, \varepsilon)$ è il passo base quindi:

$$\delta(\delta(\delta(\delta(q_0, 0), 1), 0), 0), 0)$$

Prova della correttezza:

$$\hat{\delta}(q_0, 0100) = \delta(\delta(\delta(\delta(q_0, 0), 1), 0), 0), 0) =$$

$$\delta(\delta(\delta(q_0, 1), 0), 0) =$$

$$\delta(\delta(q_1, 0), 0) =$$

$$\delta(q_2, 0) = q_1$$

2.4.3 DFA e Linguaggi con Cicli e senza Cicli

- L'assenza di cicli in un automa finito M lungo cammini di accettazione implica che $L(M)$ è finito
- La presenza di un ciclo non implica che il linguaggio sia infinito, ma occorre che sia su un cammino accettante

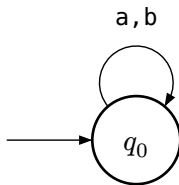
2.4.4 Automa Insieme Vuoto

- $L(M) = \{\}$

$$(Q, \Sigma, \delta, q_0, F) \text{ dove } Q = \{q_0\}, \Sigma = \{a, b\}, F = \emptyset$$

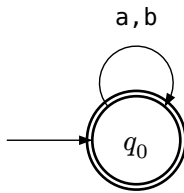
$$\delta(q_0, a) = \delta(q_0, b) = q_0$$

- Non è unico

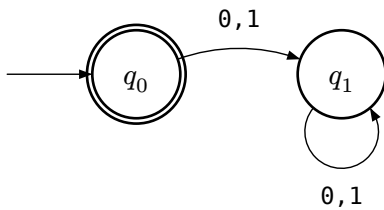


2.4.5 Automa Σ^*

- $L(M) = \Sigma^*$



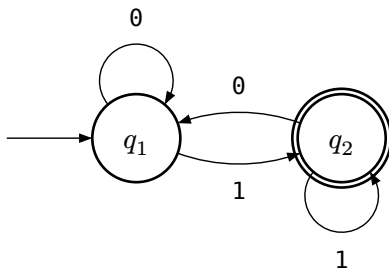
2.4.6 Automa Solo Parola Vuota



2.5 Dal DFA al Linguaggio Riconosciuto

Complicato ma sempre possibile grazie ad algoritmi appositi

2.5.1 Esempio 1.7



Forniamo la descrizione formale:

$$M_2 = (\{q_1, q_2\}, \{0, 1\}, \delta, q_1, \{q_2\})$$

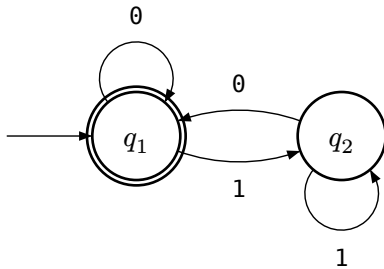
$$\begin{aligned}\delta(q_1, 0) &= q_1 \\ \delta(q_2, 0) &= q_1 \\ \delta(q_1, 1) &= q_2 \\ \delta(q_2, 1) &= q_2\end{aligned}$$

Non è presente la parola vuota

$$L(M_2) = \{w \text{ su } \{0, 1\} \mid w \text{ termina con } 1\}$$

$$L(M_2) = \{w \in \{0, 1\}^* \mid w = w'1, w' \in \{0, 1\}^*\} = \{0, 1\}^*\{1\}$$

2.5.2 Esempio 1.9



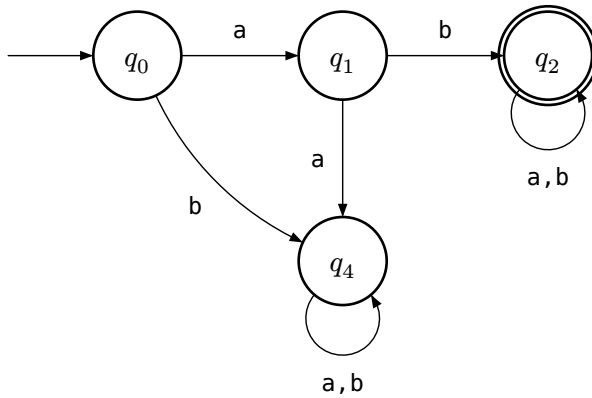
$$M_3 = (\{q_1, q_2\}, \{0, 1\}, \delta, q_1, \{q_1\})$$

Parola vuota accettata

$$L(M_3) = \{w \text{ su } \{0, 1\} \mid w = \varepsilon \text{ oppure termina con } 0\}$$

$$L(M_3) = \{w \in \{0, 1\}^* \mid w = \varepsilon \text{ oppure } w = w'0, w' \in \{0, 1\}^*\} = \{0, 1\}^*\{0\} \cup \{\varepsilon\}$$

2.5.3 Esempio Slide 1

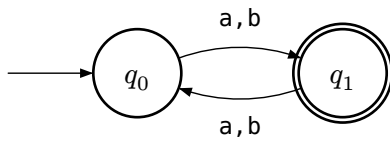


$$L(M) = \{s \mid s \text{ inizia con } ab\} =$$

$$L(M) = \{abw \mid w \in \{a, b\}^*\} =$$

$$L(M) = \{ab\}\{a, b\}^*$$

2.5.4 Esempio Slide 2

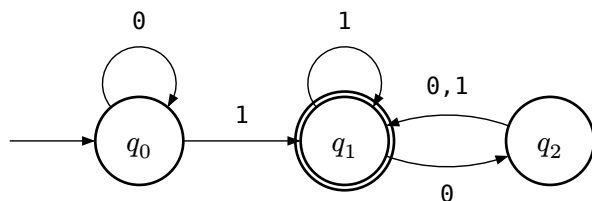


$$L(M) = \{s \mid \text{lunghezza di } s \text{ è dispari}\} =$$

$$L(M) = \{aa, ab, ba, bb\}^*\{a, b\} =$$

$$L(M) = (\{a, b\}\{a, b\})^*\{a, b\}$$

2.5.5 Esempio 1.6



$L(M) = \{w \text{ in } \{0,1\}^* \mid w \text{ termina con } 1 \text{ oppure}$
 tale che dopo l'ultimo 1 ha un numero pari di 0} =

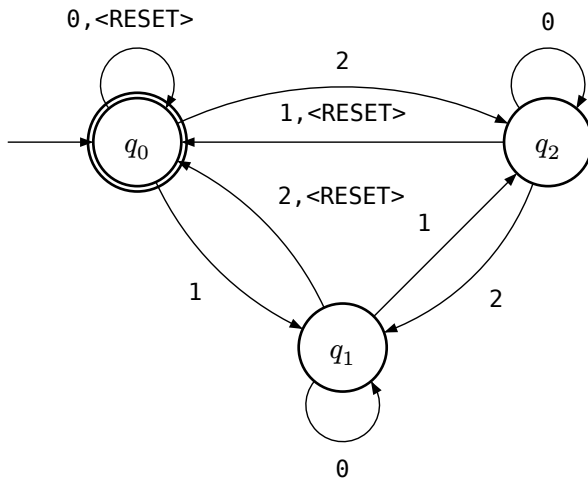
Consideriamo che $w = w'1w''$ dove $w' \in \{0,1\}^*, w'' \in \{00\}^*$

2.5.6 Esempio 1.13

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{\text{RESET}, 0, 1, 2\}$$

- Con RESET torno nello stato iniziale q_0
- Leggere 0 non fa cambiare stato
- Leggere 1 mi fa spostare nello stato successivo
- Riconosce sicuramente solo RESET e solo 0
- Se non consideriamo RESET allora accetta le parole in cui la somma è $0 \bmod 3$
- Quindi il linguaggio accettato dall'automa riguarda le parole in cui la somma è $0 \bmod 3$ e con RESET la somma viene azzerata, la macchina conserva un conto parziale e RESET lo riporta a 0 ed accetta i multipli di 3



2.6 Liguaggi Regolari

Un linguaggio L è definito regolare se e solo se esiste un DFA M che lo riconosce

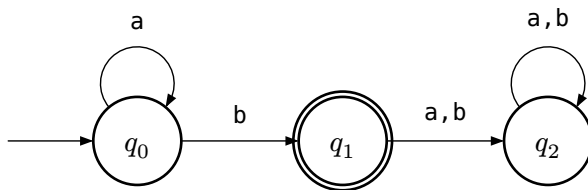
$$L = L(M)$$

- Non tutti i linguaggi sono regolari, ad es. $\{a^n b^n \mid n \geq 0\}$, dato che ogni potenza aggiunge un nuovo stato e l'automa si allunga all'infinito

2.6.1 Esempio Linguaggi Regolari

- Tutti i linguaggi finiti
- $\{a^n b \mid n \geq 0\}$

$$L = \{a^n b \mid n \geq 0\} = \{b, ab, aab, aaab, \dots\}$$

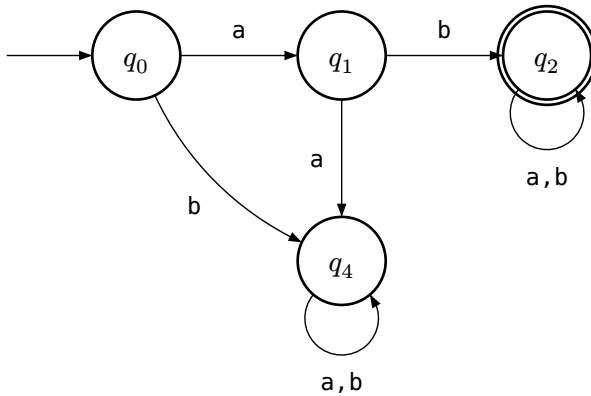


- Tutte le stringhe in $\{a, b\}^*$ con prefisso ab

$$L(M) = \{s \mid s \text{ inizia con } ab\} =$$

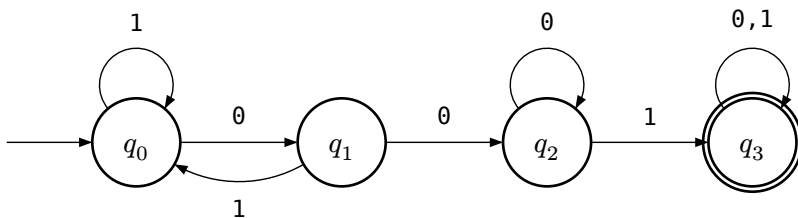
$$L(M) = \{abw \mid w \in \{a, b\}^*\} =$$

$$L(M) = \{ab\}\{a, b\}^*$$



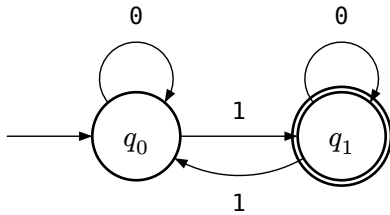
- Tutte le stringhe in $\{0, 1\}^*$ che contengono 001 come fattore

$$L = \{x001y \mid x, y \in \{0, 1\}^*\}$$



Presenza del coppia relativo al numero 1 spezzerebbe il fattore 001

- Tutte le stringhe su $\{0, 1\}$ che hanno un numero dispari di 1



2.7 Calcolo Correttezza Automa

Dovrei dimostrare che se per L ho disegnato un DFA $M = (Q, \Sigma, \delta, q_0, F)$ si ha

$$w \in L \Leftrightarrow \hat{\delta}(q_0, w) \in F$$

Tuttavia la prova per induzione potrebbe non essere facile

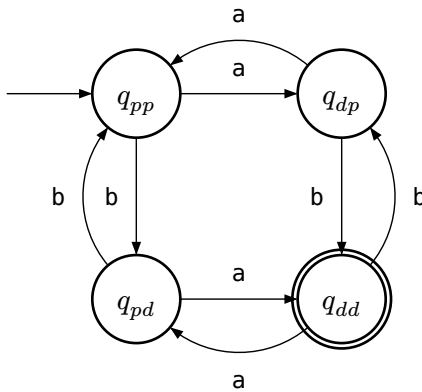
2.8 Esempio Tecnica Progettazione Diretta DFA senza Calcolo Correttezza

- Progettare un DFA che accetta

$$D = \{w \in \{a, b\}^* \mid |w| = 2h, h \geq 0 \text{ e } |w|_a = 2k + 1, k \geq 0\}$$

Quindi devono avere anche un numero dispari di b essendo $\Sigma = \{a, b\}$

$$\forall w \in D \text{ si ha che } |w|_b = 2k + 1, k \geq 0$$



Abbiamo osservato che $\forall w \in D$ si ha che $|w|_b = 2k + 1, k \geq 0$

Quindi sto contando sia a che b dispari, **entrambi** devono essere dispari

$$D_a = \{w \in \{a, b\}^* \mid |w|_a = 2k + 1, k \geq 0\}$$

$$D_b = \{w \in \{a, b\}^* \mid |w|_b = 2n + 1, n \geq 0\}$$

Quindi

$$D = D_a \cap D_b$$

E quindi sappiamo anche

$$D_{\text{even}} = \{w \in \{a, b\}^* \mid |w| = 2h, h \geq 0\}$$

$$D_a = \{w \in \{a, b\}^* \mid |w|_a = 2k + 1, k \geq 0\}$$

$$D = D_{\text{even}} \cap D_a$$

2.9 Chiusura Linguaggi Regolari e Dimostrazione Correttezza DFA

Dati due linguaggi L_1 e L_2 regolari

- $L_1 \cup L_2$
- $L_1 \cdot L_2$
- L_1^*
- L_1^R (Reversal)
- $\overline{L_1}$ (Complemento)
- $L_1 \cap L_2$

Sono tutti regolari

2.9.1 Operazioni Regolari

Siano A e B linguaggi, definiamo le operazioni regolari unione, concatenazione e star

- **Unione:** $A \cup B = \{x \mid x \in A \text{ oppure } x \in B\}$
- **Concatenazione:** $A \cdot B = \{xy \mid x \in A, y \in B\}$
- **Star:** $A^* = \{w_1 w_2 \dots w_k \mid k \geq 0, w_i \in A, 1 \leq i \leq k\} = \bigcup_{n \geq 0} (A^n)$

2.9.2 Chiusura

Una classe di oggetti è chiusa rispetto ad un'operazione se l'applicazione di questa operazione a elementi della classe restituisce un oggetto ancora della classe

- **Chiusura Classe REG:** La classe REG è chiusa rispetto alle operazioni regolari, quindi comunque prendo 2 linguaggi regolari e faccio $\cup \cdot *$ ottengo comunque un linguaggio regolare
- **Esempio:** $\mathbb{N}$ è chiuso rispetto a $+$ e $\times$, ma non è chiuso rispetto a :

2.10 Teorema 1.25 Chiusura Classe REG Unione

La classe REG è chiuso rispetto alle operazioni regolari

2.10.1 Idea della Dimostrazione

$$L_1 \text{ e } L_2 \Rightarrow L_1 \cup L_2 \text{ regolare}$$

$$\exists M_1 \text{ t.c. } L_1 = L(M_1) \text{ e } \exists M_2 \text{ t.c. } L_2 = L(M_2)$$

$$M \text{ t.c. } L(M) = L_1 \cup L_2$$

Va dimostrato per ogni linguaggio $\forall L_1, L_2$ regolari, $L_1 \cup L_2$ è regolare

Automa non copia, una volta processata una stringa e non la più usare, per farlo bisogna fare in parallelo, utilizzando una coppia di stati
Dobbiamo costruire un automa M che accetta se e solo se una delle due accetta (Unione di Linguaggi)

2.10.2 Dimostrazione

Sia $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ un DFA t.c. $L(M_1) = L_1$

Sia $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ un DFA t.c. $L(M_2) = L_2$

Definiamo $M = (Q, \Sigma, \delta, q_0, F)$ un DFA t.c. $L(M) = L_1 \cup L_2$

Dobbiamo dimostrare il tale che della definizione

$$Q = \{(r_1, r_2) \mid r_1 \in Q_1, r_2 \in Q_2\} = Q_1 \times Q_2$$

Con $q_0 = (q_1, q_2)$

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ oppure } r_2 \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

Di conseguenza

$$\forall a \in \Sigma \text{ e } \forall (r_1, r_2) \in Q, \delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

Quindi

$$w \in L(M) \Leftrightarrow w \in L_1 \cup L_2$$

$$\hat{\delta}(q_0, w) \in F \Leftrightarrow w \in L_1 \cup L_2$$

$$\Leftrightarrow \hat{\delta}_1(q_1, w) \in F_1 \text{ oppure } \hat{\delta}_2(q_2, w) \in F_2$$

Per una prova formale bisognerebbe

$\Rightarrow$ Vera per costruzione

$\Leftarrow$ Si dimostra per induzione

Ne consegue che $|Q| = |Q_1|Q_2|$, rispetto ad un automa creato col processo creativo, con questo metodo potrebbe avere un numero di stati maggiore ma funziona sempre

- Se i due linguaggi non sono sullo stesso alfabeto, prima di fare la costruzione dovrei estendere i due linguaggi e i due automi sull'alfabeto unione, in modo da poter garantire che l'automato prodotto sia ben definito per ogni stato (coppia di stati) su ogni simbolo

2.11 Teorema 1.25 Bis Chiusura Classe REG

Intersezione

Sebbene l'intersezione non sia una delle tre operazioni regolari base (unione, concatenazione, stella), la classe REG è chiusa anche rispetto all'intersezione

La costruzione è identica a quella dell'unione, ma cambiano gli stati finali: l'automa deve accettare se e solo se **entrambi** gli automi originali accettano

Sia $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ un DFA t.c. $L(M_1) = L_1$

Sia $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ un DFA t.c. $L(M_2) = L_2$

Definiamo $M = (Q, \Sigma, \delta, q_0, F)$ un DFA t.c. $L(M) = L_1 \cap L_2$

Dobbiamo dimostrare il tale che della definizione

$$Q = \{(r_1, r_2) \mid r_1 \in Q_1, r_2 \in Q_2\} = Q_1 \times Q_2$$

Con $q_0 = (q_1, q_2)$

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ e } r_2 \in F_2\} = F_1 \times F_2$$

Di conseguenza

$$\forall a \in \Sigma \text{ e } \forall (r_1, r_2) \in Q, \delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

$$\hat{\delta}(q_0, w) \in F \Leftrightarrow w \in L_1 \cap L_2$$

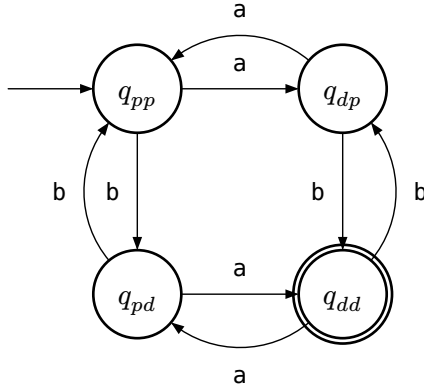
$$\Leftrightarrow \hat{\delta}_1(q_1, w) \in F_1 \text{ e } \hat{\delta}_2(q_2, w) \in F_2$$

2.11.1 Esempio 1

$$D = \{w \in \{a, b\}^* \mid |w| = 2h, h \geq 0 \text{ e } |w|_a = 2k + 1, k \geq 0\}$$

Quindi devono avere anche un numero dispari di b essendo $\Sigma = \{a, b\}$

$$D = \{w \in \{a, b\}^* \mid |w|_a = 2n + 1, n \geq 0 \text{ e } |w|_b = 2k + 1, k \geq 0\}$$



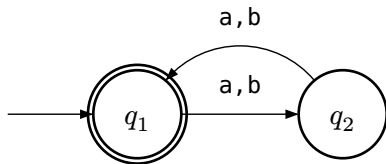
Ma sappiamo che

$$D_{\text{even}} = \{w \in \{a, b\}^* \mid |w| = 2h, h \geq 0\}$$

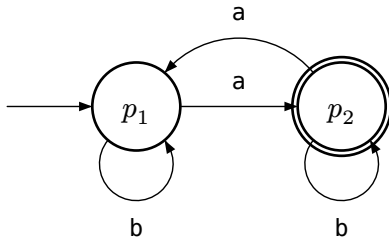
$$D_a = \{w \in \{a, b\}^* \mid |w|_a = 2k + 1, k \geq 0\}$$

$$D = D_{\text{even}} \cap D_a$$

M_{even}

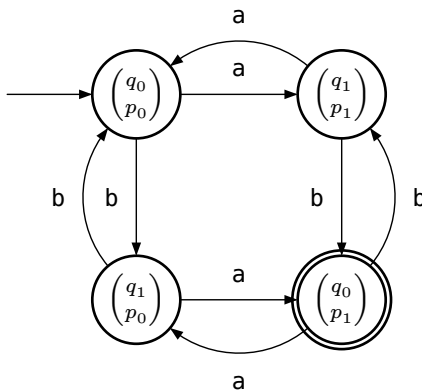


M_a



Usiamo **Lazy Construction**: Partiamo dallo stato iniziale dell'automa prodotto e via via lo completiamo considerando la transizione partendo da quello stato su tutti i simboli dell'alfabeto.

- Quando non ho più transizione uscenti finisco la costruzione.
- L'automa ha $|Q_1| \parallel |Q_2|$ stati, ma può capitare che non ci siano transazioni che li collegano a partire dallo stato iniziale e quindi non li disegno, ma ci sono



Se avessimo considerato

$$D = \{w \in \{a, b\}^* \mid |w|_a = 2n + 1, n \geq 0 \text{ oppure } |w|_b = 2k + 1, k \geq 0\}$$

Sarebbero cambiati solo gli stati finali, e l'unico non finale sarebbe stato $\begin{pmatrix} q_1 \\ p_0 \end{pmatrix}$

2.11.2 Esempio 2

- Costruire un DFA che riconosce le parole che hanno un numero dispari di b oppure un numero pari di a

$$L = \{w \in \{a, b\}^* \mid |w|_b = 2k + 1, k \geq 0 \text{ oppure } |w|_a = 2h, h \geq 0\}$$

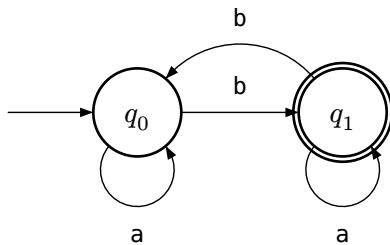
Lo posso riscrivere come

$$L_1 = \{w \in \{a, b\}^* \mid |w|_b = 2k + 1, k \geq 0\}$$

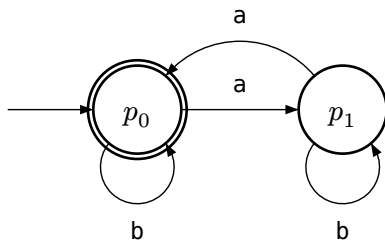
$$L_2 = \{w \in \{a, b\}^* \mid |w|_a = 2h, h \geq 0\}$$

$$L = L_1 \cup L_2$$

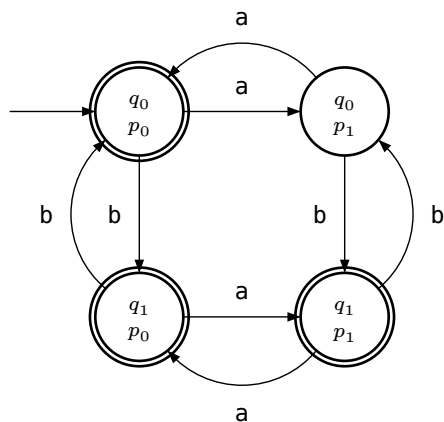
M_1



M_2



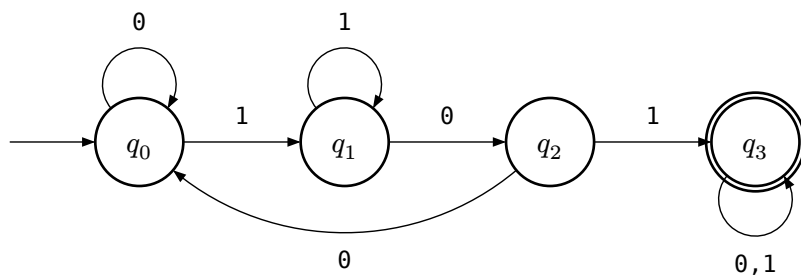
Uso Lazy Construction M



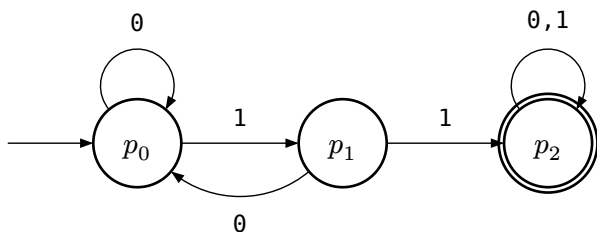
2.11.3 Esempio 3

- Progettare un DFA che riconosce tutte le stringhe su $\{0,1\}$ che contengono 101 oppure 11 come sottostringa

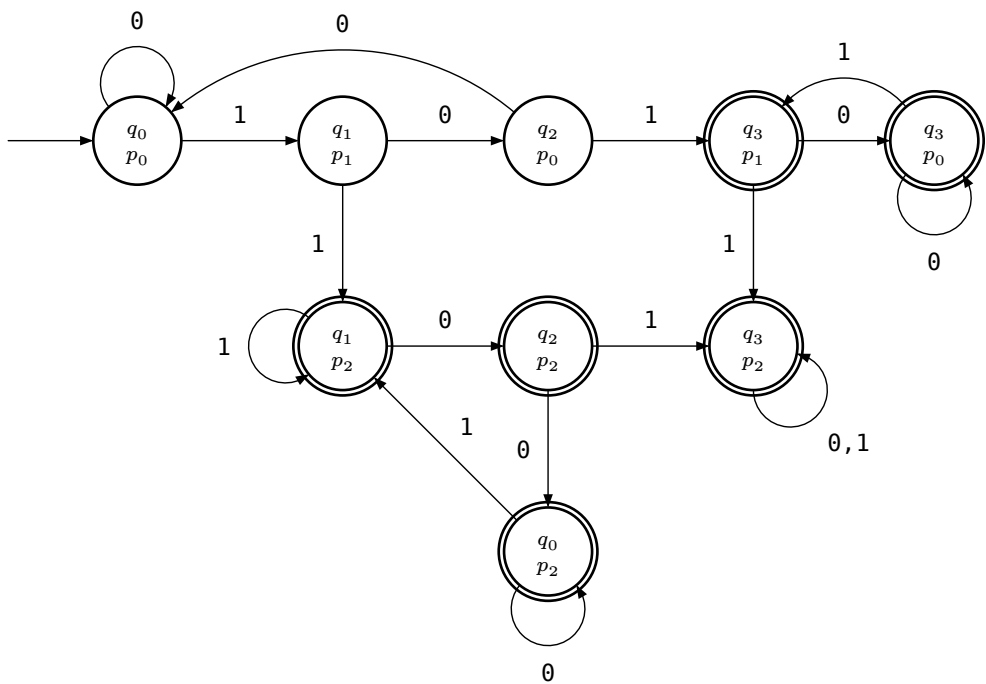
M_1 : 101 come fattore



M_2 : 11 come fattore



Applichiamo Lazy Construction

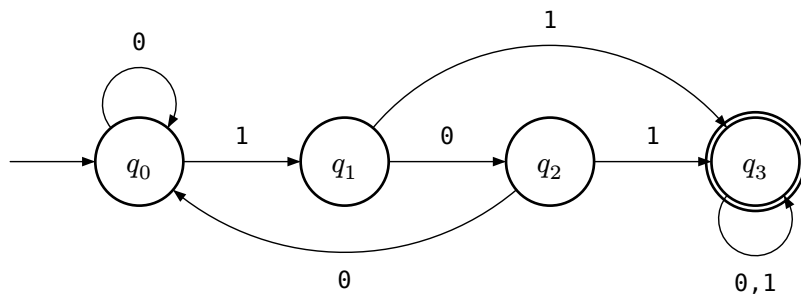


$$|Q| = |Q_1| \cdot |Q_2|$$

Sono disegnati solo gli stati raggiungibili dallo stato iniziale, ci sono stati non raggiungibili come $\begin{pmatrix} q_0 \\ p_1 \end{pmatrix}$, ma sono comunque presenti e definiti nella funzione di transizione

$$L(M) = L_1 \cup L_2$$

Uso metodo creativo



Abbiamo più stati rispetto al metodo creativo, ma l'automa è corretto grazie al Teorema 1.25

2.12 Teorema (4.5 HUM) Chiusura Classe REG

Complemento

La classe REG è chiusa rispetto all'operazione di complemento

$$\bar{L}_1 = \Sigma^* - L_1$$

Creo una copia di M_1 che risolve una condizione e poi complemento gli stati per ottenere quindi il complemento di L_1 .

Si prova attraverso la dimostrazione costruttiva sull'unione.

Un'altra prova è rispetto la chiusura all'intersezione, non costruttiva.

Si possono applicare le leggi di DeMorgan

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

- L_1, L_2 regolari
- $\bar{L}_1, \bar{L}_2$ regolari
- $\overline{\bar{L}_1 \cup \bar{L}_2}$ regolari
- $\overline{\bar{L}_1 \cup \bar{L}_2}$ regolari
- $L_1 \cap L_2$ regolari

2.13 Teorema 1.26 Chiusura Classe REG

Concatenazione

- **Idea:** L'automa accetta solo se l'input può essere diviso in due parti, la prima accettata dal primo automa e la seconda parte dal secondo automa, quindi una computazione in parallelo

2.14 Organizzazione Riposta DFA con Teorema Chiusura

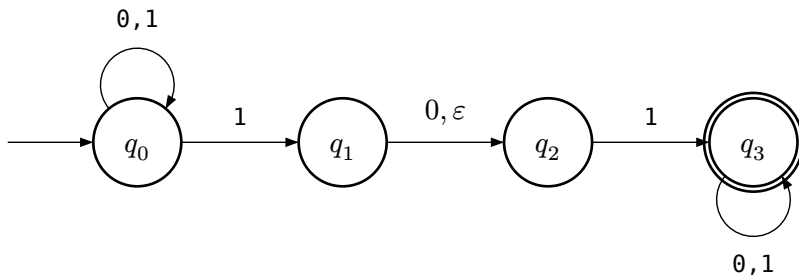
1) Scrivete esplicitamente che fornite un DFA che riconosce il linguaggio sfruttando le proprietà di chiusura (e dite quale). Disegnate i 2 automi da cui partite e riportate le quintuple, esplicitando anche le tabelle di transizione

2) Riportate la definizione dell'automa prodotto, cioè la quintupla così come data nella prova del Teorema 1.25, specificando chi è lo stato iniziale e quali sono gli stati finali di $Q_1 \times Q_2$

3) Per la funzione di transizione, dopo aver riportato la definizione nel 2) dovete esplicitarla: o scrivete la tabella di transizione o il diagramma di stato. Questo servirà quando cercheremo di fornire (nelle prossime lezioni) una forma al linguaggio accettato, che diventa complicato capire leggendo la tabella di transizione. Nel fare il diagramma di stato, dichiarate che usate la lazy construction, quindi che rappresentate solo gli stati raggiungibili dallo stato iniziale. Eventuali stati di $Q_1 \times Q_2$ non disegnati, sono stati su cui la funzione di transizione è definita ma non viene esplicitata perchè non contribuiscono alla determinazione del linguaggio accettato dall'automa.

3 Automi Finiti Non Deterministici

- Consentiamo ε -transizioni, aggiungendo la stringa vuota ε
 - La computazione si divide in più copie parallele per ogni scelta
 - Con ε , quindi, una computazione resta nello stato ed un'altra segue la transizione, ma la stringa rimane sempre unica
- Eliminiamo l'obbligo che per ogni stato esista esattamente una transizione per ogni simbolo dell'alfabeto



Quindi una computazione resta in q_1 dove non leggo 1, mentre in parallelo un'altra va in q_3 dove posso leggere 1, quindi esplorazione di tutti i possibili cammini.

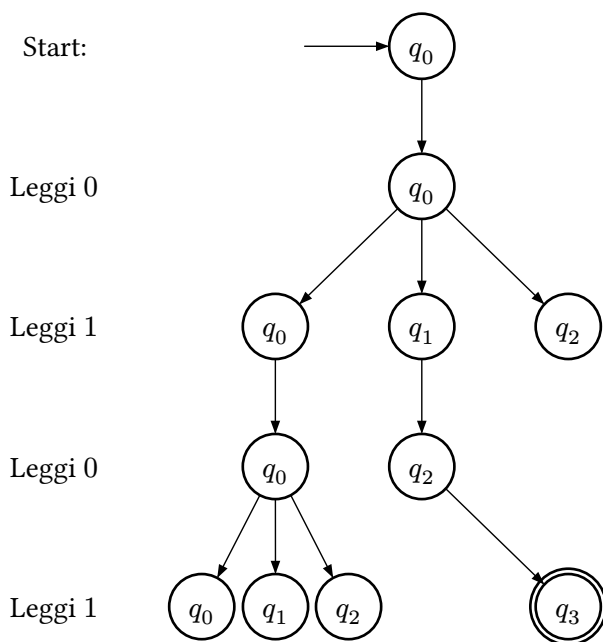
Con ε non consumo l'input.

Dove non ci sono transizione per alcuni simboli dell'alfabeto abbiamo $\emptyset$.

Una computazione è accettata se almeno un cammino per quella stringa è accettato.

Si può sempre esprimere un NFA come un DFA0

- Albero della computazione rispetto l'automa precedente



3.1 Linguaggio Accettato o Riconosciuto NFA

Un NFA accetta tutte e sole le stringhe di Σ^* per le quali esiste un cammino di accettazione, ovviamente non è detto che esista sempre un cammino e che sia unico

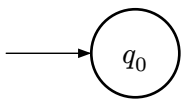
- Accetto la stringa vuota se lo stato iniziale è finale, oppure se esiste un path dallo stato iniziale ad uno stato finale ottenuto seguendo esclusivamente transizioni etichettate con la stringa vuota

Sia $A = \{Q, \Sigma, \delta, q_0, F\}$ un automa non deterministico, il linguaggio accettato o riconosciuto da A è

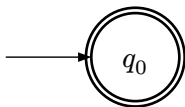
$$L(A) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

3.2 Semplici NFA

- $L(M) = \emptyset$

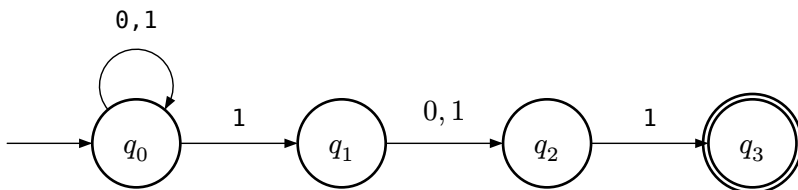


- $L(M) = \{\varepsilon\}$



3.3 Esempio NFA

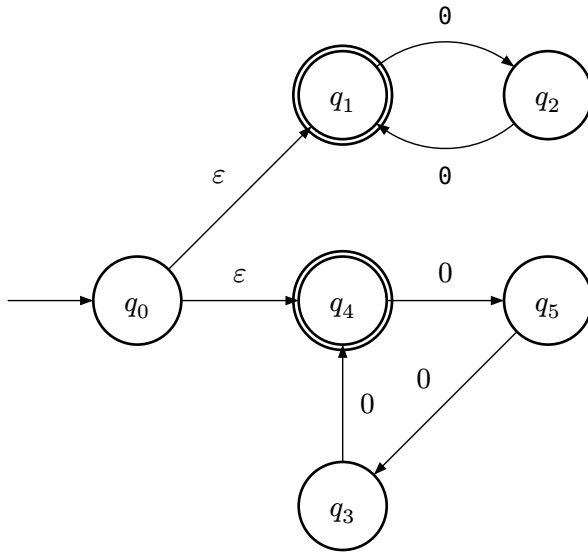
3.3.1 Esempio 1



$$L(M) = \{w \in \{0, 1\}^+ \mid w \text{ ha } 1 \text{ nella terza posizione dalla fine}\}$$

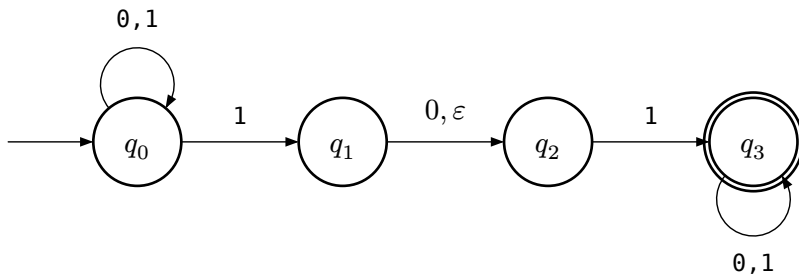
$$L(M) = \{w \in \{0, 1\}^+ \mid w = xly, x \in \{0, 1\}^*, y \in \{0, 1\}^2\}$$

3.3.2 Esempio 2



$L(M)$ = accetta tutte le stringhe della forma 0^k con k multiplo di 2 o 3

3.3.3 Esempio 3



$$N = (Q, \Sigma, \delta, q_1, F)$$

$$Q = \{q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$\left(\begin{array}{c|ccc} & 0 & 1 & \varepsilon \\ \hline q_1 & \{q_1\} & \{q_1, q_2\} & \emptyset \\ q_2 & \{q_3\} & \emptyset & \{q_3\} \\ q_3 & \emptyset & \{q_4\} & \emptyset \\ q_4 & \{q_4\} & \{q_4\} & \emptyset \end{array} \right)$$

q_1 è lo stato iniziale

$$F = \{q_4\}$$

3.4 Definizione Formale NFA

Un automa a stati finiti non deterministico è una quintupla

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q : Insieme finito degli stati
- Σ : Alfabeto finito
- $q_0 \in Q$: Stato iniziale
- $\delta : (Q \times \Sigma_\varepsilon) \rightarrow P(Q)$: Funzione di transizione con $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ che traduce il comportamento non deterministico dell'automa, $P(Q)$ è l'insieme potenza, ovvero la collezione di tutti i possibili insiemi
- $F \subseteq Q$: Insieme degli stati finali

3.5 Epsilon-Chiusura (HUM 2.5.3-2.5.4)

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito non deterministico, sia $q \in Q$.

La ε -chiusura $E(q)$ di q è un sottoinsieme di Q definito ricorsivamente come segue:

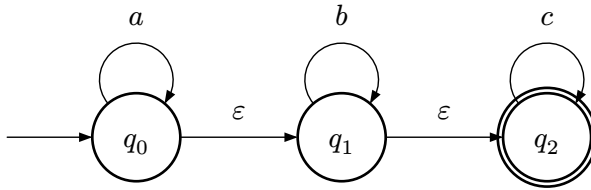
- Passo base: $q \in E(q)$
- Passo ricorsivo: $\forall p \in E(q), \delta(p, \varepsilon) \subseteq E(q)$

Quindi sia $R \subseteq Q$, la ε -chiusura $E(R)$ di R è

$$E(R) = \cup_{q \in R} E(q)$$

- ε -chiusura non è mai $\emptyset$, essendo l'insieme con almeno il primo stato.
- Rappresenta tutti gli stati raggiungibili con le ε -transizioni
- Per ogni stato in R , considero le ε -chiusure di ogni stato

3.5.1 Esempio ε -chiusura



Calcolo $E(q) = ?$

- $q_0 \in E(q_0)$, quindi $E(q_0) = \{q_0\}$,
- $\delta(q_0, \varepsilon) = \{q_1\} \rightarrow q_1 \in E(q_0)$, quindi $E(q_0) = \{q_0, q_1\}$
- $\delta(q_1, \varepsilon) = \{q_2\} \rightarrow q_2 \in E(q_0)$, quindi $E(q_0) = \{q_0, q_1, q_2\}$
- $\delta(q_2, \varepsilon) = \emptyset$, quindi $E(q_0) = \{q_0, q_1, q_2\}$

Quindi considero la δ e ε -chiusura

- Altro metodo
- $E(q_0) = \{q_0\} \cup E(q_1)$
- $E(q_1) = \{q_1\} \cup E(q_2)$
- $E(q_2) = \{q_2\}$

Quindi è equivalente

- $E(q_0) = \{q_0, q_1, q_2\}$
- $E(q_1) = \{q_1, q_2\}$
- $E(q_2) = \{q_2\}$

3.6 Funzione di Transizione Estesa NFA

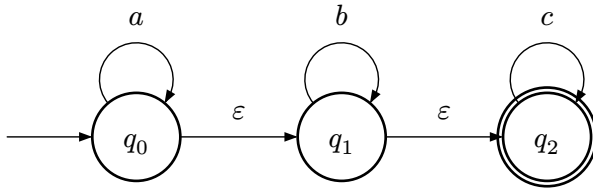
Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito non deterministico, la funzione di transizione estesa $\hat{\delta} : Q \times \Sigma^* \rightarrow P(Q)$ è definita ricorsivamente come segue

- Passo base: $\forall q \in Q, \hat{\delta}(q, \varepsilon) = E(q)$
- Passo ricorsivo: $\forall q \in Q, w \in \Sigma^*, a \in \Sigma,$

$$\hat{\delta}(q, wa) = E\left(\bigcup_{p \in \hat{\delta}(q, w)} \delta(p, a)\right) = \bigcup_{p \in \hat{\delta}(q, w)} E(\delta(p, a))$$

Quindi per ogni possibili cammino in cui lo stato segue una transizione in a

3.6.1 Esempio Funzione di Transizione Estesa



- $E(q_0) = \{q_0, q_1, q_2\}$
- $E(q_1) = \{q_1, q_2\}$
- $E(q_2) = \{q_2\}$

Calcolo $\hat{\delta}(q_0, ca)$

- $\hat{\delta}(q_0, ca) = E\left(\bigcup_{p \in \hat{\delta}(q_0, c)} \delta(p, a)\right) =$
- $\hat{\delta}(q_0, c) = E\left(\bigcup_{p \in \hat{\delta}(q_0, \varepsilon)} \delta(p, c)\right) =$
- $\hat{\delta}(q_0, \varepsilon) = E(q_0) = \{q_0, q_1, q_2\}$

Quindi risalendo con la ricorsione

- $\hat{\delta}(q_0, c) = E\left(\bigcup_{p \in \hat{\delta}(q_0, \varepsilon)} \delta(p, c)\right) =$
- $= E(\delta(q_0, c) \cup \delta(q_1, c) \cup \delta(q_2, c)) =$
- $E(\emptyset \cup \emptyset \cup \{q_2\}) = E(q_2) = \{q_2\}$

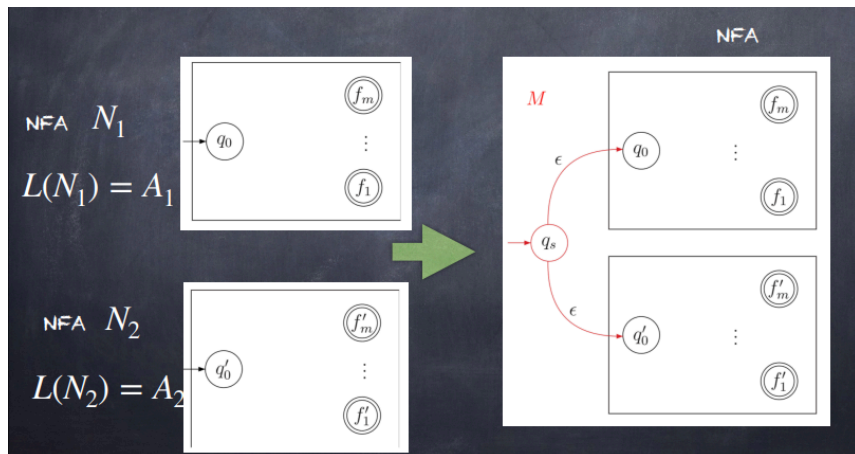
Quindi risalendo nella ricorsione

- $\hat{\delta}(q_0, ca) = E\left(\bigcup_{p \in \hat{\delta}(q_0, c)} \delta(p, a)\right) =$
- $E(\delta(q_2, a)) = E(\emptyset) = \emptyset$

Questo automa accetta $L = \{a^i b^j c^k \mid i, j, k \geq 0\}$

3.7 Teorema 1.45 Chiusura Classe REG Unione NFA

Dato due qualsiasi linguaggi regolari A_1 e $A_2 \Rightarrow A_1 \cup A_2$ è



Dimostrazione:

Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed

$N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2

Costruiamo $N = (Q, \Sigma, \delta, q, F)$ per riconoscere $A_1 \cup A_2$

- $Q = \{q_0\} \cup Q_1 \cup Q_2$

Gli stati di N sono tutti gli stati di N_1 e N_2 con l'aggiunta di un nuovo stato iniziale q_0

- Stato q_0 è lo stato iniziale di N
- Insieme degli stati accettati $F = F_1 \cup F_2$

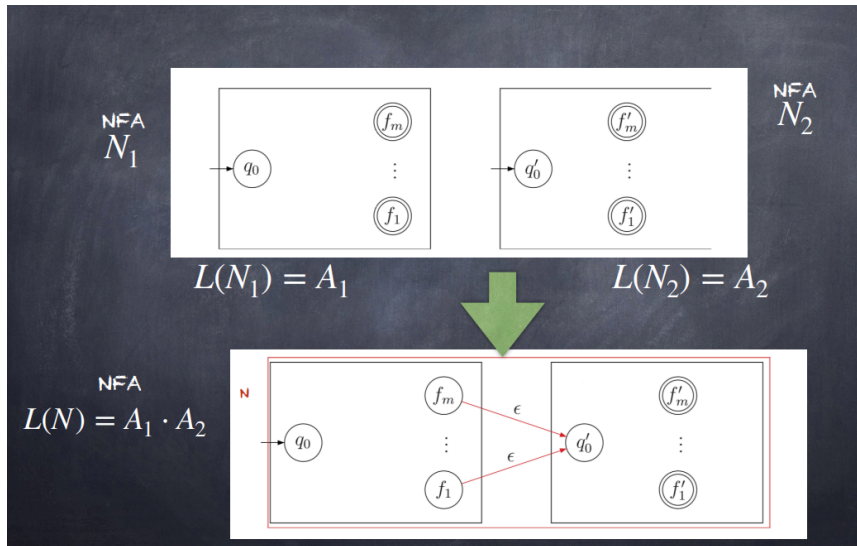
Gli stati accettati di N sono tutti gli stati accettati di N_1 e N_2 , in questo modo N accetta se N_1 accetta o N_2 accetta

- Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

3.8 Teorema 1.47 Chiusura Classe REG

Concatenazione NFA



Dimostrazione:

Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed

$N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2

Costruiamo $N = (Q, \Sigma, \delta, q_1, F_2)$ per riconoscere $A_1 \cdot A_2$

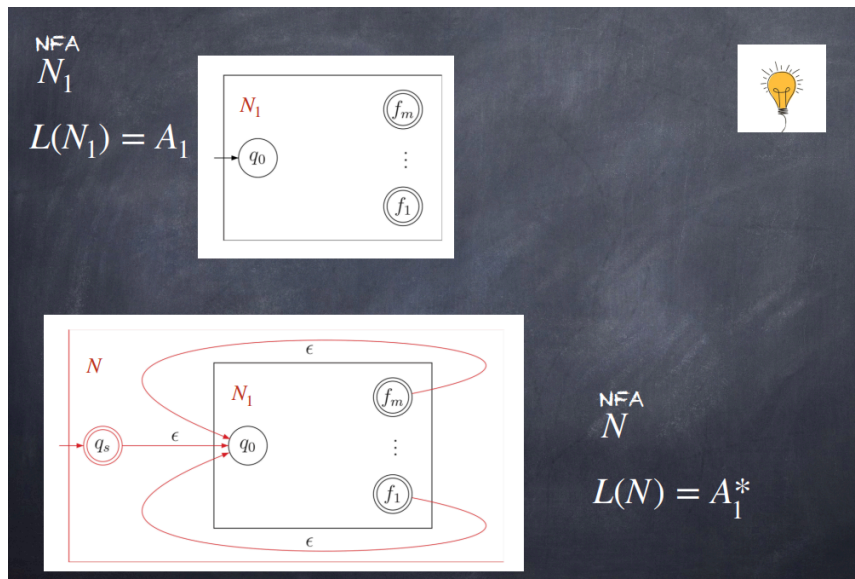
- $Q = Q_1 \cup Q_2$

Gli stati di N sono tutti gli stati di N_1 e N_2

- Stato q_1 è uguale allo stato iniziale di N_1
- Gli stati accettati F_2 sono uguali agli stati accettati di N_2
- Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ e } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

3.9 Teorema 1.49 Chiusura Classe REG Chiusura di Kleene



Dimostrazione:

Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere A_1^*

- $Q = \{q_0\} \cup Q_1$

Gli stati di N sono gli stati di N_1 più un nuovo stato iniziale

- Stato q_0 nuovo stato iniziale
- $F = \{q_0\} \cup F_1$

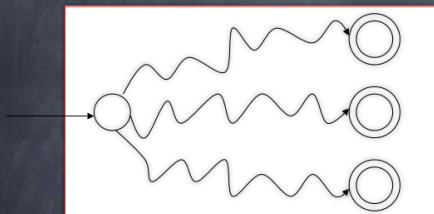
Gli stati accettati sono i vecchi stati accettati più il nuovo stato iniziale

- Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ e } a = \epsilon \\ \{q_1\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

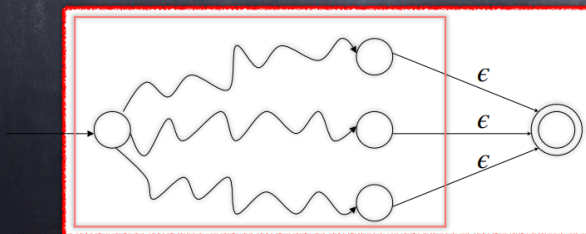
3.10 Chiusura Classe REG Reverse

NFA (vale anche per DFA)



In generale

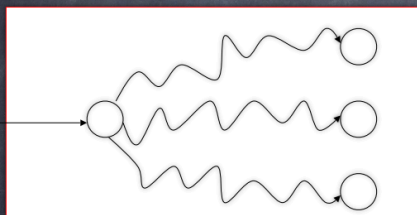
NFA equivalente



Singolo stato di accettazione

Caso estremo

NFA (o DFA) senza stati di accettazione...



Aggiungere uno stato finale
Senza transizioni

Sia $M = (Q, \Sigma, \delta, q_0, F)$ che riconosce L .

Supponiamo di voler progettare un automa finito $R = (Q', \Sigma, \delta', q'_0, F')$ che riconosce il reverse di L (per non perdere di generalità, sia R un NFA). Poiché $\text{Rev}(L)$ deve riconoscere le stesse stringhe di L ma lette al contrario, l'idea è quella di far partire la computazione dalla fine (stati finali) e di andare a ritroso verso l'inizio (stato iniziale).

Formalmente, per ogni $p \in \delta(q_a)$, per ogni $q \in Q, a \in \Sigma$ si ha che $q \in \delta'(p, a)$.

In questo modo stiamo “invertendo le frecce” di M .

Un problema può essere il fatto che la computazione per partire dalla fine, dovrebbe partire dagli stati finali, ma lo stato iniziale degli automi è unico. Quindi l'automa R avrà gli stati di M , con l'aggiunta di un nuovo stato che sarà iniziale e dal quale, con epsilon transizioni, andrò negli stati finali. Lo stato iniziale di M sarà lo stato finale di R .

Formalmente, $F' = \{q_0\}$ mentre q'_0 sarà un nuovo stato, cioè $Q' = Q \cup \{q'_0\}$.

Inoltre $\delta'(q'_0, \varepsilon) = F$, mentre $\delta'(q'_0, a) = \emptyset$, per a diverso dalla stringa vuota.

Non è necessario provare rigorosamente che

$\delta'(q'_0, \varepsilon) \in F$ se e solo se $\delta'(q'_0, w^R) \in F$, perché segue dalla costruzione. Nelle slides della lezione c'è un esempio di costruzione.

3.11 Chiusura Classe REG Prefissi e Suffissi

- Prefissi di L : $\{x \in \Sigma^* \mid \exists y \in \Sigma^* : xy \in L\}$
- Suffissi di L : $\{y \in \Sigma^* \mid \exists x \in \Sigma^* : xy \in L\}$

Si può dimostrare che se L è regolare allora:

- $\text{Pref}(L)$ è regolare
- $\text{Suff}(L)$ è regolare

3.11.1 Chiusura Classe REG Prefissi

Innanzitutto, ricordiamo che dato un linguaggio L , indichiamo con $\text{Pref}(L) = \{x \in \Sigma^* \mid \exists y \in L, x, y \in \Sigma^*\}$.

Seguiamo il ragionamento fatto durante la lezione e supponiamo che l'automa $M = (Q, \Sigma, \delta, q_0, F)$ che riconosce L sia un NFA in cui non ci sono archi uscenti dagli stati finali. Questa ipotesi è facile da soddisfare aggiungendo all'automa un nuovo stato finale in cui entro leggendo la stringa vuota a partire da ogni stato finale di M .

Formalmente, consideriamo un automa $M' = (Q', \Sigma, \delta', q_0, \{q_F\})$ che riconosce L , con $Q' = Q \cup \{q_F\}$ dove $\delta'(q, a) = \delta(q, a)$, per ogni q in Q , a in Σ , e in più, per ogni q in F si ha che $q_F \in \delta'(q, \varepsilon)$.

Senza perdere di generalità, essendo un NFA, possiamo eliminare tutti gli stati pozzo, cioè quelli che non consentiranno di raggiungere gli stati finali.

Costruiamo ora $P = (Q', \Sigma, \delta'', q_0, \{q_F\})$ dove $\delta''(q, a) = \delta'(q, a)$ per ogni $q \in Q'$, $a \in \Sigma$ e inoltre per ogni $q \in Q'$, $q_F \in \delta''(q, \varepsilon)$, se q raggiunge uno stato finale. Non è necessario provare rigorosamente che $L(P) = \text{Pref}(L)$, cioè l'insieme dei prefissi di L , perché se $w = xy$, e w è accettata, allora $\hat{\delta}(q_0, w) = \hat{\delta}(q_0, xy) = \hat{\delta}(\hat{\delta}(q_0, x), y)$.

Sia $t \in \hat{\delta}(q_0, x)$. Allora $\hat{\delta}(t, y) = \{q_F\}$ se e solo se $\{q_F\} \in \hat{\delta}'(q_0, x)$, perché c'è la epsilon transizione da ogni stato come t che viene raggiunto da $\hat{\delta}'(q_0, x)$ verso q_F .

Inoltre, l'ipotesi garantisce che se raggiungo q_F non esco verso nessun altro stato. Se così non fosse, non garantirei di riconoscere solo i prefissi di L , ma potrei continuare e leggere altre stringhe.

3.11.2 Chiusura Classe REG Suffissi

Innanzitutto, ricordiamo che dato un linguaggio L , indichiamo con $\text{Suff}(L) = \{y \in \Sigma^* \mid w = xy \in L, x, y \in \Sigma^*\}$.

Anche per questo esercizio, seguiamo il ragionamento fatto durante la lezione e supponiamo che l'automa $M = (Q, \Sigma, \delta, q_0, F)$ che riconosce L sia un NFA in cui non ci sono archi entranti nello stato iniziale. Se infatti questo non accadesse, potremmo trovarci in una situazione simmetrica a quella dei prefissi, riconoscendo stringhe non suffissi di L .

Questa ipotesi è facile da soddisfare aggiungendo all'automa un nuovo stato iniziale da cui usciamo leggendo la stringa vuota verso lo stato iniziale di M .

Formalmente, consideriamo un automa $M' = (Q', \Sigma, \delta', q'_0, F)$ che riconosce L , con $Q' = Q \cup \{q'_0\}$ dove $\delta'(q, a) = \delta(q, a)$, per ogni q in Q , a in Σ , mentre $\{q'_0\} \in \delta'(q'_0, \varepsilon)$ e $\delta'(q'_0, a) = \emptyset$.

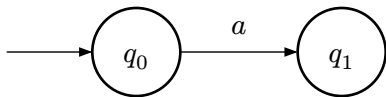
Costruiamo ora $S = (Q', \Sigma, \delta'', q'_0, F)$ dove $\delta''(q, a) = \delta'(q, a)$ per ogni $q \in Q'$, $a \in \Sigma$ e inoltre per ogni $q \in Q'$, $q \in \delta''(q'_0, \varepsilon)$, se q è raggiungibile dallo stato iniziale.

Non è necessario provare rigorosamente che $L(S) = \text{Suff}(L)$, cioè l'insieme dei suffissi di L , perché se $w = xy$, e w è accettata, allora $\hat{\delta}'(q_0, \varepsilon) \subseteq F$, perché c'è la epsilon transizione da ogni stato raggiunto da $\hat{\delta}'(q_0, \varepsilon)$ ad uno stato finale.

Quindi con la epsilon transizione, da q_0 vado in $\hat{\delta}'(q_0, \varepsilon)$ a partire dal quale so che posso leggere anche y e arrivare in uno stato finale, quindi accettando y . L'ipotesi che sullo stato iniziale non ci siano archi entranti, garantisce che posso solo “saltare” la x nella lettura

4 Ogni NFA è un DFA e Viceversa

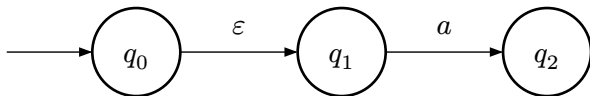
Per i DFA $\hat{\delta}(q, a) = \delta(q, a)$, $a \in \Sigma$ è sempre vero



$$\delta(q_0, a) = q_1$$

$$\hat{\delta}(q_0, a) = \delta(\hat{\delta}(q_0, \varepsilon), a) = \delta(q_0, a)$$

Per gli NFA, invece, $\hat{\delta}(q, a) = \delta(q, a)$, $a \in \Sigma$ non è vero in generale



$$\delta(q_0, a) = \emptyset$$

$$\hat{\delta}(q_0, a) = \hat{\delta}(q_0, \varepsilon a) =$$

$$= E\left(\bigcup_{p \in \hat{\delta}(q_0, \varepsilon)} \delta(q_0, a)\right) =$$

$$= E(\delta(q_0, a) \cup \delta(q_1, a)) =$$

$$= E(\emptyset \cup q_2) = \emptyset \cup \{q_2\} = \{q_2\}$$

4.1 Ogni DFA è un NFA

$$\text{DFA } M = \{Q, \Sigma, \delta, q_0, F\}, \quad \delta : Q \times \Sigma \rightarrow Q$$

$$\text{NFA } N = \{Q, \Sigma_\epsilon, \delta_N, q_0, F\}, \quad \delta_N : Q \times \Sigma_\epsilon \rightarrow P(Q)$$

$$L(N) = L(M)$$

$$\delta(q, a) = p \Rightarrow \delta_{N(q,a)} = \{p\}$$

Quindi considero un singolo stato di un DFA come il singleton di quello stato nel NFA, a parità di linguaggio

$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}_{M(q_0,w)} \in F\} =$$

$$\{w \in \Sigma^* \mid \hat{\delta}_{N(q_0,w)} \cap F \neq \emptyset\} = L(N)$$

4.2 Teorema 1.39 NFA e DFA equivalenti

Per ogni automa finito non deterministico A esiste un automa finito deterministico B tale che $L(A) = L(B)$

- Se nel NFA parto da q_0 ma ci sono anche ε -transizioni, allora anche quegli stati sono iniziali
- Se nel NFA termino in uno stato finale allora nel DFA ogni stato che contiene uno stato finale del NFA rappresenta un cammino di accettazione

Dimostrazione:

$$\forall \text{ NFA } N = \{Q_N, \Sigma, \delta_N, q_N, F_N\}$$

$$\exists \text{ DFA } M = \{Q_M, \Sigma, \delta_M, q_M, F_M\}$$

$$\text{t.c. } L(N) = L(M)$$

Dim:

$$Q_M = P(Q_N), q_M = E(q_N)$$

$$F_M = \{R \in Q_M \mid R \cap F_N \neq \emptyset\}$$

$$\delta_M(R, a) = E(\cup_{r \in R} \delta_N(r, a))$$

Ora dimostriamo che $L(N) = L(M)$

$$\forall w \in \Sigma^*, \hat{\delta}(q_M, w) = \hat{\delta}(q_N, w)$$

- Passo base:

Sia $w = \varepsilon$

$$\hat{\delta}(q_M, \varepsilon) \stackrel{\text{def DFA}}{=} q_M \stackrel{\text{Costruzione}}{=} E(q_N) \stackrel{\text{def NFA}}{=} \hat{\delta}(q_N, \varepsilon)$$

- Passo induttivo:

Sia $w = xa, x \in \Sigma^*, a \in \Sigma$

Supponiamo $\hat{\delta}_M(q_M, x) = \hat{\delta}_N(q_N, x)$

$$\hat{\delta}_M(q_M, xa) \stackrel{\text{def DFA}}{=} \delta_M(\hat{\delta}_M(q_M, x), a) \stackrel{\text{Costruzione}}{=}$$

$$E\left(\bigcup_{r \in \hat{\delta}_M(q_M, x)} \delta_N(r, a)\right)$$

Applico ipotesi induttiva

$$= E\left(\bigcup_{r \in \hat{\delta}_N(q_N, x)} \delta_N(r, a)\right) = \hat{\delta}_N(q_n, xa)$$

4.2.1 Corollario Teorema 1.39

Un linguaggio è regolare se e solo se esiste un automa finito non deterministico che lo riconosce

- Quindi se esiste un NFA, un DFA o usando le proprietà di chiusura

5 Espressioni Regolari

Teorema di Kleene: Un linguaggio è regolare se e solo se esiste un'espressione regolare che lo rappresenta, quindi vale il viceversa

5.1 Definizione Formale Espressioni Regolari

- Passo Base:

Per ogni $a \in \Sigma$, a è un'espressione regolare

ε è un'espressione regolare

$\emptyset$ è un'espressione regolare

- Passo Ricorsivo:

Se E_1, E_2 sono espressioni regolari, allora

(E_1) è un'espressione regolare

$(E_1 \cup E_2)$ è un'espressione regolare

$(E_1 E_2)$ è un'espressione regolare

(E_1^*) è un'espressione regolare

5.2 Precedenza Operatori

Operatore	Precedenza
*	1
.	2
$\cup$	3

5.3 Definizione Ricorsiva Linguaggi Rappresentati REG

Data un'espressione regolare E , indicheremo con $L(E)$ il linguaggio che essa rappresenta, definito come segue:

- Passo base:

Per ogni $a \in \Sigma$, $L(A) = \{a\}$

$L(\varepsilon) = \{\varepsilon\}$

$L(\emptyset) = \emptyset$

- Passo Ricorsivo:

Se E_1, E_2 sono espressioni regolari allora

$L((E_1)) = L(E_1)$ è un'espressione regolare

$L(E_1 \cup E_2) = L(E_1) \cup L(E_2)$ è un'espressione regolare

$L(E_1 E_2) = L(E_1)L(E_2)$ è un'espressione regolare

$L(E_1^*) = L(E_1)^*$ è un'espressione regolare

5.4 Proprietà Algebriche RE

- $E_1 \cup E_2 = E_2 \cup E_1$
- $(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$
- $\emptyset \cup E_1 = E_1$
- $(E_1 E_2) E_3 = E_1 (E_2 E_3)$
- $\emptyset E_1 = E_1 \emptyset = \emptyset$
- $\varepsilon E_1 = E_1 \varepsilon = E_1$
- $E_1 (E_2 E_3) = E_1 E_2 \cup E_1 E_3$
- $(E_1 \cup E_2) \cup E_3 = E_1 E_3 \cup E_2 E_3$
- $E_1 \cup E_1 = E_1$
- $((E_1)^*)^* = E_1^*$
- $E^+ = EE^*$
- $(E_1 \cup E_2)^* = (E_1^* E_2^*)^*$

6 Pumping Lemma

Se A è un linguaggio regolare, allora $\exists p > 0$ tale che $\forall$ stringa $s \in A$ di lunghezza almeno p (cioè $|s| \geq p$), esistono x, y, z tali che $s = xyz$ e valgono le seguenti condizioni:

- $xy^iz \in A, \forall i \geq 0$
- $|y| \geq 1$
- $|xy| \leq p$

6.1 Tecnica Per Provare la Non Regolarità

- Supponiamo A regolare (quindi vale PL)
 - Deve esistere $p > 0$ tale che tutte le stringhe di A di lunghezza maggiore o uguale a p possono essere iterate (non sappiamo p chi sia, ma esiste)
3. Troviamo una stringa s di A che ha lunghezza maggiore o uguale a p che però non può essere iterata considerando TUTTI i possibili modi di fattorizzarla in x, y, z (con le condizioni)
 4. Per ogni fattorizzazione, troviamo i tale che xy^iz non appartiene a A . Assurdo!

7 Macchine di Turing

7.1 Descrizione Informale

- Utilizza un nastro semi-infinito
- Presenta una testina di lettura e scrittura che può muoversi avanti e dietro
- Ha due stati speciali q_{accept} e q_{reject} che hanno effetto immediato

7.2 Definizione Formale MdT

Un macchina di Turing deterministica è una settupla

$$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

- Q : Insieme finito degli stati
- Σ : Alfabeto dei simboli in input, con $\sqcup \notin \Sigma$
- Γ : Alfabeto finito dei simboli di nastro, con $\sqcup \in \Gamma, \Sigma \subset \Gamma, L, R \notin \Gamma$
- δ : Funzione di transizione

$$(Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

- $q_0 \in Q$: Stato iniziale
- $q_{\text{accept}} \in Q$: Stato di accettazione
- $q_{\text{reject}} \in Q$: Stato di rifiuto, $q_{\text{accept}} \neq q_{\text{reject}}$

7.3 Funzione di Transizione MdT

Se $\delta(q, \gamma) = (q', \gamma', d)$ sappiamo che $q, q' \in Q, \gamma, \gamma' \in \Gamma, d \in \{L, R\}$

7.4 Diagramma di Stato

Grafo i cui nodi sono gli stati della macchina e le etichette sugli archi hanno la seguente forma

$$\text{simbolo} \rightarrow \text{simbolo}, d$$

Con il simbolo $\in \Gamma$ e $d \in \{L, R\}$

7.5 Computazione di una MdT Informale

- Inizia dallo stato iniziale q_0
- Con l'input $w \in \Sigma^*$ posizionato sulla parte più a sinistra del nastro, e con la testina posizionata sulla cella più a sinistra del nastro
- Se input ε , allora il nastro contiene solo $\sqcup$
- La computazione di M procede fino a quando non viene raggiunto uno stato di accettazione o rifiuto. Se nessuno dei due stati viene raggiunto, la computazione di M continua per sempre
- Dato che Σ non contiene $\sqcup$ all'inizio della computazione il primo simbolo $\sqcup$ segna la fine della computazione
- La computazione termina se ha raggiunto q_{accept} e q_{reject} , oppure può non terminare

7.6 Configurazione di una MdT

Una configurazione C di una MdT $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ è una stringa $C = uqv \in \Gamma^* Q \Gamma^S$, con $\Gamma^S = \Gamma^* (\Gamma \setminus \{\sqcup\}) \cup \{\sqcup, \varepsilon\}$ con

- $q \in Q$: Stato corrente di M
- $uv \in \Gamma^*$ è il contenuto del nastro
 - Convenzione nel eliminare tutti i simboli $\sqcup$ che seguono ultimo carattere di v se $v \neq \varepsilon$
 - Tutti i simboli $\sqcup$ che seguono u altrimenti
- Testina posizionata sul primo simbolo di v se $v \neq \varepsilon$, su $\sqcup$ altrimenti

Una configurazione C di una MdT M si dice:

- **Iniziale:** (Con input w) se $C = q_0 w, w \in \Sigma^*$
- **Di arresto:** Se $C = uqv, u, v \in \Gamma^*$ e $q \in \{q_{\text{accept}}, q_{\text{reject}}\}$
 - Non esiste nessuna configurazione C' tale che $C \rightarrow C'$
- **Di accettazione:** Se $C = uqv, u, v \in \Gamma^*$ e $q = q_{\text{accept}}$
- **Di rifiuto:** Se $C = uqv, u, v \in \Gamma^*$ e $q = q_{\text{reject}}$

7.6.1 Configurazioni Particolari

- Se $C = qv$ allora la testina è posizionata sulla prima cella del nastro

- Se $C = uq$ allora la testina è posizionata sulla prima cella della porzione del nastro contenete solo $\sqcup$

7.7 Passo di Computazione

Sia $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ una MdT deterministica

Siano $q_i, q_j \in Q$, $a, b, c \in \Gamma$, e $u, v \in \Gamma^*$

Diremo che

$$uaq_i bv \text{ produce } uq_j acv$$

Se $\delta(q_i, b) = (q_j, c, L)$

Diremo che

$$uaq_i bv \text{ produce } uacq_j v$$

Se $\delta(q_i, b) = (q_j, c, R)$

Quindi $C_1 \rightarrow C_2$ prende il nome di passo di computazione

7.8 Computazione

Siano C, C' configurazioni

$C \xrightarrow{*} C'$ se esistono configurazioni $C_1, \dots, C_k, k \geq 1$ tali che:

- $C_1 = C$
- $C_i \rightarrow C_{i+1}$, per $i \in \{1, \dots, k-1\}$
- $C_k = C'$

Diremo che $C \xrightarrow{*} C'$ è una **computazione**

Sia M una MdT e C una configurazione ci sono tre possibili casi:

- $C \xrightarrow{*} C'$ con $C' = uq_{\text{accept}}v$ configurazione di accettazione
 - M si ferma in q_{accept}
- $C \xrightarrow{*} C'$ con $C' = uq_{\text{reject}}v$ configurazione di rifiuto
 - M si ferma in q_{reject}
- Per ogni configurazione C' tale che $C \xrightarrow{*} C'$ esiste una configurazione C'' tale che $C \xrightarrow{*} C' \rightarrow C''$
 - M non si arresta

7.9 Parola Accetta da una MdT

Un MdT M accetta una parola $w \in \Sigma^*$ se esiste una computazione $C \xrightarrow{*} C'$, dove $C = q_0, w$ è la configurazione iniziale di M con input w e $C' = uq_{\text{accept}}v$ è una configurazione di accettazione

Quindi M accetta $w \in \Sigma^*$ se e solo se esistono configurazioni

$C_1, \dots, C_k$ di M tali che:

- $C_1 = q_0 w$ è la configurazione iniziale di M con input w
- $C_i \rightarrow C_{i+1}$ per ogni $i \in \{1, \dots, k-1\}$
- C_k è una configurazione di accettazione

7.10 Parola Rifiutata da una MdT

Un MdT M rifiuta una parola $w \in \Sigma^*$ se esiste una computazione $C \xrightarrow{*} C'$, dove $C = q_0, w$ è la configurazione iniziale di M con input w e $C' = uq_{\text{reject}}v$ è una configurazione di rifiuto

Quindi M si ferma su $w \in \Sigma^*$ se M accetta w oppure rifiuta w , cioè esistono configurazioni $C_1, \dots, C_k$ di M tali che:

- $C_1 = q_0 w$ è la configurazione iniziale di M con input w
- $C_i \rightarrow C_{i+1}$ per ogni $i \in \{1, \dots, k-1\}$
- C_k è una configurazione di arresto

7.11 Linguaggio Riconosciuto da una MdT

Sia $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ un MdT, il linguaggio $L(M)$ riconosciuto da M è l'insieme delle stringhe che M accetta

$$L(M) = \{w \in \Sigma^* \mid \exists u, v \in \Gamma^* \ q_0 w \xrightarrow{*} uq_{\text{accept}}v\}$$

Quindi

$$L(M) = \{w \in \Sigma^* \mid M \text{ accetta } w\}$$

- Data una MdT M , esiste sempre il linguaggio riconosciuto da M ma non è detto che esista il linguaggio deciso da M , ciò è possibile se e solo se M è un decider
- Se M è un decider, esiste il linguaggio deciso da M e coincide con il linguaggio riconosciuto da M
- Se M non è un decider, esiste il linguaggio riconosciuto da M ma non il linguaggio deciso da M

7.12 MdT Decisore

Sia $R(M) = \{w \in \Sigma^* \mid M \text{ rifiuta } w\}$

In generale $L(M) \cup R(M)$ non coincide con Σ^* , ma se coincide allora M è un decider

Definizione Formale:

Un MdT $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ è un decisore se per ogni $w \in \Sigma^*$, esistono $u, v \in \Gamma^*$ e $q \in \{q_{\text{accept}}, q_{\text{reject}}\}$ tali che:

$$q_0 w \xrightarrow{*} uqv$$

7.13 Linguaggio Deciso

Un linguaggio L è deciso da un MdT $M =$

$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ se M è un decisore ed $L = L(M)$

Una MdT $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ decide un linguaggio L se per ogni $w \in \Sigma^*$, $q_0 w \xrightarrow{*} C$ con $C = uqv$ configurazione di arresto ed $L = L(M)$

7.14 Linguaggio Riconoscibile

Un linguaggio $L \subseteq \Sigma^*$ è Turing riconoscibile se esiste una macchina di Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ tale che

- M riconosce L

$$L = L(M) = \{w \in \Sigma^* \mid \exists u, v \in \Gamma^* \ q_0 w \xrightarrow{*} u q_{\text{accept}} v\}$$

7.15 Linguaggio Decidibile

Un linguaggio $L \subseteq \Sigma^*$ è decidibile se esiste una macchina di Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ tale che

- M riconosce L

$$L = L(M) = \{w \in \Sigma^* \mid \exists u, v \in \Gamma^* \ q_0 w \xrightarrow{*} u q_{\text{accept}} v\}$$

- M si arresta su ogni input

Per ogni $w \in \Sigma^*$, $q_0 w \xrightarrow{*} uqv$ con $q \in \{q_{\text{accept}}, q_{\text{reject}}\}$

Rappresenta un sottoinsieme proprio dei linguaggi Turing riconoscibili, quindi un linguaggio L è Turing riconoscibile ma non decidibile se:

- Esiste una MdT che riconosce L
- Non esiste nessuna MdT tale che M accetta tutte le stringhe di L e rifiuta tutte quelle che appartengono al complemento $\bar{L}$

8 Varianti di MdT

Siano T_1 e T_2 due famiglie di modelli computazionali. Per dimostrare che i modelli in T_1 hanno lo stesso potere computazionale dei modelli in T_2 occorre far vedere che per ogni macchina $M_1 \in T_1$ esiste $M_2 \in T_2$ equivalente ad M_1 e viceversa

- Due macchine sono equivalenti se hanno lo stesso linguaggio

8.1 Macchine di Turing che Stanno Ferme

Questa caratteristica non aggiunge potere computazionale al modello scelto

La parte non ovvia è mostrare che possiamo trasformare qualsiasi macchina di Turing che ha la possibilità di stare ferma in una macchina di Turing equivalente che non ha tale capacità

- Lo dimostriamo costruendo una MdT in cui simuliamo ogni transizione in cui resta ferma come una transizione che va prima a destra e poi ritorna a sinistra

Sia $T_{(L,R)}$ l'insieme delle macchine di Turing

$$(Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

Chiamiamo $T_{(L,R,S)}$ il nuovo insieme di macchine di Turing tali che $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ in cui $Q, \Sigma, \Gamma, q_0, q_{\text{accept}}, q_{\text{reject}}$ sono definiti come in una MdT deterministica e la funzione di transizione δ definita nel modo seguente:

$$(Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$$

Se $\delta(q, \gamma) = (q', \gamma', d)$ e se M si trova nello stato q con la testina posizionata su una cella γ , alla fine della transizione

- M si trova nello stato q'
- $\gamma' \in \Gamma$ è il simbolo scritto sul nastro su cui la testina si trovava all'inizio della transizioni
- La testina si trova sulla stessa cella dell'inizio della transizione se $d = S$, a sinistra se $d = L$ e a destra se $d = R$

Le nozioni di configurazione di linguaggio deciso e di linguaggio riconosciuto da $M' \in T_{(L,R)}$ sono estese in maniera ovvia alle macchine M in $T_{(L,R,S)}$

I modelli in $T_{(L,R)}$ hanno lo stesso potere computazionale dei modelli in $T_{(L,R,S)}$

Se $M' = (Q, \Sigma, \Gamma, \delta', q_0, q_{\text{accept}}, q_{\text{reject}}) \in T_{(L,R)}$

Definiamo $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}) \in T_{(L,R,S)}$

Con $\delta(q, \gamma) = \delta'(q, \gamma)$ per ogni $(q, \gamma) \in (Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma$

Ovviamente $L(M) = L(M')$

Viceversa se $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}) \in T_{(L,R,S)}$

Definiamo $M' = (Q \cup Q^c, \Sigma, \Gamma, \delta', q_0, q_{\text{accept}}, q_{\text{reject}}) \in T_{(L,R)}$ dove

- Se $\delta(q_i, \gamma) = (q_j, \gamma', d)$ e $d \in \{L, R\}$ allora $\delta'(q_i, \gamma) = \delta(q_i, \gamma)$
- Se $\delta(q_i, \gamma) = (q_j, \gamma', S)$ allora $\delta'(q_i, \gamma) = (q_j^c, \gamma', R)$ e $\delta'(q_j^c, \eta) = (q_j, \gamma', L)$ per ogni $\eta \in \Gamma$
- $Q^c = \{q^c \mid (q, \gamma, S) \in \delta(Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma\}$

Anche in questo caso $L(M) = L(M')$

8.2 Macchine di Turing Multinastro

Dato un numero intero positivo K , una macchina di Turing con k nastri è una settupla

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

Dove $Q, \Sigma, \Gamma, q_0, q_{\text{accept}}, q_{\text{reject}}$ sono definiti come in una MdT deterministica e la funzione di transizione δ è definita come segue

$$\delta : (Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

Con Γ^k il prodotto cartesiano di k coppie di Γ e

$\{L, R, S\}^k$ è il prodotto cartesiano di k coppie di $\{L, R, S\}$

8.2.1 Funzione di Transizione

Il codominio della funzione di transizione di una macchina di Turing a k nastri è un insieme di sequenze $(q_j, b_1, \dots, b_k, d_1, \dots, d_k)$ di lunghezza $(2k + 1)$ con

- $q_j \in Q$
- $b_t \in \Gamma, t \in \{1, \dots, k\}$
- $d_t \in \{L, R, S\}, t \in \{1, \dots, k\}$

Se $\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, \dots, b_k, d_1, \dots, d_k)$ e se M si trova nello stato q_i con le k testine posizionate sulle celle contenenti $a_1, \dots, a_k$ alla fine della transizione

- M si trova nello stato q_j
- $b_t \in \Gamma$ è il simbolo scritto sulla cella del t -esimo nastro su cui la testina si trovava all'inizio della transizione, $t \in \{1, \dots, k\}$
- La testina sul t -esimo nastro si trova sulla stessa cella in cui si trovava all'inizio della computazione se $d_t = S$, a sinistra se $d_t = L$ e a destra se $d_t = R, t \in \{1, \dots, k\}$

8.2.2 Computazione

Una MdTM $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ inizia la computazione

- Partendo dallo stato iniziale q_0
- Con l'input $w \in \Sigma^*$ posizionato nella parte più a sinistra del primo nastro
- Gli altri nastri contengono solo $\sqcup$
- Tutte le testine sono posizionate sulle prime celle dei rispettivi nastri

Le nozioni di configurazione, passo di computazione, di linguaggio deciso e di linguaggio riconosciuto da una MdT sono estese in maniera ovvia alle macchine MdTM

Una configurazione ha la forma

$$(u_1 q v_1, \dots, u_k q v_k)$$

Dove $q \in Q$ è lo stato corrente di M e per $t \in \{1, \dots, k\}$, $u_t q v_t \in \Gamma^* Q \Gamma^S$, con $\Gamma^S = \Gamma^* (\Gamma \setminus \{\sqcup\}) \cup \{\sqcup, \varepsilon\}$

La testina del t -esimo nastro è posiziona sul primo simbolo di v_t , se $v_t \neq \varepsilon$, su $\sqcup$ altrimenti

$(q_0, \dots, q_0)$ è una configurazione iniziale

$(u_1 q_{\text{accept}} v_1, \dots, u_k q_{\text{accept}} v_k)$ è una configurazione di accettazione

8.2.3 Linguaggio Riconosciuto

$$L(M) = \{w \in \Sigma^* \mid \exists u_t, v_t \in \Gamma^*, t \in \{1, \dots, k\} : \}$$

$$(q_0 w, \dots, q_0) \xrightarrow{*} (u_1 q_{\text{accept}} v_1, \dots, u_k q_{\text{accept}} v_k)$$

8.3 Equivalenza MdTM e MdT

Per ogni MdT multinastro $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ esiste una macchina di Turing a nastro singolo S equivalente a M , cioè tale che $L(M) = L(S)$

• **Dimostrazione:**

Supponiamo che M abbia k nastri

Per ogni configurazione di M

$$(u_1 q v_1, \dots, u_k q v_k)$$

La macchina S che simula M deve codificare su un solo nastro:

- Il contenuto $u_1 v_1, \dots, u_k v_k$ dei k nastri
- La posizione di ciascuna testina del nastro

Idea della dimostrazione:

- Il contenuto del nastro di S è la concatenazione di k blocchi separati da $\#$
- Un elemento $\dot{\gamma}$, con $\gamma \in \Gamma$, nel blocco t -esimo indica la posizione della testina del nastro t -esimo, $t \in \{1, \dots, k\}$, e quindi l'alfabeto dei simboli di nastro di S è $\Gamma \cup \{\dot{\gamma} \mid \gamma \in \Gamma\} \cup \{\#, \dot{\#}\}$

Quindi sia $w = w_1, \dots, w_n$ una stringa input, $w_t \in \Sigma, t \in \{1, \dots, n\}$

- **Generazione della configurazione iniziale di M :**

S passa alla configurazione

$$q' \# w_1 \dots w_n \# \dot{\sqcup} \# \dots \# \dot{\sqcup} \#$$

Con la conseguente aggiunta di stati addizionali

- Per simulare un passo di computazione di M , per effetto dell'applicazione di $\delta(q, a_1, \dots, a_k) = (s, b_1, \dots, b_k, d_1, \dots, d_k)$, la macchina S scorre il dato sul nastro dal primo $\#$ al $(k + 1)$ -esimo $\#$ da sinistra a destra e viceversa due volte
 - La prima volta S determina quali sono i simboli correnti di M , memorizzando nello stato i simboli marcati sui singoli nastri

- La seconda volta S esegue su ogni sezione le azione che simulano quelle di M

Nel corso della simulazione S potrebbe spostare delle testine sul simbolo $\#$, in questo caso S deve spostare tutto il contenuto verso destra di una posizione, a partire da $\dot{\#}$ fino all'ultimo $\#$ e scrivere $\sqcup$ nella cella vuota creata al posto di $\dot{\#}$

Poi la MdT entro nello stato che ricorda il nuovo stato di M e riposiziona la testina all'inizio del nastro

Se si ferma M , anche S si ferma rimuovendo i separatori $\#$ e sostituendo i caratteri $\dot{a}_t$ con a_t

8.4 MdTM e MdT Linguaggi Riconoscibile

Un linguaggio L è Turing riconoscibile se e solo se esiste una macchina di Turing multinastro M che lo riconosce, cioè tale che $L(M) = L$

• Dimostrazione:

Se L è Turing riconoscibile allora esiste una MdT M tale che $L(M) = L$, poichè M è una MdT a k nastri con $k = 1$, esiste una macchina di Turing multinastro M tale che $L(M) = L$

Viceversa, se esiste una macchina di Turing multinastro M tale che $L(M) = L$, per il teorema precedente esiste una macchina di Turing a nastro singolo S tale che $L(S) = L(M) = L$, quindi L è Turing riconoscibile

8.5 Ordine Radix

Sia $\Sigma = \{a_0, \dots, a_k\}$ un alfabeto e sia $a_0 < a_1 < \dots < a_k$ un ordinamento degli elementi di Σ

Siano $x, y \in \Sigma^*$, diremo che $x \leq y$ rispetto all'ordine radix se x e y verificano le condizioni seguenti

- $|x| < |y|$
- $|x| = |y|$ e $x = zax', y = zby', z, x', y' \in \Sigma^*, a, b \in \Sigma, a \leq b$

9 Macchina di Turing Non deterministica

9.1 Descrizione Formale

Una macchina di Turing non deterministica è una settupla

$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ con

- $Q, \Sigma, \Gamma, q_0, q_{\text{accept}}, q_{\text{reject}}$ sono definiti come in una MdT deterministica
- La funzione di transizione δ è definita come segue

$$\delta : (Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

Quindi per ogni $q \in Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}$, per ogni $\gamma \in \Gamma$ risulta

$$\delta(q, \gamma) = \{(q_1, \gamma_1, d_1), \dots, (q_k, \gamma_k, d_k)\}, \text{ con } k \geq 0 \text{ e}$$

$$(q_j, \gamma_j, d_j) \in Q \times \Gamma \times \{L, R\}, \text{ per } j \in \{1, \dots, k\}$$

- La computazione continua ad essere una successione finita di configurazioni $C \xrightarrow{*} C'$, quindi **non è un albero**
- Ma dato che ci possono essere più configurazioni $u'q'v'$ che sono prodotte da uqv in un passo, ed è possibile organizzarle in un albero

9.2 Albero delle computazioni

In una macchina di Turing non deterministica, le computazioni su una stringa input w possono essere organizzate in un albero radicato in cui la radice è la configurazione iniziale $q_0 w$

I nodi sono configurazioni e i figli di ogni nodo rappresentano le possibili configurazioni raggiungibili da quel nodo in un passo di computazione

In particolare, ogni configurazione in cui lo stato è q_{reject} o q_{accept} è una foglia

9.3 Linguaggio Riconosciuto da una MdT non Deterministica

Sia $N = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ una MdT non deterministica, il linguaggio $L(N)$ riconosciuto da N , è l'insieme

$$L(N) = \left\{ w \in \Sigma^* \mid \text{esiste una computazione } q_0 w \xrightarrow{*} u q_{\text{accept}} v, u, v \in \Gamma^* \right\}$$

10 Equivalenza Modello Deterministico e non Deterministico

Per ogni macchina di Turing non deterministica N esiste una macchina di Turing deterministica D equivalente ad N , cioè tale che $L(N) = L(D)$

10.1 Idea della Prova

- Ogni computazione di N deriva da una sequenza di scelte che D deve riprodurre
- Per ogni input w , D esplora l'albero delle computazioni di N su w
- Se D trova lo stato di accettazione su uno qualsiasi dei rami dell'albero, accetta
 - **Visita per livelli**, se esiste una computazione $q_0 w \xrightarrow{*} u q_{\text{accept}} v$, D prima o poi effettuerà la sequenza di scelte corrispondenti
- Questo assicura che $L(D) = L(N)$

La macchina MdT deterministica D che simula N ha tre nastri

- **Nastro 1:** Contiene la stringa di input e non viene modificato
- **Nastro 2:** Mantiene una copia del nastro di N corrispondente a qualche diramazione della sua computazione non deterministica
- **Nastro 3:** Deve tenere traccia della posizione di D nell'albero delle computazioni di N

10.2 Rappresentazione delle Computazioni

Sia $N = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ una macchina di Turing non deterministica

Sia b il massimo numero di scelte per N su ogni stato e ogni carattere

$$b = \max\{|\delta(q, \sigma)| \mid q \in Q, \sigma \in \Sigma\}$$

Consideriamo l'alfabeto $\Gamma_b = \{1, \dots, b\}$

Per ogni coppia stato-simbolo (q, σ) esistono al più b scelte a ognuna delle quali associamo un diverso simbolo in Γ_b

Quindi per ogni configurazione $uq\sigma v$ esistono al più b successive configurazioni a ognuna delle quali risulta associato un simbolo diverso di Γ_b

Ogni albero di computazione di N è un albero b -ario, cioè ogni nodo ha al più b figli

Ogni cammino in ogni albero di computazione può essere rappresentato da una stringa sull'alfabeto $\Gamma_b = \{1, \dots, b\}$

- A ogni computazione è associata una stringa su Γ_b
- Viceversa, a ogni stringa su Γ_b è associata al più una computazione
- Non è sempre vero che una stringa rappresenta una computazione

10.3 Rappresentazione dei Cammini

Se una stringa rappresenta una computazione, ogni simbolo nella stringa rappresenta una scelta tra le possibili alternative proposte dalla δ in un passo di computazione

La stringa vuota corrisponde alla configurazione iniziale, è l'indirizzo della radice dell'albero

Una visita per livelli dell'albero corrisponde alla lista delle stringhe su Γ_b in ordine radix

10.4 Descrizione dei 3 Nastri

- Sul **primo nastro** è memorizzata la stringa input w e il contenuto del primo nastro non verrà alterato dalle computazioni di D
- Sul **secondo nastro** viene eseguita la simulazione di N , il nastro 2 mantiene una copia del nastro di N corrispondente a qualche diramazione dell'albero delle computazioni
- Sul **terzo nastro** vengono generate le codifiche delle possibili computazioni di N con input w , tenendo traccia delle posizioni

10.5 Descrizione di D

1. Inizialmente il nastro 1 contiene l'input w e i nastri 2 e 3 contengono solo $\sqcup$
2. D copia il contenuto del nastro 1 sul nastro 2
3. Utilizza il nastro 2 per simulare N con input w sulla ramificazione della sua computazione non deterministica corrispondente alla stringa sul nastro 3
 - Prima di ogni passo di N , consulta il simbolo sul nastro 3 per determinare quale scelta fare tra quelle consentite dalla funzione di transizione di N
 - Se si raggiunge una configurazione di accettazione D accetta l'input, altrimenti passa al passo 4
4. D genera sul nastro 3 la stringa successiva a quella corrente in ordine radix e torna al passo 2

D accetta se e solo se N accetta w , quindi $L(D) = L(N)$

10.6 Corollario Macchina di Turing non Deterministica

Corollario:

Un linguaggio L è Turing riconoscibile se e solo se esiste una macchina di Turing non deterministica N che lo riconosce, cioè tale che $L(N) = L$

Dimostrazione:

Se L è il linguaggio $L(N)$ riconosciuto da una macchina di Turing non deterministica N , per il teorema precedente esiste una MdT deterministica D tale che $L(D) = L(N) = L$
Quindi L è Turing riconoscibile

Viceversa, se L è Turing riconoscibile allora esiste una macchina di Turing deterministica $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ che lo riconosce, cioè tale che $L(M) = L$

La macchina di Turing non deterministica

$M' = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, dove $\delta'(q, \gamma) = \{\delta(q, \gamma)\}$, per ogni $q \in Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}$ e $\gamma \in \Gamma$, è tale che $L(M') = L(M) = L$

10.7 Decisori non Deterministici

Una macchina di Turing non deterministica è un decisore se, per ogni stringa input w , tutte le computazioni a partire da q_0w terminano

Una macchina di Turing non deterministica N decide L se N è un decisore e $L = L(N)$

Sia N un decisore non deterministico e sia w una stringa

- N **accetta** w se e solo se esiste almeno una computazione $q_0w \xrightarrow{*} uq_{\text{accept}}v$, dove q_0w è la configurazione iniziale di N con input w e $uq_{\text{accept}}v$ è una configurazione di accettazione
 - L'albero della computazione di N su w è finito e contiene almeno una foglia con stato q_{accept}
- N **non accetta** w se e solo se nessuna computazione della configurazione iniziale q_0w con input w termina in una configurazione di accettazione
 - L'albero delle computazione di N su w è finito e non contiene nessuna foglia con stato q_{accept}

Se N è una macchina di Turing non deterministica tale che, per ogni input w , N si ferma sempre su tutte le ramificazioni dell'albero delle computazioni su w , allora esiste una MdT deterministica D che simula N e che si arresta su ogni input

Diremo che una MdT non deterministica è un decisore se tutte le sue ramificazioni si fermano su ogni input

Corollario:

Un linguaggio L è decidibile se e solo se esiste una macchina di Turing non deterministica N che lo decide

- Quindi, per ogni input w , l'albero delle computazioni di N su w ha un numero finito di nodi

Teorema:

Se ogni nodo in un albero ha un numero finito di figli e ogni cammino dell'albero ha un numero finito di nodi, allora l'albero ha un numero finito di nodi

Corollario:

Un linguaggio L è decidibile se e solo se esiste una macchina di Turing non deterministica N che lo decide

Prova:

Se un linguaggio L è decidibile, esiste una macchina di Turing deterministica M che è un decisore e che accetta L

M può essere facilmente trasformata in una macchina di Turing non deterministica che decide L

Viceversa, se un linguaggio L è deciso da una macchina di Turing non deterministica N , definiamo una macchina di Turing deterministica D' , modificando la precedente definizione di D come segue

Descrizione di D'

1. Inizialmente il nastro 1 contiene l'input w e i nastri 2 e 3 contengono solo $\sqcup$
2. D' copia il contenuto del nastro 1 sul nastro 2
3. Utilizza il nastro 2 per simulare N con input w sulla ramificazione della sua computazione non deterministica corrispondente alla stringa sul nastro 3
 - Prima di ogni passo di N , consulta il simbolo sul nastro 3 per determinare quale scelta fare tra quelle consentite dalla funzione di transizione di N
 - Se si raggiunge una configurazione di accettazione D' accetta l'input, altrimenti passa al passo 4
4. Rifiuta se tutti i cammini dell'albero delle computazioni di N su w non hanno portato a una configurazione di accettazione, altrimenti D' passa al passo 5
5. D' genera sul nastro 3 la stringa successiva a quella corrente in ordine radix e torna al passo 2

Quindi possiamo dedurre che D' è un decisore per L

Se N accetta il suo input w , allora D' troverà un cammino che termina in una configurazione di accettazione e D' accetta w

Se N non accetta il suo input w , nessuna delle sue computazioni su w termina in una configurazione di accettazione, siccome N è un decisore, ciascuno dei cammini ha un numero finito di nodi poichè ogni arco nel cammino rappresenta un passo di computazione di N su w , quindi, D' si fermerà e rifiuterà quando l'intero albero sarà stato esplorato

D' deve avere un controllo sulle stringhe che rappresentano codifiche di computazioni su w che terminano in una configurazione di rifiuto oppure in una configurazione che non produce nessuna altra configurazione. Se x è una stringa che codifica una tale computazione, D' deve bloccare la generazione di stringhe in ordine radix con prefisso x . Analogamente, se la stringa x è una stringa non valida, cioè non corrisponde a una computazione, D' deve bloccare la generazione di stringhe in ordine radix con prefisso x

11 Problemi di Decisione

Un problema di decisione è un problema che ha come soluzione una risposta sì o no

- **Decidibile:** Se il linguaggio associato è decidibile
- **Semidecidibile:** Se il linguaggio associato è Turing riconoscibile
- **Indecidibile:** Se il linguaggio associato non è decidibile

11.1 Richiami di Logica

Variabili Booleane: Variabili che possono assumere valore vero o falso

Operazioni Booleane: $\vee$ (OR), $\wedge$ (AND), $\neg$ (NOT)

Denotiamo $\neg x$ con $\bar{x}$

Dato un insieme di variabili booleane X , le formule booleane su X sono definite induttivamente come segue:

- Le costanti 0, 1 e le variabili $x, \bar{x}$, con $x \in X$, sono formule booleane
- Se Φ, Φ_1, Φ_2 sono formule booleane allora $(\Phi_1 \vee \Phi_2), (\Phi_1 \wedge \Phi_2), \bar{\Phi}$ sono formule booleane

Una formula booleana Φ è soddisfacibile se esiste un insieme di valori 0 e 1 per le variabili di Φ che renda la formula uguale a 1

Cammino Hamiltoniano: In un grafo orientato è un cammino orientato che passa per ogni vertice del grafo una e una sola volta

11.2 Istanze Problema di Decisione

Istanze: Gli elementi di un insieme considerati dal problema di decisione, ovvero un particolare input per quel problema

L'insieme delle istanze: Rappresenta l'unione del sottoinsieme delle istanze con risposta sì e del sottoinsieme con risposta no

11.3 Problemi di Ricerca

Chiamiamo problemi di ricerca quelli per i quali cerchiamo una soluzione se esiste

Dato un problema di ricerca possiamo in genere usare come sottoprogramma un algoritmo per il corrispondente problema di decisione, se tale algoritmo esiste

11.4 Livelli di Descrizione di una Macchina di Turing

- **Descrizione Formale:** Livello più basso e dettagliato, specificando gli elementi della settupla che definisce la MdT
- **Descrizione Implementativa:** Descriviamo verbalmente il modo in cui memorizza i dati sul nastro
- **Descrizione di Alto Livello:** Usiamo frasi del linguaggio per descrivere un algoritmo, ignorando come la macchina gestisce il nastro o la testina

11.5 Codifiche

La corrispondenza che a una istanza associa una stringa deve essere un codifica, cioè deve rappresentare univocamente l'istanza

- Useremo $\langle \mathcal{O} \rangle$ per denotare una stringa che codifica l'oggetto $\mathcal{O}$
- Useremo $\langle \mathcal{O}_1, \dots, \mathcal{O}_k \rangle$ per denotare una stringa che codifica gli oggetti $\mathcal{O}_1, \dots, \mathcal{O}_k$
- Una MdT è codificabile con una stringa
- Una MdT e una stringa w sono codificabili con una stringa

11.6 Codifica di una MdT

In generale per codificare una macchina di Turing

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ occorre stabilire come codificare

- I simboli dell'alfabeto in input
- I simboli dell'alfabeto di nastro
- Gli stati
- I possibili movimenti della testina
- I valori della funzione di transizione
- Lo stato iniziale q_0 e gli stati $q_{\text{accept}}, q_{\text{reject}}$

Un possibile codifica di una macchina di Turing $M =$

$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ mediante una stringa su $\{0, 1\}$

- Fissiamo un alfabeto infinito universale $\Sigma_U = \{a_1, a_2, \dots\}$ e assumiamo che tutti i simboli di input e di nastro siano estratti da Σ_U
- Fissiamo un insieme infinito universale di stati $Q_U = \{q_0, q_1, \dots\}$ e assumiamo che tutte le MdT usano nomi di stati scelti da Q_U
- Codifichiamo un elemento $a_i \in \Sigma_U$ con la stringa $e(a_i) = 0^{i+1}$
- Codifichiamo il simbolo $\sqcup$ con la stringa $e(\sqcup) = 0$
- Codifichiamo un alfabeto $\Delta = \{b_1, b_2, \dots, b_r\}$ mediante la stringa

$$e(\Delta) = 111e(b_1)1e(b_2)1 \cdots 1e(b_r)111$$

Quindi $e(\Sigma)$ e $e(\Gamma)$ sono definiti

- Codifichiamo un elemento $q_i \in Q_U$ con la stringa $e(q_i) = 0^{i+1}$
- Codifichiamo i movimenti della testina con

$$e(L) = 0, e(R) = 00, e(S) = 000$$

- Codifichiamo una transizione m di una MdT, ad esempio $\delta(q, a) = (p, b, D)$ con

$$e(m) = 11e(q)1e(a)1e(p)1e(b)1e(D)11$$

- Infine codifichiamo una l'intera MdT $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ con transizioni $m_1, \dots, m_t$ mediante la stringa

$$11111e(q_0)1e(q_{\text{accept}})1e(q_{\text{reject}})1e(\Sigma)1e(\Gamma)1e(m_1)1 \cdots 1e(m_t)11111$$

- Codifichiamo una stringa $w = b_1 b_2 \cdots b_r$ mediante

$$e(w) = 11e(b_1)1e(b_2)1 \cdots 1e(b_r)11$$

- Codifichiamo una MdT M e una stringa w con la stringa

$$\langle M, w \rangle = e(M)e(w)$$

11.7 Linguaggio Associato a un Problema di Decisione

Il linguaggio L associato a un problema di decisione $\mathbb{P}$ è il linguaggio delle codifiche delle istanze che hanno risposta sì

Se esiste una macchina di Turing che decide L il problema viene detto **decidibile**, altrimenti viene detto **indecidibile**

Se esiste una macchina di Turing che riconosce L il problema viene detto **semidecidibile**

11.7.1 INDEPENDENT-SET

Sia $G = (V, E)$ un grafo non orientato, con V insieme di nodi ed E insieme di archi

Un sottoinsieme V' di nodi di G è un independent-set in G se per ogni $u, v \in V'$, la coppia (u, v) non è un arco, cioè u e v non sono adiacenti

$\text{INDEPENDENT-SET} = \{\langle G, k \rangle \mid G \text{ è un grafo non orientato,}$

$k \text{ è un intero positivo e } G \text{ ha un independent set di cardinalità } k\}$

11.8 Problemi di Decisione nella Teoria degli Automi

11.8.1 Problema Accettazione di un DFA

Sia $\mathcal{B}$ un DFA e w una parola, il corrispondente linguaggio è

$$A_{\text{DFA}} = \{\langle \mathcal{B}, w \rangle \mid \mathcal{B} \text{ è un DFA che accetta la parola } w\}$$

Teorema:

A_{DFA} è un linguaggio decidibile

11.8.2 Problema del Vuoto

$$E_{\text{DFA}} = \{\langle \mathcal{A} \rangle \mid \mathcal{A} \text{ è un DFA e } L(\mathcal{A}) = \emptyset\}$$

Teorema:

E_{DFA} è un linguaggio decidibile

11.9 Problema dell'Equivalenza di due DFA

$$EQ_{\text{DFA}} = \{\langle \mathcal{A}, \mathcal{B} \rangle \mid \mathcal{A}, \mathcal{B} \text{ sono DFA e } L(\mathcal{A}) = L(\mathcal{B})\}$$

Teorema:

EQ_{DFA} è un linguaggio decidibile

11.9.1 Alcuni Esempi con NFA

Potremmo formulare i tre precedenti attraverso rappresentazioni equivalenti come NFA o espressioni regolari

$$A_{\text{NFA}} = \{\langle \mathcal{B}, w \rangle \mid \mathcal{B} \text{ è un NFA che accetta la parola } w\}$$

$$A_{\text{REX}} = \{\langle \mathcal{R}, w \rangle \mid \mathcal{R} \text{ è un'espressione regolare e } w \in L(\mathcal{R})\}$$

12 Metodo della Diagonalizzazione

12.1 Funzione

Dati due insiemi non vuoti X e Y , una funzione $f : X \rightarrow Y$ da X in Y è una relazione che associa a ogni elemento x in X uno e uno solo $y = f(x)$ in Y

- X è il dominio della funzione
- Y è il codominio della funzione

12.2 Funzione Iniettiva

Una funzione $f : X \rightarrow Y$ è iniettiva se

$$\forall x, x' \in X, x \neq x' \Rightarrow f(x) \neq f(x')$$

12.3 Funzione Suriettiva

Una funzione $f : X \rightarrow Y$ è suriettiva $\Leftrightarrow \forall y \in Y, \exists x \in X : y = f(x)$

12.4 Funzione Biettiva

Una funzione $f : X \rightarrow Y$ è una funzione biettiva di X su Y se f è iniettiva e suriettiva

12.5 Cardinalità

Due insiemi X e Y hanno la stessa cardinalità se esiste una funzione biettiva $f : X \rightarrow Y$ di X su Y

$$|X| = |Y| \Leftrightarrow \text{esiste una funzione biettiva } f : X \rightarrow Y$$

- $|X| \leq |Y| \Leftrightarrow \text{esiste una funzione iniettiva } f : X \rightarrow Y$
- $|X| \leq |Y|, |X| \neq |Y| \Rightarrow |X| < |Y|$
- $|X| \leq |Y|, |X| \geq |Y| \Rightarrow |X| = |Y|$

12.5.1 Esempio $\mathbb{N}$

Sia $\mathbb{N}$ l'insieme dei numeri interi positivi e sia $\mathbb{N}_p = \{2n | n \in \mathbb{N}\}$ l'insieme dei numeri interi positivi pari

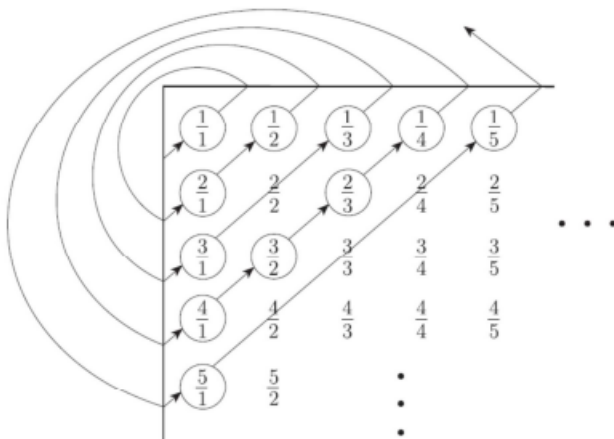
$$f : \mathbb{N} \rightarrow \{2n \mid n \in \mathbb{N}\}$$

Dove $f(n) = 2n$, per ogni $n \in \mathbb{N}$, è biettiva

n	$f(n)$
1	2
2	4
3	6
$\vdots$	$\vdots$

12.5.2 Esempio $\mathbb{Q}^+$

Consideriamo l'insieme $\mathbb{Q}^+ = \left\{ \frac{x}{y} \mid x, y \in \mathbb{N}, x > 0, y > 0 \right\}$ dei numeri razionali positivi, quindi $\mathbb{N}$ e $\mathbb{Q}^+$ hanno la stessa cardinalità e lo possiamo vedere costruendo la seguente matrice, con la riga i -esima che contiene tutti i numeratori e la riga j -esima che contiene tutti i denominatori



12.5.3 Esempio 3

Sia $\mathbb{N}^2 = \{(x, y) | x, y \in \mathbb{N}, x, y > 0\}$, la funzione $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ definita come segue è biettiva

$$\begin{aligned}\forall (x, y) \in \mathbb{N}^2 \quad f(x, y) &= \frac{(x+y)(x+y+1)}{2} + x = \\ &= \frac{1}{2}((x+y)^2 + 3x + y) \in \mathbb{N}\end{aligned}$$

La funzione f è nota come la funzione coppia di Cantor

	1	2	3	4	
1	1	2	4	7	...
2	3	5	8	...	...
3	6	9	...	...	...
·	...	...	...	...	...
i	...	...	...	...	...
·	...	...	...	...	...
·	...	...	...	...	...

12.6 Numerabilità

Un insieme X è numerabile se esiste una funzione biettiva $f : \mathbb{N} \rightarrow X$ di $\mathbb{N}$ su X

Un insieme X è enumerabile se esiste una funzione biettiva calcolabile $f : \mathbb{N} \rightarrow X$ di $\mathbb{N}$ su X

Un insieme X è contabile se è finito o numerabile

12.6.1 $\mathbb{R}$ non è Numerabile

Dimostrazione per Contraddizione:

Supponiamo quindi che esista una funzione biettiva f di $\mathbb{N}$ su $\mathbb{R}$

Costruiamo una tabella che contiene alcuni valori ipotetici della biiezione e scegliamo x in modo che sia diverso dagli elementi della tabella procedendo in diagonale

n	$f(n)$	
1	3. <u>1</u> 4159...	$x = 0.4641 \dots$
2	55.5 <u>5</u> 555...	
3	0.123 <u>4</u> 5...	
4	0.500 <u>0</u> 0...	
$\vdots$	$\vdots$	

Idea della Prova:

Supponiamo per assurdo che esista una funzione biettiva f di $\mathbb{N}$ su $\mathbb{R}$, allora costruiamo la tabella seguente

n	$f(n)$				
1	$d_{1,1}$	$d_{1,2}$	...	...	...
2	$d_{2,1}$	$d_{2,2}$	...	...	...
.	...	...	...	...	...
.	...	...	...	...	...
i	$d_{i,1}$	$d_{i,2}$	...	...	...
.	...	...	...	...	...
.	...	...	...	...	...

Se consideriamo $r = 0, d_{1,1}d_{2,2}...$ e il numero reale $x = 0, d'_{1,1}d'_{2,2}...$ che abbiamo ottenuto prima in modo che sia diverso dagli elementi della tabella procedendo in diagonale, non è immagine di nessun intero positivo

In generale $x \neq f(i)$ per un qualsiasi i , perchè $d'_{i,i} \neq d_{i,i}$

12.7 Metodo della Diagonalizzazione Linguaggi non Turing riconoscibili

Corollario:

Esistono linguaggi che non sono Turing riconoscibili

Per la prova si usa il metodo della diagonalizzazione e consiste nel provare le seguenti condizioni

- Dato un alfabeto Σ , l'insieme Σ^* è numerabile
- L'insieme delle codifiche delle macchine di Turing è numerabile
- L'insieme dei linguaggi Turing riconoscibili è numerabile
- L'insieme dei linguaggi sull'alfabeto Σ ha cardinalità maggiore del numerabile

12.7.1 Numerabilità dell'Insieme delle Parole

Teorema:

Se ogni insieme S_n è numerabile, anche $S = \bigcup_{n \in \mathbb{N}} S_n$ è numerabile

Corollario:

Σ^* è numerabile

Dimostrazione:

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n$$

Sia $\Sigma = \{a_1, a_2, \dots, a_k\}$ un alfabeto

Possiamo definire una corrispondenza biunivoca tra $\mathbb{N}$ e Σ^* che permette di enumerare le stringhe in ordine Radix

$$w_1 = \varepsilon, w_2 = a_1, \dots$$

In generale per la stringa w_n abbiamo

$$n = |\{y \in \Sigma^* \mid y < w_n \text{ (Rispetto all'ordine Radix)}\}| + 1$$

Dato che è possibile definire un algoritmo per calcolare tale indice abbiamo una funzione biettiva e calcolabile di $\mathbb{N}$ in Σ^*

12.7.1.1 Dimostrazione alternativa

Idea della Dimostrazione:

Provare che $|\mathbb{N}| \leq |\Sigma^*|$ e poi che $|\Sigma^*| \leq |\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$

Per provare che $|\mathbb{N}| \leq |\Sigma^*|$ bisogna definire una funzione iniettiva da $\mathbb{N}$ in Σ^* .

Vediamo prima su un esempio come potremmo definire una tale funzione.

Sia $\Sigma = \{a, b\}$.

Consideriamo l'applicazione f tale che $f(n) = a^n$.

Quindi ad esempio $f(1) = a$, $f(5) = a^5 = aaaaa$.

L'applicazione f è iniettiva.

Per ogni $n \in \mathbb{N}$ sia $f_n : \Sigma^n \rightarrow \mathbb{N}$ un'applicazione iniettiva.

Inoltre poniamo $f_0(\epsilon) = 1$.

Esempio. Sia $\Sigma = \{a, b\}$.

$f_1(a) = 1$, $f_1(b) = 2$.

$f_2(aa) = 1$, $f_2(ab) = 2$, $f_2(ba) = 3$, $f_2(bb) = 4$.

Definiamo una funzione g tale che $g(w) = (|w| + 1, f_{|w|}(w))$.

Ad esempio, $g(b) = (2, 2)$, $g(ab) = (3, 2)$ e $g(bb) = (3, 4)$.

L'applicazione $g : w \in \Sigma^* \rightarrow (|w| + 1, f_{|w|}(w)) \in \mathbb{N} \times \mathbb{N}$ è iniettiva.

Dimostrazione:

Sia $\sigma \in \Sigma$, l'applicazione $f : n \in \mathbb{N} \rightarrow \sigma^n \in \Sigma^*$ è iniettiva, quindi

$$|\mathbb{N}| \leq |\Sigma^*|$$

Per ogni $n \in \mathbb{N}$ sia $f_n : \Sigma^n \rightarrow \mathbb{N}$ un'applicazione iniettiva. Inoltre poniamo $f_0(\epsilon) = 1$.

Definiamo una funzione g tale che $g(w) = (|w| + 1, f_{|w|}(w))$.

L'applicazione $g : w \in \Sigma^* \rightarrow (|w| + 1, f_{|w|}(w)) \in \mathbb{N} \times \mathbb{N}$ è iniettiva.

Siano $x, y \in \Sigma^*$ tali che $x \neq y$.

Se $|x| \neq |y|$ allora

$g(x) = (|x| + 1, f_{|x|}(x)) \neq (|y| + 1, f_{|y|}(y)) = g(y)$ perché la prima coordinata è diversa.

Se $|x| = |y|$ allora $f_{|x|}(x) \neq f_{|y|}(y)$ e

$g(x) = (|x| + 1, f_{|x|}(x)) \neq (|y| + 1, f_{|y|}(y)) = g(y)$ perché la seconda coordinata è diversa.

Ne consegue

$$|\Sigma^*| \leq |\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$$

Quindi

$$|\Sigma^*| = |\mathbb{N}|$$

12.7.2 Numerabilità Insieme Codifiche MdT

L'insieme $\{\langle M \rangle \mid M \text{ è una macchina di Turing}\}$ è numerabile

Dimostrazione:

È possibile codificare una MdT M con una stringa su un alfabeto Σ e l'applicazione $f : \langle M \rangle \rightarrow \langle M \rangle \in \Sigma^*$ è iniettiva

Quindi

$$|\{\langle M \rangle \mid M \text{ è una macchina di Turing}\}| \leq |\Sigma^*| = |\mathbb{N}|$$

Sia w una stringa su un alfabeto Δ , il linguaggio $\{w\}$ è decidibile, sia M_w una fissata MdT che decide $\{w\}$, l'applicazione g tale che $g(w) = \langle M_w \rangle$ è iniettiva

Quindi

$$|\mathbb{N}| = |\Delta^*| \leq |\{\langle M \rangle \mid M \text{ è una macchina di Turing}\}|$$

Da cui

$$|\{\langle M \rangle \mid M \text{ è una macchina di Turing}\}| = |\mathbb{N}|$$

12.7.3 Numerabilità dei Linguaggi Turing Riconoscibili

L'insieme $\{L \mid L \text{ è un linguaggio Turing riconoscibile}\}$ è numerabile

Dimostrazione:

Possiamo associare a ogni linguaggio Turing riconoscibile una fissata MdT che lo riconosce e questa corrispondenza è iniettiva

Quindi

$$|\{L \subseteq \Sigma^* \mid L \text{ è un linguaggio Turing riconoscibile}\}| \leq |\{\langle M \rangle \mid M \text{ è una macchina di Turing}\}| = |\mathbb{N}|$$

Sia w una stringa su un alfabeto Δ , il linguaggio $\{w\}$ è Turing riconoscibile

L'applicazione h tale che $h(w) = \{w\}$ è iniettiva

Quindi

$$|\mathbb{N}| = |\Delta^*| \leq |\{L \subseteq \Sigma^* \mid L \text{ è un linguaggio Turing riconoscibile}\}|$$

Da cui

$$|\{L \subseteq \Sigma^* \mid L \text{ è un linguaggio Turing riconoscibile}\}| = |\mathbb{N}|$$

12.7.4 Non Numerabilità dell'Insieme dei Linguaggi

Sia $\Sigma^* = \{w_1, w_2, \dots\}$

Sia $\mathcal{B}$ l'insieme delle sequenze binarie infinite cioè delle sequenze infinite di 0 e 1

È possibile associare a ogni linguaggio L una sequenza binaria infinita $\mathcal{X}_L$, la sequenza caratteristica definita in questo modo:

- Il bit i -esimo di $\mathcal{X}_L$ è 1 se l' i -esima stringa w_i è in L
- Il bit i -esimo di $\mathcal{X}_L$ è 0 se l' i -esima stringa $w_i \notin L$

$$\begin{array}{lcl} \Sigma^* = \{ & \epsilon, & 0, \quad 1, \quad 00, \quad 01, \quad 10, \quad 11, \quad 000, 001, \dots \} ; \\ A = \{ & & 0, \quad \quad 00, \quad 01, \quad \quad \quad 000, 001, \dots \} ; \\ \chi_A = & 0 & 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad \dots \end{array}$$

La sequenza binaria del linguaggio delle stringhe binarie che iniziano con 0

Questa relazione è una corrispondenza biunivoca tra l'insieme $\mathcal{B}$ delle sequenze binarie infinite e l'insieme $\mathcal{P}(\Sigma^*)$ dei linguaggi su Σ

- A ogni linguaggio L su Σ è associata una sola sequenza binaria infinita $\mathcal{X}$ tale che $\mathcal{X} = \mathcal{X}_L$
- A ogni sequenza binaria infinita $\mathcal{X}$ è associato un solo linguaggio L su Σ tale che $\mathcal{X} = \mathcal{X}_L$

Quindi l'applicazione $f : \mathcal{P}(\Sigma^*) \rightarrow \mathcal{B}$ definita da $f(L) = \mathcal{X}_L$ è biettiva

12.7.5 Non Numerabilità dell'Insieme delle Sequenze Binarie

L'insieme $\mathcal{B}$ non è numerabile

Idea della Dimostrazione:

Mostriamo che non esiste nessuna funzione biettiva di $\mathbb{N}$ sull'insieme $\mathcal{B}$ delle sequenze binarie infinite, supponiamo per assurdo che esista una funzione biettiva f da $\mathbb{N}$ su $\mathcal{B}$

n	$f(n)$				
1	$d_{1,1}$	$d_{1,2}$	...	...	...
2	$d_{2,1}$	$d_{2,2}$	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
i	$d_{i,1}$	$d_{i,2}$	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...

Le cifre sulla diagonale di questa matrice $d_{1,1}, d_{2,2}, \dots$ definiscono una sequenza binaria infinita $\mathcal{X}$

La sequenza binaria infinita $\overline{\mathcal{X}} = \overline{d}_{1,1}, \overline{d}_{2,2}, \dots$ che si ottiene scegliendo in ogni posizione il complemento della corrispondente cifra in $\mathcal{X}$ non è immagine di nessun intero positivo

Quindi, per ogni i , $\overline{\mathcal{X}} \neq f(i)$ perchè $\overline{d}_{i,i} \neq d_{i,i}$

12.7.6 Non Numerabilità Insieme delle Parti dei Linguaggi

L'insieme $\mathcal{P}(\Sigma^*)$ dei linguaggi su Σ non è numerabile

Dimostrazione:

Esiste un applicazione biettiva di f in $\mathcal{P}(\Sigma^*)$ in $\mathcal{B}$,
quindi $|\mathcal{B}| = |\mathcal{P}(\Sigma^*)|$

12.7.6.1 Dimostrazione alternativa

L'insieme $\mathcal{P}(\Sigma^*)$ dei linguaggi su Σ non è numerabile.

(Seconda) Dimostrazione

Supponiamo per assurdo che $\mathcal{P}(\Sigma^*)$ sia numerabile.

Quindi esiste un'applicazione biettiva h di $\mathbb{N}$ in $\mathcal{P}(\Sigma^*)$.

Sia $L_1, L_2, \dots$ la lista dei linguaggi, cioè degli elementi di $\mathcal{P}(\Sigma^*)$, dove $L_1 = h(1)$, $L_2 = h(2)$ e in generale $L_i = h(i)$.

Σ^* è enumerabile, sia g una biezione di $\mathbb{N}$ in Σ^* e siano $w_1, w_2, \dots$ gli elementi di Σ^* , con $w_i = g(i)$.

Definiamo una matrice (infinita) avente come indici di riga i linguaggi $L_1, L_2, \dots$ e indici di colonna le stringhe $w_1, w_2, \dots$.

	w_1	w_2	$\dots$	w_j	
L_1	0	1	$\dots$	0	$\dots$
L_2	1	1	$\dots$	1	$\dots$
L_3	0	0	$\dots$	1	$\dots$
$\vdots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
L_i	1	0	$\dots$	0	$\dots$
$\vdots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\vdots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$

Nella matrice scriviamo 1 all'incrocio tra la riga L_i e la colonna w_j se $w_j \in L_i$, altrimenti scriviamo 0. Quindi la riga corrispondente ad L_i è la sequenza caratteristica di L_i .

Definiamo un linguaggio L come segue:

$$\forall i > 0 \quad w_i \in L \Leftrightarrow w_i \notin L_i$$

$L \in \mathcal{P}(\Sigma^*)$ ma per ogni $i > 0$, $L \neq L_i$.

Infatti, sia h tale che $L = L_h$. La domanda " $w_h \in L?$ " conduce a una contraddizione.

Assumiamo $w_h \in L$. Per la definizione di L deve essere $w_h \notin L_h$. Ma per ipotesi $L = L_h$, assurdo.

Assumiamo $w_h \notin L$. Per la definizione di L deve essere $w_h \in L_h$. Ma per ipotesi $L = L_h$, assurdo.

12.8 Esistono dei Linguaggi non Turing Riconoscibili

Corollario:

L'insieme $\{L \subseteq \Sigma^* \mid L \text{ è Turing riconoscibile}\}$ è numerabile mentre $\mathcal{P}(\Sigma^*)$ non è numerabile, quindi

$$|\{L \subseteq \Sigma^* \mid L \text{ è Turing riconoscibile}\}| = |\mathbb{N}| \neq |\mathcal{P}(\Sigma^*)|$$

Inoltre

$$h : \{L \subseteq \Sigma^* \mid L \text{ è Turing riconoscibile}\} \rightarrow \mathcal{P}(\Sigma^*) \text{ dove}$$

$$h(L) = L \text{ è iniettiva}$$

Ne consegue

$$|\{L \subseteq \Sigma^* \mid L \text{ è Turing riconoscibile}\}| < |\mathcal{P}(\Sigma^*)|$$

Quindi esistono linguaggi che non sono Turing riconoscibili

13 Indecidibilità

13.1 Linguaggio Indecidibile

Teorema:

Il linguaggio A_{TM} è Turing riconoscibile ma non è decidibile

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ è un macchina di Turing e } \\ M \text{ accetta la parola } w \}$$

13.2 Macchina di Turing Universale

Una macchina di Turing universale U , quando riceve in input una codifica $\langle M, w \rangle$ di una macchina di Turing M e di una stringa w , simula la computazione di M sull'input w

13.3 Teorema Esistenza MdT Universale

Esiste una MdT universale

(Schema del comportamento di U)

$$\langle M, w \rangle \rightarrow \boxed{\text{MT Universale } U} \rightarrow \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w \\ \text{rifiuta} & \text{se } M \text{ rifiuta } w \\ \text{non termina} & \text{se } M \text{ non termina} \end{cases}$$

Durante la sua computazione U

- Usa il primo nastro per simulare la computazione di M
- Lascia sul secondo nastro la codifica di M
- Ha sul terzo nastro la codifica dello stato corrente di M

U individua l'istruzione corrente sul secondo nastro, usando il contenuto del terzo nastro e il simbolo corrente codificato sul primo nastro, quindi decodifica l'istruzione e la esegue

13.4 A_{TM} è Turing Riconoscibile

Il linguaggio è Turing riconoscibile

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ è una macchina di Turing e} \\ M \text{ accetta la parola } w\}$$

Dimostrazione:

La seguente MdT U riconosce A_{TM}

U = Sull'input $\langle M, w \rangle$ dove M è una MdT e w è una stringa

- Simula M sull'input w
- Se M accetta w , accetta l'input $\langle M, w \rangle$
- Se M rifiuta w , rifiuta l'input $\langle M, w \rangle$

Quindi U accetta una stringa y se e solo se è della forma $\langle M, w \rangle$ dove M è una MdT, w è una stringa e M accetta w

Quindi, U accetta una stringa y se e solo se $y = \langle M, w \rangle$ è un elemento di A_{TM}

Da cui otteniamo che

$$L(U) = A_{\text{TM}}$$

Nota:

U non termina su $\langle M, w \rangle$ se e solo se M non termina su w , quindi U non decide A_{TM}

13.5 A_{TM} è Indecidibile

Il linguaggio A_{TM} è indecidibile, ma è Turing riconoscibile

Dimostrazione:

Supponiamo per assurdo che A_{TM} sia decidibile, quindi che esista un decisore H che riconosca A_{TM}

- H accetta $\langle M, w \rangle$ se $\langle M, w \rangle \in A_{TM}$
- H rifiuta $\langle M, w \rangle$ se $\langle M, w \rangle \notin A_{TM}$

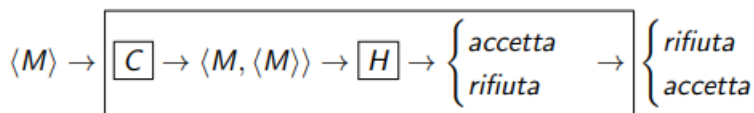
$$H(\langle M, w \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w \\ \text{rifiuta} & \text{se } M \text{ non accetta } w \end{cases}$$

Costruiamo una nuova MdT D che usa H come sottoprogramma, che chiama H su $\langle M \langle M \rangle \rangle$

- H accetta se M accetta $\langle M \rangle$
- H rifiuta se M rifiuta $\langle M \rangle$

Ora costruiamo D in modo che

- Rifiuta se M accetta
- Accetta se M rifiuta



Dato che D può essere facilmente costruita a partire da H , se esiste H allora esiste anche D

Descrizione di D : $D =$ Sull'input $\langle M \rangle$, dove M è una MdT

- Simula H quando H riceve in input $\langle M \langle M \rangle \rangle$
- Fornisce come output l'opposto di H , cioè se H accetta $\langle M \langle M \rangle \rangle$, rifiuta
- Se H rifiuta $\langle M \langle M \rangle \rangle$, accetta

$$D(\langle M \rangle) = \begin{cases} \text{rifiuta} & \text{se } M \text{ accetta } \langle M \rangle \\ \text{accetta} & \text{se } M \text{ non accetta } \langle M \rangle \end{cases}$$

Ma se diamo in input la sua stessa codifica otteniamo

$$D(\langle D \rangle) = \begin{cases} \text{rifiuta} & \text{se } D \text{ accetta } \langle D \rangle \\ \text{accetta} & \text{se } D \text{ non accetta } \langle D \rangle \end{cases}$$

Cioè D accetta $\langle D \rangle$ se e solo se D non accetta $\langle D \rangle$ il che è una contraddizione, quindi H non può esistere

13.6 Linguaggi co-Turing Riconoscibili

Un linguaggio L è co-Turing riconoscibile se il suo complemento è Turing riconoscibile

13.7 La Classe dei Linguaggi Decidibili è Chiusa Rispetto al Complemento

Soluzione:

Sia A un linguaggio decidibile, sia M_A una MdT che decide A

Definiamo la MdT $M_{\bar{A}}$ sull'input w , $M_{\bar{A}}$ simula M_A e accetta w se e solo se M_A rifiuta w

Poiché M_A si arresta su ogni input anche $M_{\bar{A}}$ si arresta su ogni input

Il linguaggio di $M_{\bar{A}}$ è $\bar{A}$, dato che accetta w se e solo se M_A rifiuta w , quindi se e solo se $w \notin A$

Quindi $M_{\bar{A}}$ è una MdT che decide $\bar{A}$ ed $\bar{A}$ è decidibile

Formalmente sia M_A un decisore che decide A

$$M_A = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

Sia

$$M_{\bar{A}} = (Q, \Sigma, \Gamma, \delta', q_0, q_{\text{accept}}, q_{\text{reject}})$$

Dove per ogni $q \in Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}$, per ogni $\gamma \in \Gamma$

$$\delta'(q, \gamma) = \begin{cases} \delta(q, \gamma) & \text{se } \delta(q, \gamma) = (q', \gamma', d) \\ & \text{con } q' \notin \{q_{\text{accept}}, q_{\text{reject}}\} \\ (q_{\text{accept}}, \gamma', d) & \text{se } \delta(q, \gamma) = (q_{\text{reject}}, \gamma', d) \\ (q_{\text{reject}}, \gamma', d) & \text{se } \delta(q, \gamma) = (q_{\text{accept}}, \gamma', d) \end{cases}$$

È Semplice verificare che $M_{\bar{A}}$ è un decisore e che $L(M_{\bar{A}}) = \bar{A}$

13.8 Linguaggio Decidibile Se e Solo se Turing e co-Turing Riconoscibile

Dimostrazione:

Dobbiamo mostrare che L è decidibile $\Leftrightarrow L$ e il suo complemento sono entrambi Turing Riconoscibili

$\Rightarrow$

Se L è decidibile allora esiste un decider tale che M accetta w se e solo se $w \in L$, in particolare M riconosce L e quindi L è Turing riconoscibile
Dato che L decidibile allora anche $\bar{L}$ decidibile, quindi $\bar{L}$ è Turing riconoscibile

$\Leftarrow$

Supponiamo che L e il suo complemento siano entrambi Turing riconoscibili

Sia M_1 una MdT che riconosce L e M_2 una MdT che riconosce $\bar{L}$

Definiamo una MdT M

M = Sull'input w :

- Esegue sia M_1 che M_2 su input w in parallelo
- Se M_1 accetta, accetta
- Se M_2 accetta, rifiuta

Vogliamo provare che M decide L , quindi dobbiamo provare che

- M è un decisore
- M riconosce L , cioè $L = L(M)$

M è un decisore, infatti per ogni stringa w o $w \in L$, oppure $w \in \bar{L}$

Poichè M si ferma ogni volta che M_1 accetta o M_2 accetta, allora M si ferma sempre, quindi è un decisore

Ora dobbiamo provare $L = L(M)$

- $w \in L$ se e solo se M_1 accetta w , quindi M accetta w

$$w \in L \Rightarrow M \text{ accetta } w$$

- $w \notin L$ se e solo se M_2 accetta w , quindi M rifiuta w

$$w \notin L \Rightarrow M \text{ non accetta } w$$

Siccome M accetta w se e solo se $w \in L$ possiamo concludere che $L(M) = L$

Nota: Abbiamo usato il contronominale per dimostrare questa doppia inclusione

13.9 $\overline{A_{\text{TM}}}$ non è Turing Riconoscibile

Dimostrazione:

Supponiamo per assurdo che $\overline{A_{\text{TM}}}$ sia Turing riconoscibile

Sappiamo che A_{TM} è Turing riconoscibile

Quindi A_{TM} sarebbe Turing riconoscibile e co-Turing riconoscibile, quindi per il precedente teorema sarebbe decidibile, ma è un assurdo dato che A_{TM} è indecidibile

13.10 Chiusura Linguaggi Turing Riconoscibili

Rispetto al Complemento

La classe dei linguaggi Turing riconoscibili non è chiusa rispetto al complemento, infatti A_{TM} è Turing riconoscibile, ma $\overline{A_{\text{TM}}}$ non è Turing riconoscibile

Note:

- Questo teorema non può essere usato per provare la proprietà di chiusura dei linguaggi decidibili rispetto al complemento
- Non può essere considerato una definizione di linguaggio decidibile

14 Riducibilità

14.1 Esempio Descrizione Informale

$$\Sigma = \{0, 1\}$$

Denotiamo con $\langle n \rangle$ la rappresentazione binario di $n \in \mathbb{N}$

$$\text{EVEN} = \{w \in \Sigma^* \mid w = \langle n \rangle, n \in \mathbb{N} \text{ pari}\}$$

$$\text{ODD} = \{w \in \Sigma^* \mid w = \langle n \rangle, n \in \mathbb{N} \text{ dispari}\}$$

Costruiamo la MdT INCR

$$w \rightarrow \boxed{\text{INCR}} \rightarrow \begin{cases} w & \text{se } w \notin (1\Sigma^* \cup \{0\}); \\ w' = \langle n+1 \rangle & \text{se } w = \langle n \rangle \end{cases}$$

INCR = Sulla stringa di input w :

- Se $w \notin (1\Sigma^* \cup \{0\})$ si ferma sul primo carattere di w , altrimenti esegue i passi successivi
- Trasforma w in $\$w$
- Nello stato q_r scorre l'input da sinistra a destra fino a incontrare il simbolo $\sqcup$
- Passa nello stato q_ℓ , si sposta a sinistra cambiando ogni carattere 1 che vede in 0 fino a leggere un carattere diverso da 1
 - Se questo carattere è 0 lo cambia in 1, e si sposta a sinistra fino a leggere $\$$, elimina $\$$ e sposta a sinistra di una casella la stringa di caratteri 0 e 1 sul nastro e poi si ferma
 - Se questo carattere è $\$$, l'input era una stringa di caratteri uguali a 1, allora cambia $\$$ in 1 e si ferma su quest'ultimo carattere

Abbiamo definito una funzione calcolabile

$$f : w \in \Sigma^* \rightarrow w' \in \Sigma^*$$

Tale che

$$w = \langle n \rangle \in \text{EVEN} \Leftrightarrow f(w) = w' = \langle n + 1 \rangle \in \text{ODD}$$

Nota: f è definito su tutto Σ^*

$$S : w \rightarrow \boxed{\text{INCR}} \rightarrow w' \rightarrow \boxed{R}$$

- Se esiste una MdT R_c che decide ODD, la MdT S decide EVEN
- Se EVEN è indecidibile proviamo che anche ODD lo è

14.2 Riducibilità Descrizione Informale

Idea:

Convertire le istanze di un problema P nelle istanze di un problema P' in modo che un algoritmo P' , se esiste, possa essere utilizzato per progettare un algoritmo per P

- Sia A il linguaggio associato a P e B il linguaggio associato a P' , allora
 - B decidibile $\Rightarrow A$ decidibile
 - A indecidibile $\Rightarrow B$ indecidibile

14.3 Richiamo di Logica

Definizione

Una proposizione è una frase che dichiara qualcosa e che può essere vera o falsa ma non può essere entrambe.

Siano p, q due proposizioni. Il contronominale di

$$p \rightarrow q$$

è

$$\neg q \rightarrow \neg p$$

Il contronominale di $p \rightarrow q$ è logicamente equivalente a $p \rightarrow q$

$$(p \leftrightarrow q) \equiv (p \rightarrow q) \wedge (q \rightarrow p)$$

$$(p \leftrightarrow q) \equiv (p \rightarrow q) \wedge (q \rightarrow p)$$

$$\begin{aligned} p \leftrightarrow q &\equiv (p \rightarrow q) \wedge (q \rightarrow p) \\ &\equiv (\neg q \rightarrow \neg p) \wedge (\neg p \rightarrow \neg q) \\ &\equiv (\neg p \rightarrow \neg q) \wedge (\neg q \rightarrow \neg p) \\ &\equiv \neg p \leftrightarrow \neg q \end{aligned}$$

Abbiamo provato che le proposizioni composte $p \leftrightarrow q$ e $\neg p \leftrightarrow \neg q$ sono logicamente equivalenti.

14.4 Riduzione Mediante Funzione

Definizione:

Un linguaggio $A \subseteq \Sigma^*$ è riducibile mediante una funzione a un linguaggio $B \subseteq \Sigma^*$ $\left(A \leq_m B \right)$ se esiste una funzione calcolabile $f : \Sigma^* \rightarrow \Sigma^*$ tale che

$$\forall w \in \Sigma^* \quad w \in A \Leftrightarrow f(w) \in B$$

Definizione:

Una riduzione da un linguaggio $A \subseteq \Sigma^*$ a un linguaggio $B \subseteq \Sigma^*$ è una funzione $f : \Sigma^* \rightarrow \Sigma^*$ calcolabile e tale che:

$$\forall w \in \Sigma^* \quad w \in A \Leftrightarrow f(w) \in B$$

14.5 Teorema 1 $A \leq_m B$ se e solo se $\overline{A} \leq_m \overline{B}$

$A \leq_m B$ se e solo se $\overline{A} \leq_m \overline{B}$

Dimostrazione:

Per ipotesi $A \leq_m B$ quindi esiste una riduzione f di A a B

Poichè f è una riduzione, f è calcolabile e inoltre

$$\forall w \in \Sigma^* \quad w \in A \Leftrightarrow f(w) \in B$$

Quindi

$$\forall w \in \Sigma^* \quad w \notin A \Leftrightarrow f(w) \notin B$$

Cioè

$$\forall w \in \Sigma^* \quad w \in \overline{A} \Leftrightarrow f(w) \in \overline{B}$$

Quindi per definizione f è una riduzione da $\overline{A}$ a $\overline{B}$

14.6 Teorema 5.22 $A \leq_m B$ e B è Decidibile, allora A è Decidibile

Se $A \leq_m B$ e B è decidibile, allora A è decidibile

Dimostrazione:

Sia M_B una MdT che decide B , sia f una riduzione da A a B e sia M_f una MdT che calcola f

Consideriamo la MdT M_A

M_A = Sull'input w :

- Simula M_f e calcola $f(w)$
- Simula M_B su $f(w)$
 - Se M_B accetta $f(w)$, accetta
 - Se M_B rifiuta $f(w)$, rifiuta

$$M_A : w \rightarrow \boxed{M_f \rightarrow f(w) \rightarrow M_B}$$

- M_A decide A , quindi è un decider
- Per ogni w , M_f si ferma con $f(w)$ sul nastro
- Per ogni w , M_B si ferma su $f(w)$ perchè M_B è un decider

Inoltre M_A riconosce A

$$w \in L(M_A) \Leftrightarrow f(w) \in L(M_B) \text{ (Definizione di } M_A \text{)}$$

$$\Leftrightarrow f(w) \in B \text{ (} M_B \text{ decide } B \text{)}$$

$$\Leftrightarrow w \in A \text{ (Definizione di riduzione)}$$

14.7 Teorema 5.28 $A \leq_m B$ e B è Turing Riconoscibile, allora A è Turing Riconoscibile

Se $A \leq_m B$ e B è Turing riconoscibile, allora A è Turing riconoscibile

Dimostrazione:

Sia M_B una MdT che riconosce B , sia f una riduzione di A a B e sia M_f una MdT che calcola f

Consideriamo la MdT M_A

$M_A =$ Sull'input w

- Simula M_f e calcola $f(w)$
- Simula M_B su $f(w)$
 - Se M_B accetta $f(w)$, accetta
 - Se M_B rifiuta $f(w)$, rifiuta

M_A riconosce A

$$w \in L(M_A) \Leftrightarrow f(w) \in L(M_B) \text{ (Definizione di } M_A \text{)}$$

$$\Leftrightarrow f(w) \in B \text{ (} M_B \text{ decide } B \text{)}$$

$$\Leftrightarrow w \in A \text{ (Definizione di riduzione)}$$

14.8 Corollario Se $A \leq_m B$ e A è Indecidibile, allora B è Indecidibile

Se $A \leq_m B$ e A è indecidibile, allora B è indecidibile

Dimostrazione:

Se B fosse decidibile lo sarebbe anche A in virtù del teorema 5.22

Nota:

Usiamo il contronominale

14.9 Corollario Se $A \leq_m B$ e A non è Turing Riconoscibile, allora B non è Turing Riconoscibile

Se $A \leq_m B$ e A non è Turing Riconoscibile, allora B non è Turing Riconoscibile

Dimostrazione:

Se B fosse Turing riconoscibile lo sarebbe anche A in virtù del Teorema 5.28

14.10 Funzioni Calcolabili

Una funzione $f : \Sigma^* \rightarrow \Sigma^*$ è calcolabile se esiste una macchina di Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ tale che su ogni input, M arresta con $f(w)$, e solo con $f(w)$, sul nastro

Scrittura Compatta:

$$\forall w \in \Sigma^* \quad q_0 w \xrightarrow{*} q_{\text{accept}} f(w)$$

- La MdT si deve arrestare su ogni input
- M si arresta con $f(w)$ e solo con $f(w)$ sul suo nastro
- Le funzioni possono essere anche trasformazioni di codifiche di MdT

14.10.1 Funzioni Aritmetiche Calcolabili

- $\text{incr}(n) = n + 1$
- $\text{dec}(n) = \begin{cases} n-1 & \text{se } n > 0 \\ 0 & \text{se } n = 0 \end{cases}$
- $(m.n) \rightarrow m + n$
- $(m.n) \rightarrow m - n$
- $(m.n) \rightarrow m \cdot n$

14.10.2 Esempio Funzioni Calcolabili

Data una MdT $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, denotiamo con M' la MdT che accetta le stringhe rifiutate da M e rifiuta quelle accettate, ma in generale M' non riconosce il complemento di $L(M)$

Consideriamo la funzione $f : \Sigma^* \rightarrow \Sigma^*$

$$f(y) = \begin{cases} \varepsilon & \text{se } y \neq \langle M \rangle, M \text{ MdT} \\ \langle M' \rangle & \text{se } y = \langle M \rangle \end{cases}$$

Consideriamo la MdT F sull'input y

- Se $y \neq \langle M \rangle$, restituisce ε
- Se $y = \langle M \rangle$, “costruisce” la MdT $M' =$ Sull'input x
 - Simula M su x
 - Se M accetta, rifiuta
 - Se M rifiuta, accetta

Chiamiamo δ la funzione di transizione di M e δ' quella della MdT M' . F cerca nella codifica di M le codifiche delle transizioni della forma $\delta(q, a) = (q_{\text{accept}}, a', D)$, $D \in \{L, R\}$, $a, a' \in \Gamma$, $q \in Q$ e cambia ognuna di esse con la codifica

$$\delta'(q, a) = (q_{\text{reject}}, a', D), D \in \{L, R\}, a, a' \in \Gamma, q \in Q$$

Analogamente F cerca nella codifica di M le codifiche delle transizioni della forma $\delta(q, a) = (q_{\text{reject}}, a', D)$, $D \in \{L, R\}$, $a, a' \in \Gamma$, $q \in Q$ e cambia ognuna di esse con la codifica

$$\delta'(q, a) = (q_{\text{accept}}, a', D), D \in \{L, R\}, a, a' \in \Gamma, q \in Q$$

Esiste una MdT F dato che deve solo scorrere l'input e verificare lo stesso cambiando i caratteri in esso

La stringa in output è la codifica $\langle M' \rangle$ di M' che simula M

14.10.3 Esempio Funzioni Calcolabili 2

Consideriamo A_{TM} e $B = \{ab\}$

Consideriamo la funzione $f : \Sigma^* \rightarrow \Sigma^*$, dove $a, b \in \Sigma$

$$f(y) = \begin{cases} ab & \text{se } y = \langle M, w \rangle \in A_{\text{TM}} \\ a & \text{altrimenti} \end{cases}$$

Quindi f è una funzione tale che $f(y) = a$ se y non è della forma $\langle M, w \rangle$, oppure se $y = \langle M, w \rangle$ con $\langle M, w \rangle \in A_{\text{TM}}$

Quindi per ogni $y \in \Sigma^*$

$$y \in A_{\text{TM}} \Leftrightarrow f(y) \in \{ab\}$$

Idea:

- Supponiamo per assurdo che questa funzione sia calcolabile
- Se fosse calcolabile allora grazie alla definizione di riduzione $A_{\text{TM}} \leq_m \{ab\}$
- Sappiamo che $\{ab\}$ è decidibile
- Tuttavia applicando il Teorema 5.22 se $A_{\text{TM}} \leq_m \{ab\}$ e $\{ab\}$ è decidibile, allora A_{TM} deve essere decidibile, ma è un assurdo

14.11 $\{ab\} \leq_m A_{\text{TM}}$

Sia M la MdT tale che $L(M) = L(a^*)$

Consideriamo la funzione $f : \Sigma^* \rightarrow \Sigma^*$, dove $a, b \in \Sigma$

$$f(y) = \begin{cases} \langle M, a \rangle & \text{se } y = ab \\ \langle M, b \rangle & \text{altrimenti} \end{cases}$$

Con $\langle M, a \rangle \in A_{\text{TM}}$ e $\langle M, b \rangle \notin A_{\text{TM}}$

Dobbiamo dimostrare che f è una riduzione da $\{ab\}$ ad A_{TM} . La funzione f è calcolabile, la MdT F che calcola f sull'input y :

- Se $y = ab$, scrive la stringa $\langle M, a \rangle$ e si ferma
- Se $y \neq ab$, scrive la stringa $\langle M, b \rangle$ e si ferma

Inoltre

$$y \in \{ab\} \Rightarrow y = ab \Rightarrow f(y) = \langle M, a \rangle \in A_{\text{TM}}$$

$$y \notin \{ab\} \Rightarrow y \neq ab \Rightarrow f(y) = \langle M, b \rangle \notin A_{\text{TM}}$$

Quindi per ogni stringa y

$$y \in \{ab\} \Leftrightarrow f(y) \in A_{\text{TM}}$$

In conclusione $\{ab\} \leq_m A_{\text{TM}}$

14.12 $A_{\text{TM}} \leq_m \text{HALT}_{\text{TM}}$

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ è un MdT e } M \text{ si arresta su } w\}$$

Teorema:

$$A_{\text{TM}} \leq_m \text{HALT}_{\text{TM}}$$

Dimostrazione:

Definiamo una funzione calcolabile $f : \Sigma^* \rightarrow \Sigma^*$ tale che per ogni stringa $\langle M, w \rangle$ con M MdT e w stringa e M' MdT

$$f(\langle M, w \rangle) = \langle M', w \rangle$$

Inoltre

$$\langle M, w \rangle \in A_{\text{TM}} \Leftrightarrow \langle M', w \rangle \in \text{HALT}_{\text{TM}}$$

Consideriamo la MdT F sull'input $\langle M, w \rangle$

- Costruisce la MdT $M' =$ Sull'input x
 - Simula M su x
 - Se M accetta, accetta
 - Se M rifiuta, cicla
- Fornisce in output $\langle M', w \rangle$

La funzione f calcolata da F è una riduzione da A_{TM} a HALT_{TM}

$$\langle M, w \rangle \Leftrightarrow M \text{ accetta } w \Leftrightarrow M' \text{ si arresta su } w$$

$$\Leftrightarrow \langle M', w \rangle \in \text{HALT}_{\text{TM}}$$

14.13 HALT_{TM}

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ è un MdT e } M \text{ si arresta su } w\}$$

È indecidibile

$$14.14 \quad A_{\text{TM}} \leq_m \overline{E_{\text{TM}}}$$

$$E_{\text{TM}} = \{\langle M \rangle \mid M \text{ è una MdT e } L(M) = \emptyset\}$$

Proviamo che

$$A_{\text{TM}} \leq_m \overline{E_{\text{TM}}}$$

Dimostrazione:

Definiamo una funzione calcolabile $f : \Sigma^* \rightarrow \Sigma^*$ che per ogni stringa $\langle M, w \rangle$ con M MdT e w stringa e M_1 MdT

$$f(\langle M, w \rangle) = \langle M_1 \rangle$$

Inoltre

$$\langle M, w \rangle \in A_{\text{TM}} \Leftrightarrow \langle M_1 \rangle \in \overline{E_{\text{TM}}}$$

Sia $M_1 = \text{Sull'input } x$

- Se $x \neq w$ allora M_1 si ferma e rifiuta x
- Se $x = w$ allora M_1 simula M su w e accetta x se M accetta $x = w$

La funzione che associa a $\langle M, w \rangle$ la stringa $\langle M_1 \rangle$ è una riduzione da A_{TM} a $\overline{E_{\text{TM}}}$

f è calcolabile e possiamo costruire la MdT F che sull'input $\langle M, w \rangle$ fornisce in output $\langle M_1 \rangle$

Inoltre

$$\langle M, w \rangle \in A_{\text{TM}} \Rightarrow M \text{ accetta } w \Rightarrow L(M_1) \neq \emptyset \Rightarrow \langle M_1 \rangle \in \overline{E_{\text{TM}}}$$

e

$$\langle M, w \rangle \notin A_{\text{TM}} \Rightarrow M \text{ non accetta } w \Rightarrow L(M_1) = \emptyset \Rightarrow \langle M_1 \rangle \notin \overline{E_{\text{TM}}}$$

Quindi

$$\langle M, w \rangle \in A_{\text{TM}} \Leftrightarrow M \text{ accetta } w \Leftrightarrow L(M_1) \neq \emptyset \Leftrightarrow \langle M_1 \rangle \in \overline{E_{\text{TM}}}$$

14.15 Teorema 5.27 $\overline{E_{\text{TM}}}$

$\overline{E_{\text{TM}}}$ è indecidibile

14.16 Corollario E_{TM}

E_{TM} è indecidibile

Prova:

Se E_{TM} fosse decidibile lo sarebbe anche $\overline{E_{\text{TM}}}$, ma questo è in contraddizione col teorema 5.27

Nota:

Non esiste nessuna riduzione da A_{TM} a E_{TM}

14.17 $A_{\text{TM}} \leq_m \text{REGULAR}_{\text{TM}}$

$$\text{REGULAR}_{\text{TM}} = \{\langle M \rangle \mid M \text{ è una MdT e } L(M) \text{ è regolare}\}$$

Proviamo che $A_{\text{TM}} \leq_m \text{REGULAR}_{\text{TM}}$

Dimostrazione:

Definiamo una funzione calcolabile $f : \Sigma^* \rightarrow \Sigma^*$ che per ogni stringa $\langle M, w \rangle$ con M MdT e w stringa e R MdT

$$f(\langle M, w \rangle) = \langle R \rangle$$

Inoltre

$$\langle M, w \rangle \in A_{\text{TM}} \Leftrightarrow \langle R \rangle \in \text{REGULAR}_{\text{TM}}$$

Data una MdT M e una stringa w , sia $R = \text{Sull'input } x$:

- Se $x \in \{0^n 1^n \mid n \in \mathbb{N}\}$, allora R si ferma e accetta w
- Se $x \notin \{0^n 1^n \mid n \in \mathbb{N}\}$, allora R simula M su w e accetta x se M accetta w

$$L(R) = \begin{cases} \Sigma^* & \text{se } \langle M, w \rangle \in A_{\text{TM}} \\ \{0^n 1^n \mid n \in \mathbb{N}\} & \text{altrimenti} \end{cases}$$

La funzione che associa a $\langle M, w \rangle$ la stringa $\langle R \rangle$ è una riduzione da A_{TM} a $\text{REGULAR}_{\text{TM}}$

f è calcolabile e possiamo costruire la MdT F che sull'input $\langle M, w \rangle$ fornisce in output $\langle R \rangle$

Inoltre

$$\begin{aligned}\langle M, w \rangle \in A_{\text{TM}} &\Rightarrow M \text{ accetta } w \Rightarrow L(R) = \Sigma^* \text{ è regolare} \\ &\Rightarrow \langle R \rangle \in \text{REGULAR}_{\text{TM}}\end{aligned}$$

$$\begin{aligned}\langle M, w \rangle \notin A_{\text{TM}} &\Rightarrow M \text{ non accetta } w \Rightarrow L(R) = \Sigma^* \text{ non è regolare} \\ &\Rightarrow \langle R \rangle \notin \text{REGULAR}_{\text{TM}}\end{aligned}$$

In conclusione

$$\begin{aligned}\langle M, w \rangle \in A_{\text{TM}} &\Leftrightarrow M \text{ accetta } w \Leftrightarrow L(R) = \Sigma^* \text{ è regolare} \\ &\Leftrightarrow \langle R \rangle \in \text{REGULAR}_{\text{TM}}\end{aligned}$$

14.18 Teorema $\text{REGULAR}_{\text{TM}}$

$\text{REGULAR}_{\text{TM}} = \{\langle M \rangle \mid M \text{ è una MdT e } L(M) \text{ è regolare}\}$ è indecidibile

$$\mathbf{14.19} \quad E_{\text{TM}} \stackrel{m}{\leq} EQ_{\text{TM}}$$

$$E_{\text{TM}} = \{\langle M \rangle \mid M \text{ è una MdT e } L(M) = \emptyset\}$$

$$EQ_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ sono MdT e } L(M_1) = L(M_2)\}$$

Proviamo che $E_{\text{TM}} \stackrel{m}{\leq} EQ_{\text{TM}}$

Sia M_1 una MdT tale che $L(M_1) = \emptyset$, quindi data una MdT M avremmo che $L(M) = L(M_1)$ se e solo se $L(M) = \emptyset$

La funzione che associa a $\langle M \rangle$ la stringa $\langle M, M_1 \rangle$ è una riduzione da E_{TM} a EQ_{TM}

f è calcolabile e possiamo costruire la MdT F che sull'input $\langle M \rangle$ fornisce in output $\langle M, M_1 \rangle$

Inoltre

$$\langle M \rangle \in E_{\text{TM}} \Leftrightarrow L(M) = \emptyset \Leftrightarrow L(M) = L(M_1)$$

$$\Leftrightarrow \langle M, M_1 \rangle \in EQ_{\text{TM}}$$

14.20 Teorema EQ_{TM}

$EQ_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ sono MdT e } L(M_1) = L(M_2)\}$ è indecidibile

14.21 $A_{\text{TM}} \leq_m EQ_{\text{TM}}$

Data $\langle M, w \rangle$, consideriamo una MdT M_1 che riconosce Σ^* e una MdT M_2 che riconosce Σ^* se M accetta w

Per ogni input x :

- M_1 accetta x
- M_2 simula M su w , se M accetta, M_2 accetta

Quindi $L(M) = \Sigma^*$ e

$$L(M_2) = \begin{cases} \Sigma^* & \text{se } \langle M, w \rangle \in A_{\text{TM}} \\ \emptyset & \text{altrimenti} \end{cases}$$

Di conseguenza $L(M_1) = L(M_2)$ se e solo se M accetta w

Consideriamo la MdT $G = \text{Su input } \langle M, w \rangle$, dove M MdT e w è una stringa:

- Costruisce le due MdT M_1 e M_2
 - $M_1 = \text{Su ogni input:}$
 - Accetta
 - $M_2 = \text{Su ogni input:}$
 - Esegue M su w
 - Se M accetta, accetta
- Restituisce $\langle M_1, M_2 \rangle$

G calcola una funzione g che associa a $\langle M, w \rangle$ la stringa $\langle M_1, M_2 \rangle$, ed è una riduzione da A_{TM} a EQ_{TM} ed è calcolabile

$$\langle M, w \rangle \in A_{\text{TM}} \Leftrightarrow M \text{ accetta } w \Leftrightarrow L(M_1) = \Sigma^* = L(M_2)$$

$$\Leftrightarrow \langle M, M_1 \rangle \in EQ_{\text{TM}}$$

14.22 $A_{\text{TM}} \leq_m \overline{EQ_{\text{TM}}}$

Data $\langle M, w \rangle$, consideriamo una MdT M_1 che riconosce $\emptyset$ e una MdT M_2 che riconosce Σ^* se M accetta w

Per ogni input x :

- M_1 rifiuta x
- M_2 simula M su w , se M accetta, M_2 accetta

Quindi $L(M) = \emptyset$ e

$$L(M_2) = \begin{cases} \Sigma^* & \text{se } \langle M, w \rangle \in A_{\text{TM}} \\ \emptyset & \text{altrimenti} \end{cases}$$

Di conseguenza $L(M_1) \neq L(M_2)$ se e solo se M accetta w

Consideriamo la MdT $F = \text{Su input } \langle M, w \rangle$, dove M MdT e w è una stringa:

- Costruisce le due MdT M_1 e M_2
 - $M_1 = \text{Su ogni input:}$
 - Rifiuta
 - $M_2 = \text{Su ogni input:}$
 - Esegue M su w
 - Se M accetta, accetta
- Restituisce $\langle M_1, M_2 \rangle$

F calcola una funzione f che associa a $\langle M, w \rangle$ la stringa $\langle M_1, M_2 \rangle$, ed è una riduzione da A_{TM} a $\overline{EQ_{\text{TM}}}$ ed è calcolabile

$$\langle M, w \rangle \in A_{\text{TM}} \Leftrightarrow M \text{ accetta } w \Leftrightarrow L(M_2) = \Sigma^* \neq \emptyset = L(M_1)$$

$$\Leftrightarrow \langle M, M_1 \rangle \in \overline{EQ_{\text{TM}}}$$

14.23 Teorema $A \leq B$ se e solo se $\overline{A} \leq_m \overline{B}$

$A \leq_m B$ se e solo se $\overline{A} \leq_m \overline{B}$

14.24 Corollario 4.23 $\overline{A_{TM}}$ non è Turing Riconoscibile

$\overline{A_{TM}}$ non è Turing riconoscibile

14.25 Teorema EQ_{TM} non è Nè Turing Riconoscibile
e Nè co-Turing Riconoscibile

Dimostrazione:

Supponiamo per assurdo che EQ_{TM} sia Turing riconoscibile

Siccome $A_{TM} \leq_m \overline{EQ_{TM}}$ allora $\overline{A_{TM}} \leq_m EQ_{TM}$ per il Teorema precedente

Quindi per il Teorema 5.28, se EQ_{TM} fosse Turing riconoscibile allora $\overline{A_{TM}}$ sarebbe Turing riconoscibile, ma va in contraddizione con il Corollario 4.23

Supponiamo per assurdo che EQ_{TM} sia co-Turing riconoscibile, cioè che $\overline{EQ_{TM}}$ sia Turing riconoscibile

Siccome $A_{TM} \leq_m EQ_{TM}$ allora $\overline{A_{TM}} \leq_m \overline{EQ_{TM}}$

Quindi per il Teorema 5.28, se $\overline{EQ_{TM}}$ fosse Turing riconoscibile allora $\overline{A_{TM}}$ sarebbe Turing riconoscibile, ma è in contraddizione con il Corollario 4.23

14.26 Teorema di Rice

Sia $L = \{\langle M \rangle \mid M \text{ una MdT che verifica una proprietà } \mathcal{P}\}$ un linguaggio che soddisfa le seguenti tre condizioni:

- L'appartenenza di $\langle M \rangle$ a L dipende solo da $L(M)$, ovvero
 $\forall M_1, M_2 \text{ MdT tali che } L(M_1) = L(M_2)$

$$\langle M_1 \rangle \in L \Leftrightarrow \langle M_2 \rangle \in L$$

$\mathcal{P}$ non è banale, cioè L non è vuoto e non contiene tutte le codifiche delle MdT

- $\exists$ una MdT M_1 tale che $\langle M_1 \rangle \in L$
- $\exists$ una MdT M_2 tale che $\langle M_2 \rangle \notin L$

Allora L è indecidibile

15 Complessità di Tempo

15.1 Definizione Complessità di Tempo

Sia $M = Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}$ una MdT deterministica che si arresta su ogni input

La complessità di tempo M è la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ dove $f(n)$ è il massimo numero di passi di computazione eseguiti da M su un input di lunghezza $n, n \in \mathbb{N}$

Se M ha complessità di tempo $f(n)$, diremo che M decide $L(M)$ in tempi (deterministico) $f(n)$

Quindi se M è una MdT deterministica a nastro singolo, che si arresta su ogni input e $C_1, C_2, \dots, C_{k+1}, k \geq 1$, sono configurazioni di M , tali che:

- $C_1 = q_0 w$ è la configurazione iniziale di M con input w
- $C_i = C_{i+1}$ per ogni $i \in \{1, \dots, k\}$
- C_{k+1} è una configurazione di arresto

Il numero di passi eseguiti da M su w è k

$f(n) = \text{Max num. di passi in } q_0 w \xrightarrow{*} uqv, q \in \{q_{\text{accept}}, q_{\text{reject}}\}, \text{ al variare di } w \in \Sigma^*$

15.2 Astrazioni

- Input hanno la stessa lunghezza
- Valutazione del caso peggiore
- Uso della notazione asintotica O -grande

15.3 Analisi Asintotica

Sia f e g due funzioni

$$f : \mathbb{N} \rightarrow \mathbb{R}^+, g : \mathbb{N} \rightarrow \mathbb{R}^+$$

Diremo che $f(n)$ è $O(g(n))$ oppure $f(n) = O(g(n))$ se esiste una costante $c > 0$ e una costante $n_0 \geq 0$ tali che per ogni $n \geq n_0$

$$f(n) \leq cg(n)$$

Diremo che $g(n)$ è un limite superiore per $f(n)$

15.4 Classi di Complessità

Sia $f : \mathbb{N} \rightarrow \mathbb{R}^+$ una funzione, sia $\mathcal{M}$ l'insieme delle MdT deterministiche che si arrestano su ogni input

La classe di complessità di tempo deterministico $\text{TIME}(f(n))$ è

$$\text{TIME}(f(n)) = \{L \mid \exists M \in \mathcal{M} \text{ che decide } L \text{ in tempo } O(f(n))\}$$

- $\text{TIME}(1)$: Insieme dei linguaggi per i quali esiste un decider che li decide in tempo $O(1)$ (Tempo costante)
- $\text{TIME}(n)$: Insieme dei linguaggi per i quali esiste un decider che li decide in tempo $O(n)$ (Tempo lineare)
- $\text{TIME}(n^k)$: Insieme dei linguaggi per i quali esiste un decider che li decide in tempo $O(n^k)$ (Tempo polinomiale)
- $\text{TIME}(2^n)$: Insieme dei linguaggi per i quali esiste un decider che li decide in tempo $O(2^n)$ (Tempo esponenziale)

15.5 Osservazione sulla Complessità di Tempo

- La complessità di tempo dipende dal modello di calcolo

Le varianti delle MdT deterministiche sono polinomialmente equivalenti, cioè possono simularsi tra di loro con un sovraccarico computazionale equivalente, fanno eccezione le MdT non deterministica

- La complessità di tempo dipende dalla codifica utilizza, in particolare cambia la lunghezza del valore della funzione rispetto alla lunghezza dell'input

Se scegliamo per l'input la rappresentazione unaria,

$\langle m \rangle =$ rappresentazione unaria di m ,

la macchina che calcola f è la macchina che copia:

$\langle m \rangle \rightarrow \langle m \rangle \# \langle m \rangle$

È semplice progettare una macchina di Turing M che calcoli la funzione f e che abbia complessità di tempo $O(|\langle m^2 \rangle|)$ (**polinomiale**).

Supponiamo di scegliere per l'input m la rappresentazione in base 2.

Se $m = a_0 2^0 + \dots + a_k 2^k$, l'input $\langle m \rangle$ è $a_k \dots a_0$ e ha lunghezza $k + 1$. (È noto che $k + 1 = |\langle m \rangle|$ è $O(\log m)$).

La macchina M che calcola f deve scrivere (dopo l'ultimo carattere a destra dell'input $\langle m \rangle$) una stringa di lunghezza $m + 1 = t$.

Siccome $2^k \leq m \leq 2^{k+1} - 1$ allora

$$2^{|\langle m \rangle| - 1} + 1 \leq t \leq 2^{|\langle m \rangle|}$$

Poiché M deve eseguire almeno t passi, la complessità di tempo di M è $O(2^{|\langle m \rangle|})$ (e anche $\Omega(2^{|\langle m \rangle|})$).

15.6 Relazione fra Modelli: MdT Multinastro

Teorema:

Sia $t(n)$ una funzione tale che $f(n) \geq n$, per ogni MdT deterministica multinastro M con complessità di tempo $t(n)$ esiste una MdT deterministica a nastro singolo M' con complessità di tempo $O(t^2(n))$ equivalente a M

La funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ è la funzione definita da $t^2(n) = (t(n))$, il teorema afferma che M' utilizza $O([t(n)]^2)$ passi per simulare $t(n)$ passi di M

15.7 Tempo di Esecuzione MdT non Deterministica

Sia $N = Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}$ una MdT non deterministica che sia un decisore

Il tempo di esecuzione di N è la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ dove $f(n)$ è il massimo numero di passi eseguiti da N in ognuna delle computazioni su ogni input di lunghezza $n, n \in \mathbb{N}$

Il tempo di esecuzione di una MdT non deterministica su input w viene definito come il tempo usato dalla ramificazione più lunga

Quindi la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ dove

$$f(n) = \text{Max altezze degli alberi, ognuno dei quali}$$

rappresenta le possibili computazioni su input w , al variare di $w \in \Sigma^n$

15.8 Relazione fra Modelli: MdT non Deterministica

Teorema:

Sia $t(n)$ una funzione tale che $t(n) \geq n$

Per ogni MdT a nastro singolo non deterministica N avente tempo di esecuzione $t(n)$ esiste una MdT a nastro singolo deterministica e di complessità di tempo $2^{O(t(n))}$, equivalente ad N

15.9 La Classe P: Tempo Polinomiale

Definizione:

La classe P è l'insieme dei linguaggi L per i quali esiste una MdT deterministica M con un solo nastro che decide L in tempo $O(n^k)$ per qualche $k \geq 1$

$$P = \cup_{k \geq 1} \text{TIME}(n^k)$$

- P corrisponde alla classe di problemi che sono realisticamente risolvibili mediante programmi su computer reali

15.10 Teorema Classe P

Sia $t(n)$ una funzione tale che $t(n) \geq n$

Per ogni MdT multinastro M con complessità di tempo $t(n)$ esiste una MdT a nastro singolo M' con complessità di tempo $O(t^2(n))$, equivalente a M

Quindi, se L è deciso in tempo polinomiale su una MdT multinastro, allora L è deciso in tempo polinomiale su una MdT a nastro singolo

- P è invariante per tutti i modelli di computazione che sono polinomialmente equivalenti alla MdT deterministica a nastro singolo
- La classe P è invariante rispetto alla scelta di una codifica ragionevole dell'input

Nota:

Per mostrare che un algoritmo può essere eseguito in tempo $O(n^k)$ su un input di lunghezza n

- Dobbiamo fornire un limite superiore polinomiale al numero dei passi eseguiti dall'algoritmo
- Mostrare che ogni passo può essere eseguito in tempo polinomiale da un qualsiasi ragionevole modello di computazione deterministico

15.10.1 PATH

PATH = $\{\langle G, s, t \rangle \mid G \text{ è un grafo orientato in cui c'è un cammino da } s \text{ a } t\}$

Teorema:

PATH $\in P$

- Una generazione esaustiva dei cammini di G condurrebbe a un algoritmo di complessità esponenziale, quindi con V insieme dei nodi di G , ovvero $O(2^{\langle G, s, t \rangle})$

Il seguente algoritmo M decide PATH in tempo deterministico polinomiale

$M = \text{Sull'input } \langle G, s, t \rangle$, dove G è un grafo con nodi s e t

- Marca il nodo s
- Ripete questa operazione finchè nessun nuovo vertice viene marcato:
 - Scansiona tutti gli archi di G , se trova un arco (a, b) che va da un nodo a marcato ad un nodo b non marcato, marca il nodo b
- Se t è marcato, accetta. Altrimenti rifiuta

M decide il PATH in tempo deterministico polinomiale

Il passo 3 viene eseguito al più m volte, se m il numero dei vertici di G , quindi il totale dei passi è al più $1 + 1 + m$

Infine i passi 1,3,4 possono essere implementati in tempo polinomiale nella lunghezza sull'input su una MdT deterministica

15.10.2 RELPRIME

Due numeri interi positivi x, y sono relativamente primi se il loro massimo comun divisore è 1

$\text{RELPRIME} = \{\langle x, y \rangle \mid x \text{ e } y \text{ sono interi positivi relativamente primi}\}$

Teorema:

$\text{RELPRIME} \in P$

- Una ricerca esaustiva dei divisori non banale x e y condurrebbe a un algoritmo di complessità esponenziale

Se $\langle x \rangle = a_k \cdots a_0$ è la rappresentazione binaria di x allora $x - 2$ è $O(2^k) = O(2^{|\langle x \rangle|})$

- Per provare che $\text{RELPRIME} \in P$ ci basiamo sull'algoritmo di Euclide

Teorema Ricorsione del MCD:

Per un qualsiasi numero intero a non negativo e qualunque intero b positivo, $MCD(a, b) = MCD(b, a \pmod{b})$

Algoritmo di Euclide:

$MCD(a, b)$

```
if b=0 then MCD = a
  else MCD = MCD(b, a (mod b))
```

- Sono necessarie $O(\log b)$ chiamate ricorsive

Quindi consideriamo l'algoritmo R :

$R =$ Sull'input $\langle x, y \rangle$, dove x e y sono numeri naturali in binario:

- Simula MCD su $\langle x, y \rangle$
- Se il risulta è 1 accetta, altrimenti rifiuta

Quindi R decide RELPRIME in tempo deterministico polinomiale

15.11 Tipi di Problemi

- **Trattabili:** Problemi per i quali esistono algoritmi polinomiali per decidere i linguaggi associati
- **Intrattabili:** Problemi per i quali esistono algoritmi esponenziali per decidere i linguaggi associati

15.12 Classe EXPTIME

$$\text{EXPTIME} = \cup_{K \geq 1} \text{TIME}(2^{n^K})$$

In particolare

$$P \subset \text{EXPTIME}$$

Esistono linguaggi del tipo $\text{EXPTIME} \setminus P$ come

$$EQ_{\text{REX} \uparrow} = \{\langle Q, R \rangle \mid Q \text{ ed } R \text{ sono } ERG \text{ equivalenti}\}$$

Risulta che $EQ_{\text{REX} \uparrow} \in \text{EXPTIME} \setminus P$

15.13 HAMPATH

Un cammino Hamiltoniano in un grafo orientato è un cammino che passa per ogni vertice del grafo una e una sola volta

$$\text{HAMPATH} = \{\langle G, s, t \rangle \mid G \text{ è un grafo orientato e}$$

ha un cammino Hamiltoniano da s a t

Anche se non conosciamo un algoritmo polinomiale per determinare se un grafo contiene un cammino Hamiltoniano, se un tale cammino esiste la verifica che si tratta di un cammino Hamiltoniano può essere fatta in tempo polinomiale

Quindi esiste un algoritmo N , polinomiale in $|\langle G, s, t \rangle|$ che sull'input $\langle \langle G, s, t \rangle, c \rangle$, dove $c = (u_1, \dots, u_{|V|})$, decide il linguaggio

$$\{\langle \langle G, s, t \rangle, c \rangle \mid G \text{ è un grafo orientato e}$$

c è un cammino Hamiltoniano da s a t

Basta verificare che i nodi della sequenza siano distinti, che $u_1 = s, u_{|V|} = t$ e che per ogni $i, 2 \leq i \leq n, (u_{i-1}, u_i) \in E$

Teorema:

HAMPATH $\in NP$

Dimostrazione:

Un algoritmo N che verifica HAMPATH in tempo polinomiale:

$N =$ Sull'input $\langle \langle G, s, t \rangle, c \rangle$, dove $G = (V, E)$ è un grafo orientato

- Verifica se $c = (u_1, \dots, u_{|V|})$ è una sequenza di $|V|$ vertici di G , altrimenti rifiuta
- Verifica se i nodi della sequenza sono distinti, $u_1 = s, u_{|V|} = t$ e per ogni i con $2 \leq i \leq n$ se $(u_{i-1}, u_i) \in E$, accetta in caso affermativo, altrimenti rifiuta

$\exists c : \langle \langle G, s, t \rangle, c \rangle \in L(N) \Leftrightarrow \langle G, s, t \rangle \in \text{HAMPATH}$

15.14 COMPOSITES

COMPOSITES = $\{ \langle x \rangle \mid \text{esistono interi } p, q, \text{ con } p > 1, q > 1 \text{ tali che } x = pq \}$

Dati x, p , verificare che p è un divisore di x , con $p > 1, p \neq x$, può essere fatto in tempo polinomiale

15.15 Algoritmo di Verifica

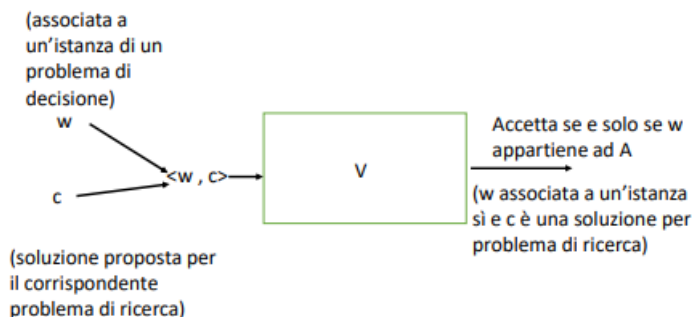
Definizione:

Un algoritmo di verifica (o verificatore) V per un linguaggio A è un algoritmo tale che

$$A = \{ w \mid \exists c \text{ tale che } V \text{ accetta } \langle w, c \rangle \}$$

La stringa c prende il nome di certificato o prova

A è il linguaggio verificato da V



Nota: Algoritmo=Decider

15.16 Complessità degli Algoritmi di Verifica

Misurata in termini della lunghezza $|w|$ di w

Definizione:

Un algoritmo V è un verificatore per A in tempo polinomiale se

- A è il linguaggio verificato da V

$$A = \{w \mid \exists c \text{ tale che } V \text{ accetta } \langle w, c \rangle\}$$

- V ha complessità di tempo polinomiale $|w|$

Nota:

Se V è un algoritmo di verifica e ha complessità polinomiale in $|w|$, allora il certificato ha lunghezza polinomiale nella lunghezza di w , ovvero esiste t tale che per ogni w , $|c| = O(|w|^t)$

Questo è imposto dal limite di tempo polinomiale per la computazione di V , altrimenti se c non avesse lunghezza polinomiale in $|w|$, non sarebbe esaminabile da V

15.17 NP

Classe dei linguaggi verificabili in tempo polinomiale

15.18 Classe Di Complessità in Tempo non Polinomiale

NP è la classe dei linguaggi verificabili in tempo polinomiale, rappresenta l'annotazione per tempo polinomiale non deterministico

Definizione:

Sia $t : \mathbb{N} \rightarrow \mathbb{R}^+$ una funzione

La classe di complessità in tempo non deterministico $\text{NTIME}(t(n))$ è

$$\text{NTIME}(t(n)) = \{L \mid \exists \text{ una MdT non deterministica } M$$

che decide L in tempo $O(t(n))$

15.19 Teorema 7.20 Linguaggio in NP

Un linguaggio L è in NP se e solo se esiste una MdT non deterministica che decide L in tempo polinomiale

15.20 Corollario 7.22

$$NP = \cup_{k \geq 1} \text{NTIME}(n^k)$$

15.21 CLIQUE

Definizione:

Una clique in un grafo non orientato G è un sottografo di G in cui ogni coppia di vertici è connessa da un arco

Una k -clique è una clique che contiene k vertici

$$\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ è un grafo non orientato in cui esiste una } k\text{-clique}\}$$

Teorema:

$$\text{CLIQUE} \in NP$$

Dimostrazione:

Un algoritmo V che verifica CLIQUE in tempo polinomiale:

$V = \text{Sull'input } \langle \langle G, k \rangle, c \rangle$:

- Verifica se c è un insieme di k nodi di G , altrimenti rifiuta

- Verifica se per ogni coppia di nodi in c , esiste un arco in G che li connette, accetta in caso affermativo, altrimenti rifiuta

$$\exists c : \langle \langle G, k \rangle, c \rangle \in L(V) \Leftrightarrow \langle G, k \rangle \in \text{CLIQUE}$$

15.22 SUBSET-SUM

Dato un insieme finito S di numeri interi e un numero intero t , esiste un sottoinsieme S' di S tale che la somma dei suoi numeri sia uguale a t

$\text{SUBSET-SUM} = \{\langle S, t \rangle \mid S = \{x_1, \dots, x_k\} \text{ ed esiste}$

$$S' \subseteq S \text{ tale che } \sum_{s \in S'} s = t$$

Teorema:

$\text{SUBSET-SUM} \in NP$

Dimostrazione:

Un algoritmo V che verifica SUBSET-SUM in tempo polinomiale:

$V = \text{Sull'input } \langle \langle S, t \rangle, c \rangle$:

- Verifica se c è un insieme di numeri la cui somma è t , altrimenti rifiuta
- Verifica se S contiene tutti i numeri in c , accetta in caso affermativo, altrimenti rifiuta

$\exists c : \langle \langle S, t \rangle, c \rangle \in L(V) \Leftrightarrow \langle S, t \rangle \in \text{SUBSET-SUM}$

15.23 $P \subseteq NP$

Teorema 1:

Dimostrazione:

Se $L \in P$, esiste un algoritmo M che decide L in tempo polinomiale

Consideriamo l'algoritmo di verifica V che sull'input y

- Se $y \neq \langle w, \varepsilon \rangle$, w stringa, rifiuta y
- Se $y = \langle w, \varepsilon \rangle$, w stringa, simula M su w
- Accetta $y = \langle w, \varepsilon \rangle$ se e solo se M accetta w

$$\begin{aligned} A &= \{w \mid \exists c \text{ tale che } V \text{ accetta } \langle w, c \rangle\} \\ &= \{w \mid V \text{ accetta } \langle w, \varepsilon \rangle\} (\text{Definizione di } V) \\ &= \{w \mid M \text{ accetta } w\} (\text{Definizione di } V) \\ &= L \end{aligned}$$

Inoltre V verifica L in tempo polinomiale perchè M ha complessità di tempo polinomiale in $|w|$

Teorema 1

$$P \subseteq NP$$

Dimostrazione alternativa

Se $L \in P$, esiste un macchina di Turing deterministica

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ che decide L in tempo polinomiale.

Sia $M' = (Q, \Sigma, \Gamma, \delta', q_0, q_{\text{accept}}, q_{\text{reject}})$ la macchina di Turing non deterministica tale che

$$\delta'(q, \gamma) = \{\delta(q, \gamma)\}$$

per ogni $q \in Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}$ e ogni $\gamma \in \Gamma$.

È facile provare che M' è una macchina di Turing non deterministica equivalente ad M e che M' decide L in tempo polinomiale.

15.24 Una Gerarchia di Classi

$$P \subseteq NP = \bigcup_{k \geq 1} \text{NTIME}(n^k) \subseteq \text{EXPTIME} = \bigcup_{k \geq 1} \text{TIME}(2^{n^k})$$

15.25 Chiusura Classe P Rispetto al Complemento

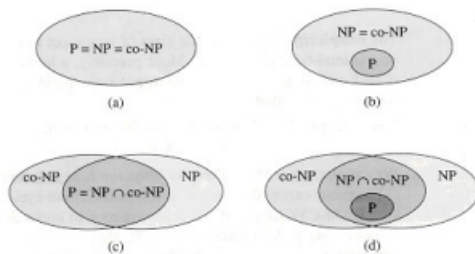
La classe P è chiusa rispetto al complemento

15.26 coNP

$$\text{coNP} = \{L \mid \bar{L} \in \text{NP}\}$$

Esempi:

- $\overline{\text{HAMPATH}}$
- $\overline{\text{CLIQUE}}$
- $\overline{\text{SUBSET-SUM}}$



Vi sono quattro possibilità:

- ① $P = NP = \text{coNP}$
- ② $P \subsetneq NP = \text{coNP}$
- ③ $NP \neq \text{coNP}$, $P = NP \cap \text{coNP} \subsetneq NP$ e $P = NP \cap \text{coNP} \subsetneq \text{coNP}$
- ④ $NP \neq \text{coNP}$, $P \subsetneq NP \cap \text{coNP} \subsetneq NP$ e $P \subsetneq NP \cap \text{coNP} \subsetneq \text{coNP}$

15.27 Funzioni Calcolabili in Tempo Polinomiale

Una funzione $f : \Sigma^* \rightarrow \Sigma^*$ è calcolabile in tempo polinomiale se esiste una macchina di Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ di complessità di tempo polinomiale tale che su ogni input, M si arresta con $f(w)$, e solo con $f(w)$, sul nastro

15.28 Riduzioni di Tempo Polinomiali

Definizione:

Siano A, B linguaggi sull'alfabeto Σ

Una riduzione di tempo polinomiale f di A a B è

- Una funzione $f : \Sigma^* \rightarrow \Sigma^*$
- Calcolabile in tempo polinomiale

Tale che

$$\forall w \in \Sigma^*, w \in A \Leftrightarrow f(w) \in B$$

Un linguaggio $A \subseteq \Sigma^*$ è riducibile in tempo polinomiale mediante una funzione a un linguaggio $B \subseteq \Sigma^*$ $\left(A \leq_p B \right)$ se esiste una riduzione di tempo polinomiale di A a B

Nota:

- Se un linguaggio A su un alfabeto Σ associato ad un problema di decisione $\mathbb{P}_D$, le stringhe $w \in \Sigma^*$ si dividono in tre gruppi
- w è la codifica di una istanza di $\mathbb{P}_D$ per la quale ammette risposta sì
- w è la codifica di una istanza di $\mathbb{P}_D$ per la quale ammette risposta no
- w non è la codifica di una istanza di $\mathbb{P}_D$

Quindi dato A e f è una riduzione di A a B , la MdT F che calcola f utilizza inizialmente un algoritmo polinomiale nella lunghezza dell'input w per decidere se è una codifica di un'istanza di $\mathbb{P}_D$

15.29 Riducibilità in Tempo Polinomiale

Teorema:

Se $A \leq_P B$ e $B \in P$, allora $A \in P$

Dimostrazione:

Per ipotesi $B \in P$, quindi esiste un algoritmo M di complessità $O(m^t)$ che decide B

Inoltre $A \leq_P B$, sia f la riduzione di tempo polinomiale di A e B e sia

F l'algoritmo di complessità $O(n^k)$ che calcola f

Consideriamo l'algoritmo N che sull'input w :

- Simula F su w e calcola $f(w)$
- Simula M sull'input $f(w)$ per decidere se $f(w) \in B$
- N accetta w se M accetta $f(w)$, N rifiuta w se M rifiuta $f(w)$

$$N : w \rightarrow \boxed{F \rightarrow f(w) \rightarrow M}$$

N decide A , infatti si ferma su w se si fermano F ed M

Per ogni w , F si ferma con $f(w)$ sul nastro e M si ferma su $f(w)$ essendo un decider

Inoltre N riconosce A

$$w \in L(N) \Leftrightarrow f(w) \in L(M) \text{ (Definizione di } N \text{)}$$

$$\Leftrightarrow f(w) \in B \text{ (} M \text{ decide } B \text{)}$$

$$\Leftrightarrow w \in A \text{ (} f \text{ è una riduzione polinomiale di } A \text{ a } B \text{)}$$

N è un algoritmo polinomiale in $n = |w|$, infatti F calcola $f(w)$ in $O(n^k)$ passi, quindi in $q(n)$ passi per qualche polinomio q

In particolare $|f(w)| \leq q(n)$ dato che la lunghezza sull'output di F è limitata dalla complessità di tempo di F

Al secondo passo M viene eseguito sull'input $f(w)$ e si arresta dopo $p(|f(w)|) \leq p(q(n))$ passi per qualche polinomio p

In conclusione N ha complessità polinomiale e quindi $A \in P$

15.30 Proprietà Transitiva di $\leq_P$

Se $A \leq_P B$ e $B \leq_P C$, allora $A \leq_P C$

Dimostrazione:

Per ipotesi esiste una riduzione di tempo polinomiale $f : \Sigma^* \rightarrow \Sigma^*$ di A a B ed esiste una riduzione di tempo polinomiale $g : \Sigma^* \rightarrow \Sigma^*$ di B in C

Consideriamo la composizione $g \circ f : \Sigma^* \rightarrow \Sigma^*$ delle funzioni f e g , definita da $(g \circ f)(w) = g(f(w))$

Risulta per ogni $w \in \Sigma^*$:

$$\begin{aligned} w \in A &\Leftrightarrow f(w) \in B \text{ (} f \text{ riduzione polinomiale di } A \text{ a } B \text{)} \\ &\Leftrightarrow g(f(w)) \in C \text{ (} g \text{ riduzione polinomiale di } B \text{ a } C \text{)} \end{aligned}$$

Inoltre $g \circ f$ è una funzione calcolabile in tempo polinomiale

Sia F l'algoritmo di complessità $O(n^k)$ che calcola la funzione f

Sia G l'algoritmo di complessità $O(m^t)$ che calcola la funzione g

Consideriamo l'algoritmo GF che sull'input w :

- Simula F su w e calcola $f(w)$
- Simula G sull'input $f(w)$ e calcola $g(f(w))$
- Fornisce in output l'output di G

GF è un algoritmo polinomiale in $n = |w|$, infatti F calcola $f(w)$ in $O(n^k)$ passi, quindi in $q(n)$ passi per qualche polinomio q

In particolare $|f(w)| \leq q(n)$ dato che la lunghezza sull'output di F è limitata dalla complessità di tempo di F

Al secondo passo G viene eseguito sull'input $f(w)$ e si arresta dopo $p(|f(w)|) \leq p(q(n))$ passi per qualche polinomio p

In conclusione GF ha complessità polinomiale

15.31 Richiami di Logica

- **Variabili Booleane:** variabili che possono assumere valore VERO o FALSO. In genere rappresentiamo VERO con 1 e FALSO con 0.
- **Operazioni Booleane:** $\vee$ (o OR), $\wedge$ (o AND), $\neg$ (o NOT)
- Denotiamo $\neg x$ con $\bar{x}$
- $0 \vee 0 = 0, 0 \vee 1 = 1 \vee 0 = 1 \vee 1 = 1$
- $0 \wedge 0 = 0 \wedge 1 = 1 \wedge 0 = 0, 1 \wedge 1 = 1$
- $\bar{0} = 1, \bar{1} = 0$

Definizione

Dato un insieme di variabili booleane X , le *formule booleane* (o *espressioni booleane*) su X sono definite induttivamente come segue:

- le costanti 0, 1 e le variabili (in forma diretta o complementata) $x, \bar{x}$, con $x \in X$, sono formule booleane.
- Se ϕ, ϕ_1, ϕ_2 sono formule booleane allora $(\phi_1 \vee \phi_2), (\phi_1 \wedge \phi_2), \bar{\phi}$ sono formule booleane.

Si definisce letterale ogni presenza in forma diretta o negata di una variabile in una espressione e numero di letterali il loro numero

Una formula booleana ϕ è soddisfacibile se esiste un insieme di valori 0 e 1 per le variabili di ϕ che renda la formula uguale a 1

Definizione

Una clausola è un OR di letterali.

Esempio: $(\bar{x} \vee x \vee y \vee z)$

Definizione

Una formula booleana ϕ è in forma normale congiuntiva (o forma normale POS) se è un AND di clausole, cioè è un AND di OR di letterali.

Esempio:

$$(\bar{x}_1 \vee x_2 \vee x_3 \vee x_4) \wedge (x_3 \vee \bar{x}_6) \wedge (x_3 \vee \bar{x}_5 \vee x_5)$$

Esiste una nozione duale di forma normale (forma normale disgiuntiva o *SOP*).

Data un'espressione booleana ϕ se ne può costruire una equivalente ϕ' in forma normale congiuntiva (o disgiuntiva). Se ϕ ha n simboli, la sua forma normale può avere un numero di simboli esponenziale in n .

Ricordiamo: due espressioni booleane ϕ, ϕ' sullo stesso insieme di variabili sono *equivalenti* se per ogni assegnamento alle variabili, ϕ e ϕ' assumono lo stesso valore.

Definizione

Una formula booleana è in forma normale 3-congiuntiva se è un AND di clausole e tutte le clausole hanno tre letterali.

Esempio:

$$(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_3 \vee \overline{x_6} \vee x_6) \wedge (x_3 \vee \overline{x_5} \vee x_5)$$

Abbreviazioni:

CNF = formula booleana in forma normale congiuntiva

3CNF = formula booleana in forma normale 3-congiuntiva

kCNF = formula booleana in forma normale k -congiuntiva

= AND di clausole e tutte le clausole hanno k letterali

$$3SAT = \{\langle \phi \rangle \mid \phi \text{ è una formula 3CNF soddisfacibile}\}$$

15.32 SAT

$SAT = \{\langle \phi \rangle \mid \phi \text{ è una formula booleana soddisfacibile}\}$

Teorema:

$SAT \in NP$

Dimostrazione:

Un certificato per $\langle \phi \rangle$ sarà un assegnamento c di valori alle variabili di ϕ

Un algoritmo V che verifica SAT in tempo polinomiale nella lunghezza di $\langle \phi \rangle$:

$V =$ Sull'input y :

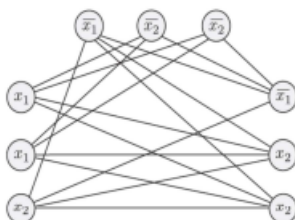
- Verifica se $y = \langle \langle \phi \rangle, c \rangle$, altrimenti rifiuta
- Sostituisce ogni variabile della formula con il suo corrispondente valore e quindi valuta l'espressione
- Accetta se ϕ assume valore 1, altrimenti rifiuta

$$\exists c : \langle \langle \phi \rangle, c \rangle \in L(V) \Leftrightarrow \langle \phi \rangle \in SAT$$

15.33 3SAT è Riducibile in Tempo Polinomiale a CLIQUE

$3SAT = \{\langle \phi \rangle \mid \phi \text{ è una formula 3CNF soddisfacibile}\}$

3CNF è un AND di clausole e tutte le clausole hanno tre letterali



Teorema:

3SAT è riducibile in tempo polinomiale a CLIQUE

Dimostrazione:

Sia ϕ una formula 3CNF a k clausole

$$(a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$$

Consideriamo la funzione f che associa a $\langle \phi \rangle$ la stringa $\langle G, k \rangle$, dove G è il grafo non orientato definito come segue

- V ha $3 \times k$ vertici, i vertici sono divisi in k gruppi di tre nodi $t_1, \dots, t_k : t_j$ corrisponde alla clausola $(a_j \vee b_j \vee c_j)$ e ogni vertice in t_j corrisponde a un letterale in $(a_j \vee b_j \vee c_j)$

Quindi $V = \{a_1, b_1, c_1, \dots, a_k, b_k, c_k\}$

- Non ci sono archi fra vertici in una tupla t_j
- Non ci sono archi tra un vertice associato a un letterale x e i vertici associati al letterale $\bar{x}$
- Ogni altra coppia di vertici è connessa da un arco

La funzione f è calcolabile in tempo polinomiale

Per provare che f è una riduzione polinomiale di 3SAT a CLIQUE occorre dimostrare che ϕ è soddisfacibile se e solo se G ha una k -clique

Supponiamo che ϕ abbia una assegnamento di soddisfacibilità, quindi esiste almeno un letterale vero in ogni clausola

- Scegliamo un letterale vero in ogni clausola e consideriamo il sottografo G' indotto dai nodi corrispondenti ai letterali scelti, quindi G' è una k -clique, dato che due qualsiasi dei vertici non appartengono alla stessa clausola e non corrispondono a una coppia $x, \bar{x}$ dato che abbiamo preso letterali veri nell'assegnamento

Viceversa supponiamo che G abbia una k -clique G'

Poichè due nodi in una tripla non sono connessi da un arco, ognuna delle k triple contiene esattamente uno dei nodi della k -clique. Consideriamo l'assegnamento dei valori alle variabili di ϕ che rende veri i letterali corrispondenti ai nodi di G' . Ogni tripla contiene un nodo di G' e quindi ogni clausola contiene un letterale vero, quindi questo è un assegnamento di soddisfacibilità $\phi \in 3SAT$.

15.34 Definizione Linguaggio NP-Completo

Definizione:

Un linguaggio B è NP-Completo se soddisfa le seguenti due condizioni:

- B in NP
- Ogni A in NP è riducibile in tempo polinomiale a B

15.35 Teorema NP-Completo

Teorema:

SE B è NP-Completo e B è in P, allora $P = NP$

Dimostrazione:

Siccome B è in NP-Completo, per ogni $A \in NP$, risulta $A \leq_P B$

Ma abbiamo provato che se $A \leq_P B$ e $B \in P$, allora $A \in P$

Quindi $NP \subseteq P$ e siccome $P \subseteq NP$ risulta $P = NP$

15.36 Riducibilità Polinomiale e NP-Completezza

Se B è in NP-Completo e $B \leq_{\overline{P}} C$, con $C \in NP$, allora C è NP-Completo

Dimostrazione:

Per ipotesi

- $C \in NP$
- Per ogni $A \in NP$, $A \leq_{\overline{P}} B$
- $B \leq_{\overline{P}} C$

Allora utilizzando la proprietà transitiva di $\leq_{\overline{P}}$

- $C \in NP$
- Per ogni $A \in NP$, $A \leq C$

Quindi C è in NP-Completo

15.37 Strategia per Mostrare che B è NP-Completo

- Mostrare che $B \in NP$
- Scegliere un linguaggio A che sia NP-Completo
- Definire una riduzione di tempo polinomiale di A in B

15.38 Teorema di Cook-Levin

SAT è NP-Completo

Conseguenza: $SAT \in P$ se e solo se $P = NP$

La prova del teorema consiste nel mostrare che ogni $A \in NP$ è riducibile in tempo polinomiale a SAT, la riduzione di tempo polinomiale si ottiene definendo per ogni input w una formula booleana ϕ che simula la MdT non deterministica che decide A sull'input w

$SAT_{CNF} = \{\langle \phi \rangle \mid \phi \text{ è una formula booleana soddisfacibile in CNF}\}$

$$SAT_{CNF} \in NP$$

$$SAT \leq_P SAT_{CNF}$$

15.39 Teorema SAT_{CNF} è NP-Completo

SAT_{CNF} è NP-Completo

15.40 Teorema 3SAT è NP-Completo

3SAT è NP-Completo

Dimostrazione:

3SAT è in NP

Per provare che 3SAT è NP-Completo basta dimostrare che

$$SAT_{CNF} \leq_P 3SAT$$

La prova consiste nel costruire una formula booleana ψ in 3CNF tale che ϕ è soddisfacibile se e solo se ψ è soddisfacibile

Inoltre ψ può essere costruita in tempo polinomiale a partire da ϕ

15.41 CLIQUE è NP-Completo

Teorema:

CLIQUE è NP-Completo

Dimostrazione:

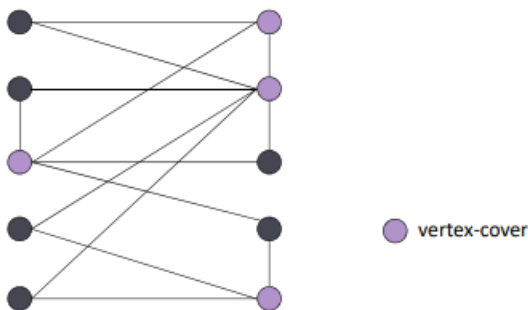
Sappiamo che CLIQUE $\in NP$

Inoltre 3SAT è NP-Completo e $3SAT \leq_P CLIQUE$

Quindi CLIQUE è NP-Completo

15.42 VERTEX-COVER

Un vertex cover V' di G , un grafo non orientato, è un sottoinsieme di V tale che per ogni $(u, v) \in E$ risulta $\{u, v\} \cap V' \neq \emptyset$, quindi V' copre ogni arco (u, v) in G



VERTEX-COVER = $\{\langle G, k \rangle \mid G \text{ è un grafo non orientato che ha un vertex cover di cardinalità } k$

Teorema:

VERTEX-COVER $\in NP$

Dimostrazione:

Un algoritmo V che verifica VERTEX-COVER in tempo polinomiale

$V = \text{Sull'input } \langle \langle G, k \rangle, c \rangle :$

- Verifica se c'è un insieme V' di k nodi di G , altrimenti rifiuta
- Verifica se V' copre ogni arco in G , accetta in caso affermativo, altrimenti rifiuta

$$\exists c : \langle \langle G, k \rangle, c \rangle \in L(V) \Leftrightarrow \text{VERTEX-COVER}$$

Teorema:

VERTEX-COVER è NP-Completo

Dimostrazione:

Abbiamo dimostrato che VERTEX-COVER è NP-Completo, ora dobbiamo dimostrare che

$$3\text{SAT} \stackrel{P}{\leq} \text{VERTEX-COVER}$$

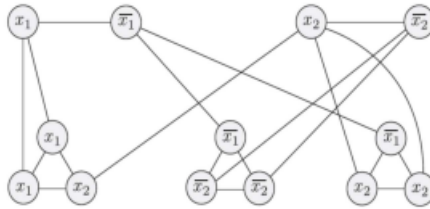


FIGURA 7.45

Il grafo che la riduzione produce a partire da

$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$